

초보자를 위한 BigQuery(SQL) 입문

카일스쿨



#0. 강의 Intro

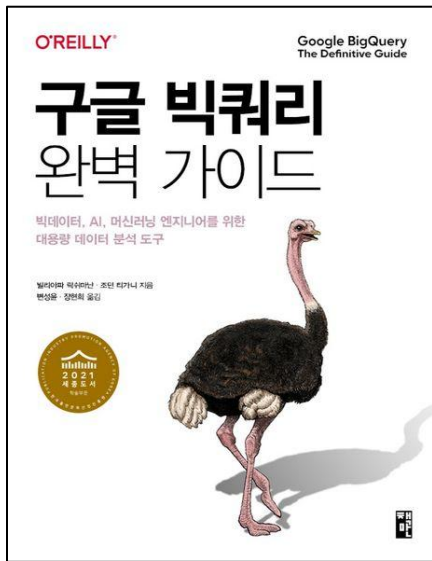
카일스쿨



강사 소개

카일스쿨

- 8년차 데이터 과학자
 - 데이터 분석, 데이터 엔지니어링, 머신러닝 엔지니어링, AI 모델 개발 경험
 - 인프런 : PM을 위한 데이터 리터러시
 - 요즘 하는 일 : 교육, 데이터팀 코칭
 - 유튜브 : 카일스쿨 운영
 - 인스타그램 : @data.scientist
-
- BigQuery 2017년부터 사용
 - 구글 빅쿼리 완벽 가이드 역자



강사 소개

직접 제작한 웹페이지

<https://zzsza.github.io/bigquery/>

 BigQuery Guide Book - 빅쿼리 가이드북

[Home](#) [데이터 리터러시 온라인 강의](#) [블로그](#)

페이지 이용 가이드

BigQuery 환경 설정

BigQuery 기초

SELECT

WHERE

GROUP BY

HAVING

★ 쿼리 작성 가이드 ★

BigQuery 중급

시간 함수

문자열 함수

조건식

JOIN

WITH

서브 쿼리

BigQuery 고급

윈도우 함수

자료형

ARRAY

STRUCT

DML

소개

- 해당 자료는 구글 클라우드 플랫폼의 BigQuery Guide Book입니다
- SQL(BigQuery)를 공부하시는 분들에게 도움이 될 수 있도록 자료를 만들었습니다

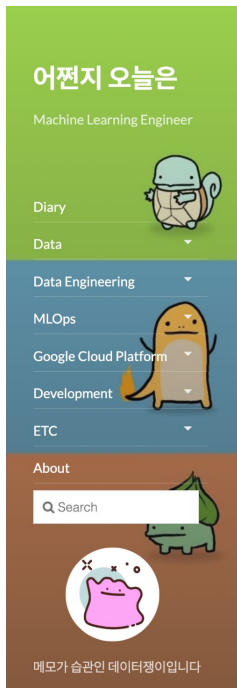
배우는 내용

- 페이지 이용 가이드
 - SQL 공부하는 목표 설정
- BigQuery 설정하기
 - BigQuery의 장점
 - BigQuery UI
- BigQuery 기초
 - SELECT
 - WHERE
 - GROUP BY
 - HAVING
 - LIMIT
 - LIKE
- BigQuery 중급
 - 시간 함수
 - 문자열 함수
 - 조건문(CASE WHEN, IF)
 - JOIN
- BigQuery 고급
 - 윈도우 함수
 - 자료형
 - UDF



강사 소개

Google Analytics, Firebase 데이터를 사용하기 위해 많은 분들이 찾아주시는 글
<https://zzsza.github.io/gcp/2020/04/12/bigquery-unnest-array-struct/>



BigQuery UNNEST, ARRAY, STRUCT 사용 방법

12 Apr 2020 in [Google Cloud Platform](#) on [BigQuery](#)

- BigQuery Unnest, Array, Struct 사용 방법에 대해 작성한 글입니다

• 목차

- 들어가며
- BigQuery ARRAY
- BigQuery STRUCT
- BigQuery UNNEST
- 응용
- 정리

들어가며

- BigQuery는 SQL 문법을 사용하고 있기 때문에, 많은 사람들이 처음에 쉽게 접할 수 있음
 - 그러나 자주보기 힘든 ARRAY, STRUCT, UNNEST를 만나면 많은 사람들이 어려워함
 - 특히 Google Analytics 데이터나 Firebase 데이터 작업시 Cannot access field name on a value with type ARRAY 이런 Error를 접함
 - 자료형들에 대한 설명과 이런 데이터를 쿼리할 때 어떻게 사용해야 하는지에 대해 정리하려고 함
- 참고로, BigQuery는 비정규화되었을 때 성능이 가장 뛰어나
 - 비정규화를 위해 중첩(Nested) 및 반복(Repeated) 열을 사용함
 - 중첩된 레코드를 사용할 때 장점
 - 빅쿼리 퍼포먼스 개선

강사 소개

PM을 위한 데이터 리터러시 강의 : <https://inf.run/jfWT>



Data Literacy For PM

6개 무료 보기

BEST 비즈니스 · 마케팅 > 기획 · 전략 · PM

PM을 위한 데이터 리터러시(프로덕트 데이터 분석)

★★★★★ (5) 36개의 수강평 · 1,282명의 수강생

👤 카일스쿨 👑

데이터 리터러시 metric hierarchy AB test

강의 수강 목표

사람마다 다른 수강 목표

- 이 강의를 듣게 된 계기
- 이 강의를 듣는 이유
- 이 강의를 듣고 **내가 어떻게 변하고 싶은지**
- 내가 기대하는 변화 후의 모습 정의

강의 수강 목표 예시

- 강의 수강 후, 회사에 저장된 데이터를 자유롭게 추출하고 싶다
- 강의 수강 후, 회사에서 주요 지표를 추출하고 싶다
- 강의 수강 후, 데이터에서 의미를 파악하고 싶다

작성 후 인프런 게시판에 공유해보기!

어떤 사람이 들으면 좋은가요?

- 구글 클라우드의 데이터 웨어하우스인 BigQuery에 대해 학습이 필요하신 분
- SQL을 공부하고 싶으신 분
- 데이터 분석을 위한 SQL을 알고 싶은 분
- 이제 막 시작하는 분들을 대상으로 만든 강의

상황별 추천 수강 방법

- 1) 초심자(Lv.1) : 아예 처음
- 2) 초심자(Lv.2) : 처음은 아니지만 익숙하지 않음
- 3) 숙련자(Lv.??) : 누군가를 가르쳐야 하는 분

상황별 추천 수강 방법 : 초심자(Lv.1)

아예 SQL을 처음 접한다면

- 일단 실행해보고 => 쿼리문을 완성하는 것에 초점을 맞추기
- 문법을 하나씩 암기할 필요가 없으며, 쿼리 작성에 익숙해지는 것이 목표
- 회사에서 데이터로 추출해야 하는 것이 있다면, 그것을 추출하는 방법을 고민하며 수강
- 문제를 꼭 풀고 다음 강의로 넘어가기
- 매일 1시간씩 공부한다면 약 1주~2주 안에 1회독
 - 그 후에 2회독(1.3-5배속)

상황별 추천 수강 방법 : 초심자(Lv.2)

- 간단하게 SELECT는 해보신 분
- 익숙해지면서, 나만의 SQL 작성 치트시트 만들기(템플릿)
 - 어떤 것을 해야 할 때 어떻게 해야할까? => 쿼리 모음, 생각의 흐름 정리
 - 일종의 오답 노트
- 회사에서 풀어야 하는 문제가 있다면 그 문제를 생각하며 수강하기
- 매일 1시간씩 공부한다면 약 1주~2주 안에 1회독 => 문제 다시 풀기

상황별 추천 수강 방법 : 숙련자(Lv.??)

- 숙련자는 1.5배속으로 수강하면서 기존에 알던 지식과 다른 관점 비교하며 학습
- 데이터 결과 검증, 쿼리 작성하는 흐름, 생산성 도구, 디버깅 파트 등은 기존에 있는 SQL 강의에 없는 내용일테니 파악하며 업무에 어떻게 적용할까? 고민해보기
- 회사의 구성원들에게 SQL을 공부하게 해야한다면 연습 문제 출제하기
- 회사에 재직 중이라면 동료들과 같이 이 강의를 수강하고 모르는 내용에 대한 답을 해주면서 데이터 문화 만들기
- 다음 강의를 기다려주세요..!

상황별 추천 수강 방법

- 혼자 학습보다 같이 진행하기
- 강의를 완강하는 비율이 생각보다 낮음
- 혼자 시간을 내는 것보다, 같이 시간을 내서 학습해보는 것을 추천
- 그 과정에서 모르는 것을 물어보고, 답을 찾는 과정을 함께 진행하기

SQL을 얼마나 알아야 할까?

- 데이터베이스에 대한 이해
- SELECT
- FROM
- WHERE
- GROUP BY
- JOIN

이정도만 파악한 후, 그 후엔 실제 문제를 풀면서 필요한 문법을 찾는 방법
(=실용적인 야생 학습)

강의를 수강한 후, 얻을 수 있는 것

- BigQuery에서 쿼리문을 실행할 수 있음
- BigQuery SQL 기본에 대한 이해
- SQL 쿼리를 작성하기 위한 기본 연습

강의의 특징

- 제 목표
 - SQL을 전혀 모르는 사람들이 시작할 수 있도록 돕는 것
 - 회사에서 데이터 분석가가 SQL 기본 교육을 하지 않고, 이 강의로 대체하는 것
- 실용적인 관점에서 강의를 진행
 - 오류가 발견되었을 때 어떻게 고칠 것인가?
 - 데이터 결과 검증을 어떻게 할 것인가?
 - 쿼리를 잘 작성할 수 있는 도구는 무엇이 있을까?
 - 문제 풀이도 같이 진행(총 47개의 문제)
- 구조화된 시각화 자료 제공
- 제가 어떤 생각으로 쿼리를 작성하는지 흐름을 공유

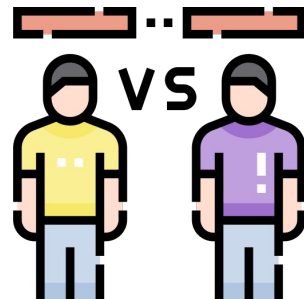
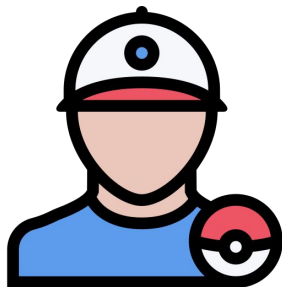
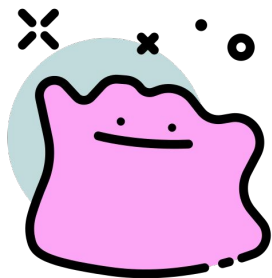
사용할 데이터셋

포켓몬 데이터

(자체 제작)

- 트레이너 데이터
- 트레이너의 포켓몬 보유 데이터
- 배틀 데이터

이커머스 구매 데이터와 동일한 형태(데이터만 다를 뿐)



시작하기 전에 알고 있으면 좋은 개념

함수 : 프로그램 소스 코드에서 일정한 동작을 수행하는 코드들

시작하기 전에 알고 있으면 좋은 개념

함수 : 프로그램 소스 코드에서 일정한 동작을 수행하는 코드들

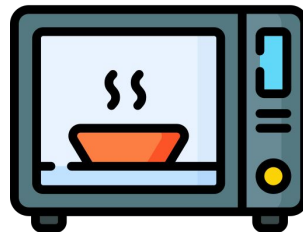
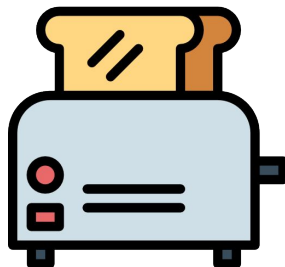
함수 : 주방에서 사용하는 **가전 제품**과 유사

빵을 데우고 싶은 경우

Input : 빵

토스트기

Output : 따뜻한 빵



시작하기 전에 알고 있으면 좋은 개념

함수 : 프로그램 소스 코드에서 일정한 동작을 수행하는 코드들

스프레드시트, 엑셀의 함수

C3

▼

fx

=IF(1=1, TRUE, FALSE)

	A	B	C	D
1				
2				
3			TRUE	
4				

시작하기 전에 알고 있으면 좋은 개념

BigQuery에서도 여러 함수를 사용해 미리 정의된 기능을 수행

예 : 시간 데이터 2024-01-08 13:00:03에서 2024-01-08 추출

#1. BigQuery 기초 지식

카일스쿨



이번 파트에서 다룰 내용

- 데이터 관련 기초 지식(OLTP, SQL, Row, Column, 저장 형태 등)
- BigQuery 관련 기초 지식(왜 사용하는가?, 장점, 비용)
- BigQuery 환경 설정

BigQuery의 기초 지식

데이터 저장 형태

- 데이터는 보통 데이터베이스 테이블 등에 저장
 - Database : 데이터의 저장소
 - Table : 데이터가 저장된 공간
 - 저장된 데이터를 제품(앱, 웹)에서 사용
- 예시 : 음식 배달 서비스의 데이터 - 유저, 음식점, 음식, 라이더 등

데이터가 저장되는 장소

MySQL, Oracle, PostgreSQL 같은 데이터베이스에 주로 저장
(AWS Aurora 등)



MySQL, Oracle, PostgreSQL 같은 데이터베이스의 특징 => OLTP

- OLTP(Online Transaction Processing)
- 거래를 하기 위해 사용되는 데이터베이스
 - 보류, 중간 상태가 없음 - 주문을 완료하거나 안하거나 : 데이터가 무결하다
 - 데이터의 추가(INSERT), 데이터의 변경(UPDATE)이 많이 발생



MySQL, Oracle, PostgreSQL 같은 데이터베이스의 특징 => OLTP

- OLTP(Online Transaction Processing)
- 거래를 하기 위해 사용되는 데이터베이스
 - 보류, 중간 상태가 없음 - 주문을 완료하거나 안하거나 : 데이터가 무결하다
 - 데이터의 추가(INSERT), 데이터의 변경(UPDATE)이 많이 발생
- SQL을 사용해 데이터를 추출할 수 있으나
분석을 위해 만든 데이터베이스가 아니라 쿼리 속도가 느릴 수 있음



SQL(Structured Query Language)

SQL : 데이터베이스에서 데이터를 가지고 올 때 사용하는 언어
데이터베이스의 데이터를 관리하기 위해 설계된 특수 목적의 프로그래밍 언어

쿼리문, 쿼리 구문, 쿼리를 짠다, SQL 쿼리 등으로 표현



테이블에 저장된 데이터의 형태

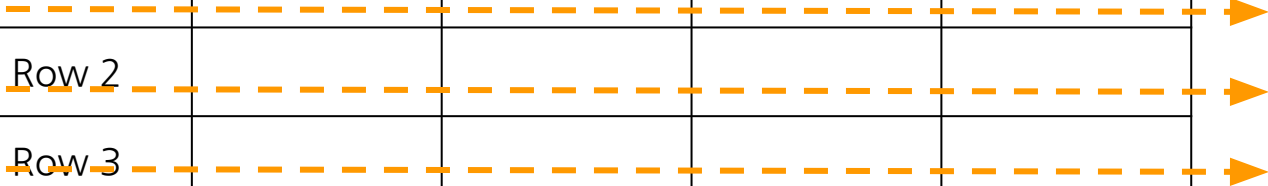
테이블 구조

	Column 1	Column 2	Column 3	Column 4
Row 1				
Row 2				
Row 3				

테이블에 저장된 데이터의 형태

테이블 구조

	Column 1	Column 2	Column 3	Column 4
Row 1				
Row 2				
Row 3				



행(Row) →, 새로운 row는 가로로 한 줄을 의미

- 하나의 row가 하나의 고유한 데이터
- 예 : 거래 History에서 하나의 row가 거래

(주의) Raw 데이터 : 원본 데이터

테이블에 저장된 데이터의 형태

테이블 구조

	Column 1	Column 2	Column 3	Column 4
Row 1				
Row 2				
Row 3				

열(Column) ↓, column은 원형 기둥이란 뜻. 세로로 연상

- 각 데이터의 특정 속성 값
- 거래의 구매 시간, 거래의 구매자

테이블에 저장된 데이터의 형태

테이블에 저장된 형태 = 엑셀, 스프레드시트와 유사

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	id	kor_name	eng_name	type1	type2	total	hp	attack	defense	special_attack	special_defense	speed	generation	is_legendary
2	1	이상해씨	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	FALSE
3	2	이상해풀	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	FALSE
4	3	이상해꽃	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	FALSE
5	4	파이리	Charmander	Fire		309	39	52	43	60	50	65	1	FALSE
6	5	리자드	Charmeleon	Fire		405	58	64	58	80	65	80	1	FALSE
7	6	리자몽	Charizard	Fire	Flying	534	78	84	78	109	85	100	1	FALSE
8	7	꼬부기	Squirtle	Water		314	44	48	65	50	64	43	1	FALSE
9	8	어니부기	Wartortle	Water		405	59	63	80	65	80	58	1	FALSE
10	9	거북왕	Blastoise	Water		530	79	83	100	85	105	78	1	FALSE

OLAP와 데이터 웨어하우스(DW)의 등장

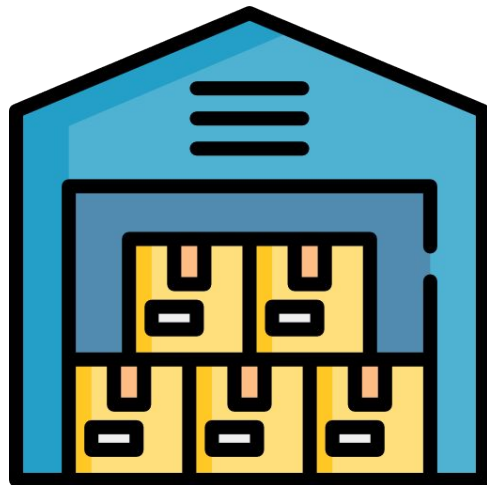
OLTP로 데이터 분석을 하다가 속도, 기능 부족의 이슈로 OLAP가 등장함

OLAP : Online Analytical Processing

- 분석을 위한 기능 제공

데이터 웨어하우스

- 데이터를 한 곳에 모아서 저장
- 여러 곳에 저장된 데이터 예시
 - Database, 웹(크롤링), 파일, API의 결과 등



BigQuery 소개

Google Cloud의 OLAP + Data Warehouse

"Google Cloud의 데이터 웨어하우스"



BigQuery

BigQuery 장점

1. 난이도

SQL를 사용해 쉽게 데이터 추출 가능

2. 속도

OLAP 도구이므로 속도가 빠름(단, 그만큼 돈을 지불)

3. Firebase, Google Analytics 4의 데이터를 쉽게 추출할 수 있음

사용 기기, 위치(시 단위까지 표현), OS 버전, 이벤트 행동 획득 가능(별도의 로깅 필요)

4. 데이터 웨어하우스를 사용하기 위해 서버(컴퓨터)를 띄울 필요 없음

구글에서 인프라를 관리함

BigQuery를 사용하는 이유

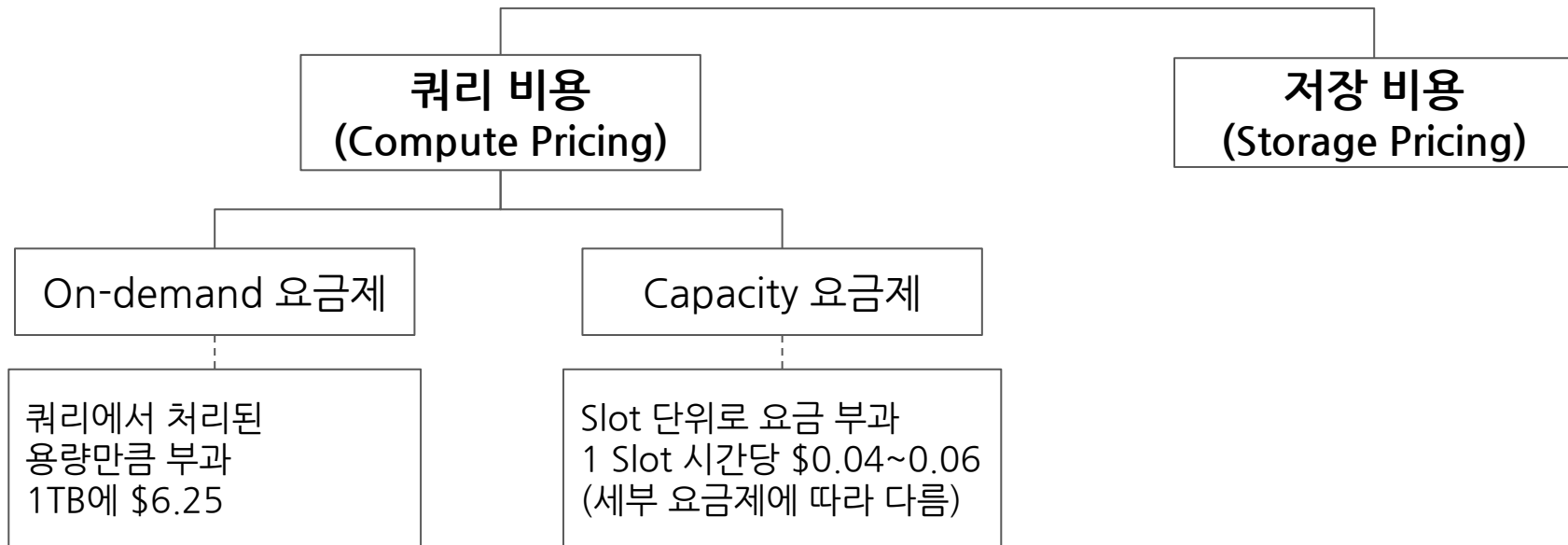
- 회사에서 앱이나 웹에서 Firebase, Google Analytics 4를 활용할 경우
- 운영을 적은 비용(인력 등)으로 진행하기 위해
- 사용하는 회사 : 쏘카, 당근, 쿠팡, 하이퍼커넥트, 마이리얼트립, 버드뷰, 원티드랩, 매스프레소, 오피지지



BigQuery 비용(US 기준)



BigQuery 비용(US 기준)



BigQuery 비용(US 기준)

쿼리 비용 (Compute Pricing)

On-demand 요금제

쿼리에서 처리된
용량만큼 부과
1TB에 \$6.25

Capacity 요금제

Slot 단위로 요금 부과
1 Slot 시간당 \$0.04~0.06
(세부 요금제에 따라 다름)

저장 비용 (Storage Pricing)

Active Logical 저장소: 1G에 \$0.02(월)
Long-term Logical 저장소: 1G에
\$0.01(월), 90일간 수정되지 않는 경우

강의 준비

- Google Cloud 설정

카일스쿨

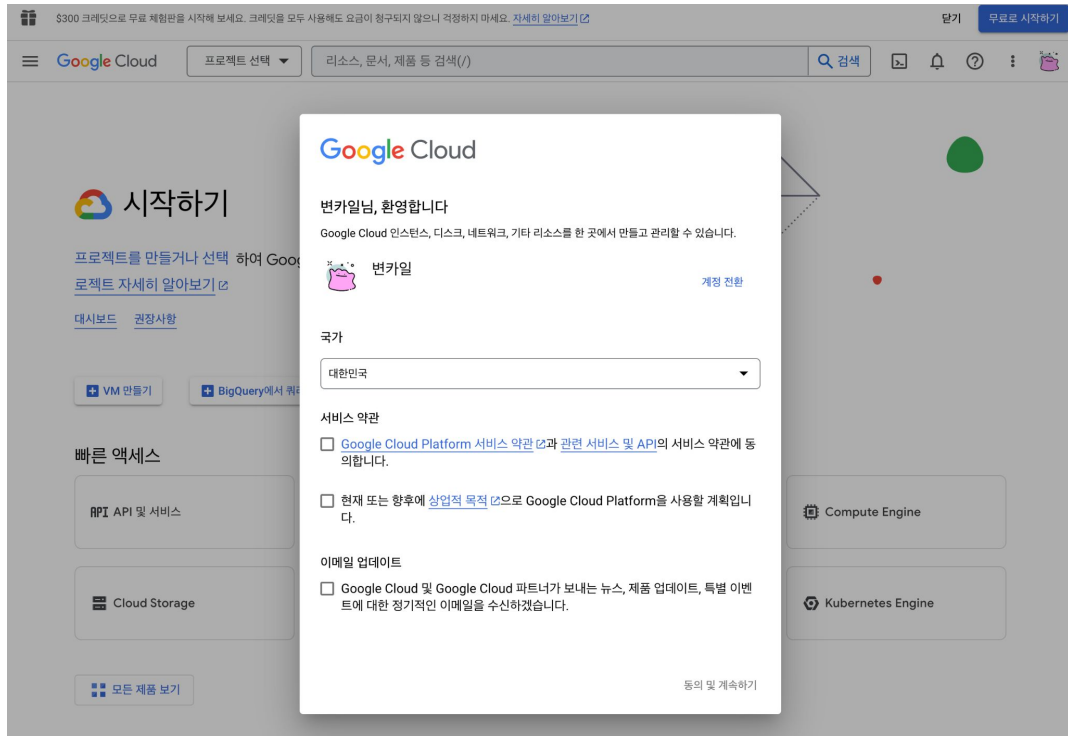


Google Cloud 설정

- Google Cloud 가입
- Google Cloud 프로젝트 생성
- Google Cloud BigQuery Activate

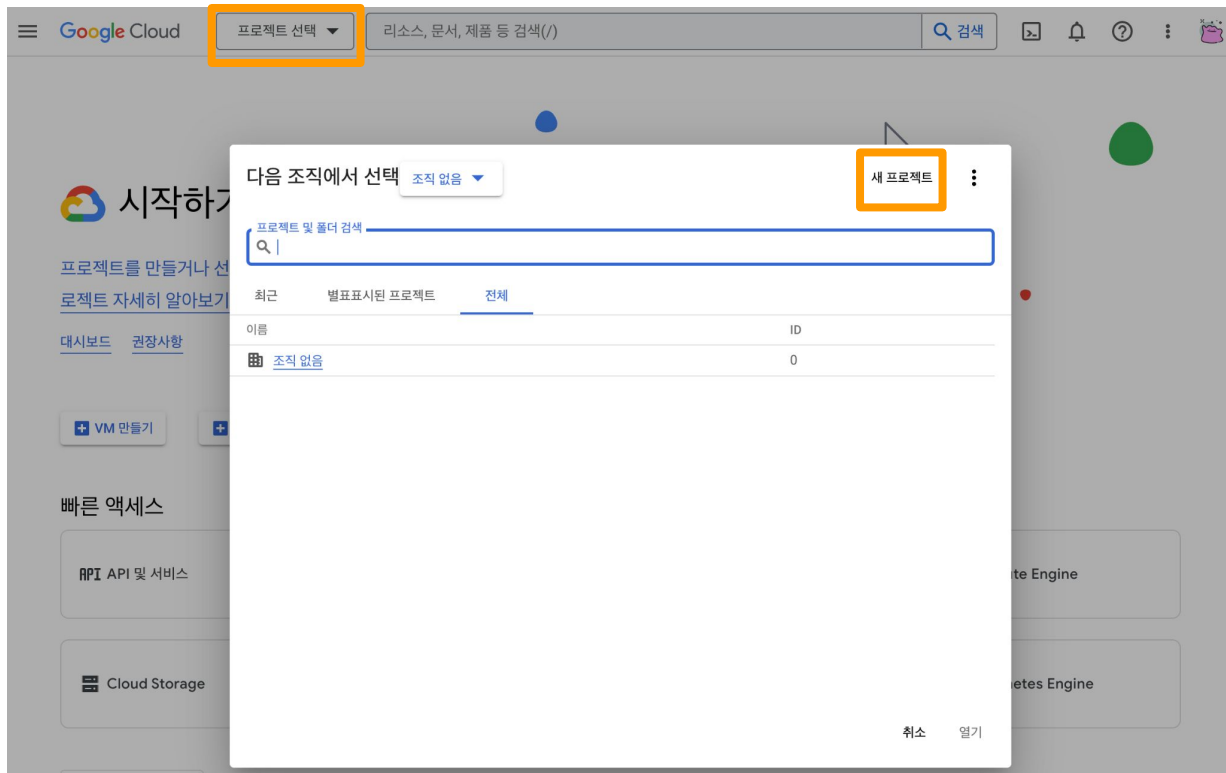
Google Cloud 설정

- 구글 이메일로 로그인 후, Google Cloud Console로 이동
- <https://console.cloud.google.com/>



Google Cloud 설정

프로젝트 선택 클릭 - 새 프로젝트 클릭



Google Cloud 설정 - 프로젝트 이름 설정

- 프로젝트 이름 : 여러분이 원하시는 이름 설정. 저는 Inflearn-BigQuery
- 동일한 프로젝트 ID를 가질 수는 없음

새 프로젝트

프로젝트 이름 *

Inflearn-BigQuery



프로젝트 ID: inflearn-bigquery입니다. 나중에 변경할 수 없습니다. [수정](#)

조직 *

조직 없음



프로젝트에 연결할 조직을 선택하세요. 선택한 후에는 변경할 수 없습니다.

위치 *

[찾아보기](#)

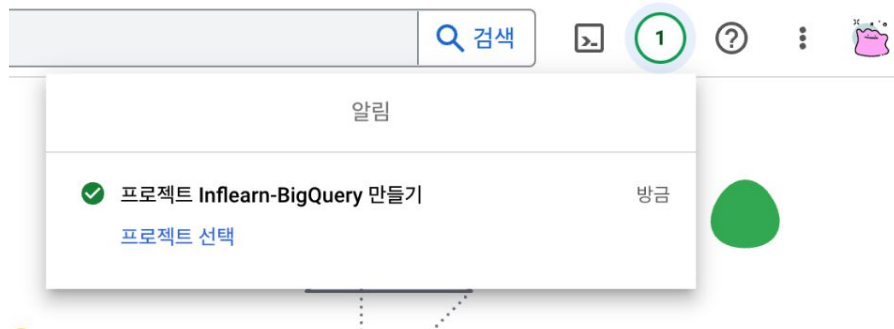
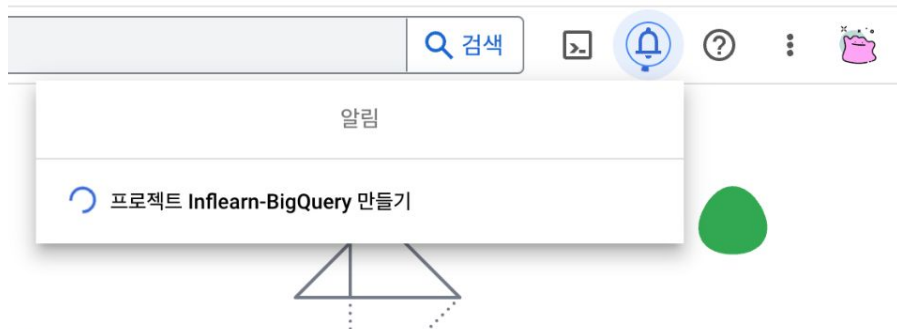
상위 조직 또는 폴더

만들기

취소

Google Cloud 설정 - 프로젝트 생성

프로젝트 만들기 몇분 후에 만들었다고 말해줌



Google Cloud 설정 - 체험판 시작

300 크레딧을 받기 위해 무료로 시작하기 클릭



\$300 크레딧으로 무료 체험판을 시작해 보세요. 크레딧을 모두 사용해도 요금이 청구되지 않으니 걱정하지 마세요. [자세히 알아보기](#)

닫기

무료로 시작하기



Google Cloud



조직 없음 ▼

리소스, 문서, 제품 등 검색(/)



검색



1



Google Cloud 설정 - 체험판 시작

약관 체크 후 계속 클릭

 Google Cloud 무료로 사용해 보기

총 2단계 중 1단계 계정 정보



변카일

kyle@kyleschool.kr

[계정 전환](#)

국가

대한민국

조직 또는 니즈를 가장 잘 나타내는 설명은 무엇인가요?

Please select

서비스 약관

☐ [Google Cloud Platform 서비스 약관](#) [추가 무료 체험판 서비스 약관](#) [관련 서비스 및 API](#)의 서비스 약관을 읽었으며 이에 동의합니다.

계속 진행하려면 체크박스를 선택하세요.

귀하는 '계속'을 클릭함으로써, Google이 (i) 클라우드 서비스를 제공, 유지 및 개선하고, 귀하의 요청을 처리하기 위한 목적에 따라 (ii) 해당 목적상 필요한 기간, 서비스 기간 또는 법정 보유기간 동안, (iii) 귀하의 국가, 주소, 성명, 생년월일, 성별, CI (연계정보), 전화번호, 신용카드 정보, 결제기록, 구성 및 설정, 서비스 이용 및 기록, 쿠키 및 자동 생성 정보, 커뮤니케이션 이력 등 개인정보(이성 필수항목)를 수집 및 이용할 것이라는 점에 동의합니다(개인정보 처리에 관한 보다 자세한 정보는 [Google Cloud 개인정보처리방침](#) 참고).

계속

모든 Cloud Platform 제품에 액세스

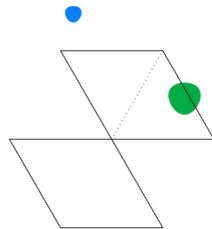
Firebase, Google Maps API 등을 포함해 앱, 웹사이트, 서비스를 구축하고 실행하는 데 필요한 모든 기능을 이용할 수 있습니다.

\$300의 무료 크레딧

90일간 사용할 수 있는 \$300 크레딧으로 Google Cloud를 실제 작업에 활용할 수 있습니다.

무료 체험판 종료 후 자동 청구되지 않음

신용카드를 요청하는 이유는 자동 가입을 방지하기 위해서입니다. 신용카드나 체크카드를 사용하는 경우 유료 계정으로 직접 업그레이드하지 않으면 요금이 청구되지 않습니다.



Google Cloud 설정 - 체험판 시작

결제 정보 등록(신용카드/체크카드)

무료 평가판 시작하기 클릭

총 2단계 중 2단계 결제 정보 확인

사용자의 결제 정보는 사기와 악용을 줄이는 데 도움이 됩니다. 신용카드나 체크카드를 사용하는 경우 자동 결제를 사용 설정하지 않으면 요금이 청구되지 않습니다.

고객 정보

 계좌 유형 ⓘ 

개인

 세금 정보 ⓘ

사업자 유형: 개인

 이름 및 주소 ⓘ

도/시

시/군/구

주소

주소 입력란 1

주소 입력란 2

Google Cloud 설정 - 체험판 시작

90일간 사용할 수 있는 300 크레딧 획득



변카일님, 환영합니다.

무료 체험판에는 90일간 사용할 수 있는 \$300 크레딧이 포함되어 있습니다. 보다 나은 서비스를 제공할 수 있도록 4가지 질문에 대해 주세요.



조직 또는 니즈를 가장 잘 설명하는 것이 무엇인가요?



Google Cloud를 방문하신 이유가 무엇인가요?

답변 선택 *

특정 제품, 서비스, API 사용



다음



Google Cloud로 어떤 작업을 하고 싶으신가요?



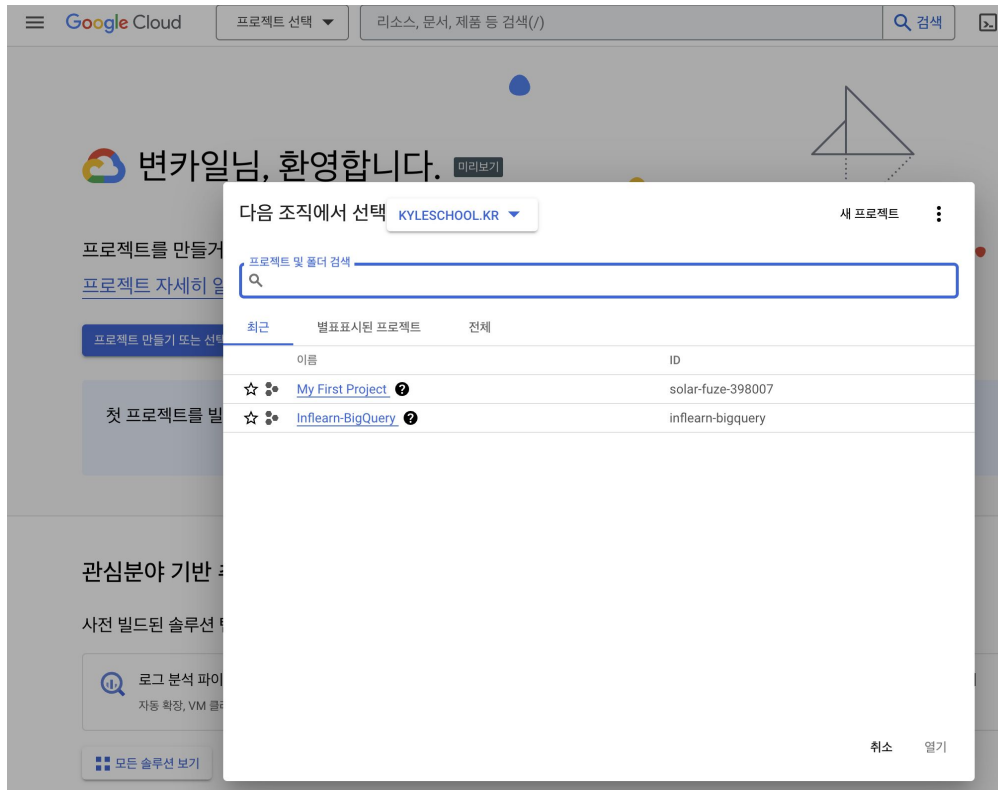
담당하는 역할을 가장 잘 설명하는 것이 무엇인가요?

닫기

완료

Google Cloud 설정 - 프로젝트 선택

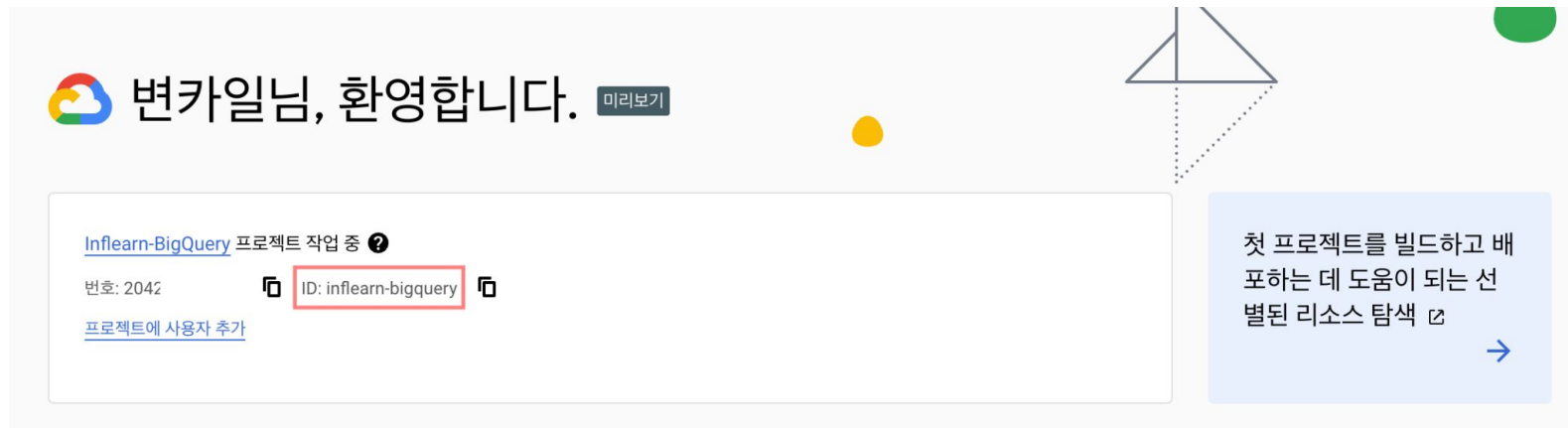
만약 프로젝트 선택에 여러가지가 보이면 아까 만들었던 것으로 선택



Google Cloud 설정 - 프로젝트 ID 확인

<https://console.cloud.google.com/welcome/new>

ID라고 되어있는 것 기억하기(숫자가 붙어있을 수 있음)



The screenshot shows the Google Cloud 'Welcome' page for a new project. At the top, it says '변카일님, 환영합니다.' (Hello, Bunkailnim) with a '미리보기' (Preview) button. Below this, the project name 'Inflearn-BigQuery' is shown with a help icon. The project number '2042' is displayed next to the project ID 'inflearn-bigquery', which is highlighted with a red box. A link '프로젝트에 사용자 추가' (Add user to project) is visible. On the right, a blue box contains instructions: '첫 프로젝트를 빌드하고 배포하는 데 도움이 되는 선별된 리소스 탐색' (Explore curated resources that help you build and deploy your first project), followed by a right-pointing arrow.

변카일님, 환영합니다. [미리보기](#)

[Inflearn-BigQuery](#) 프로젝트 작업 중 ?

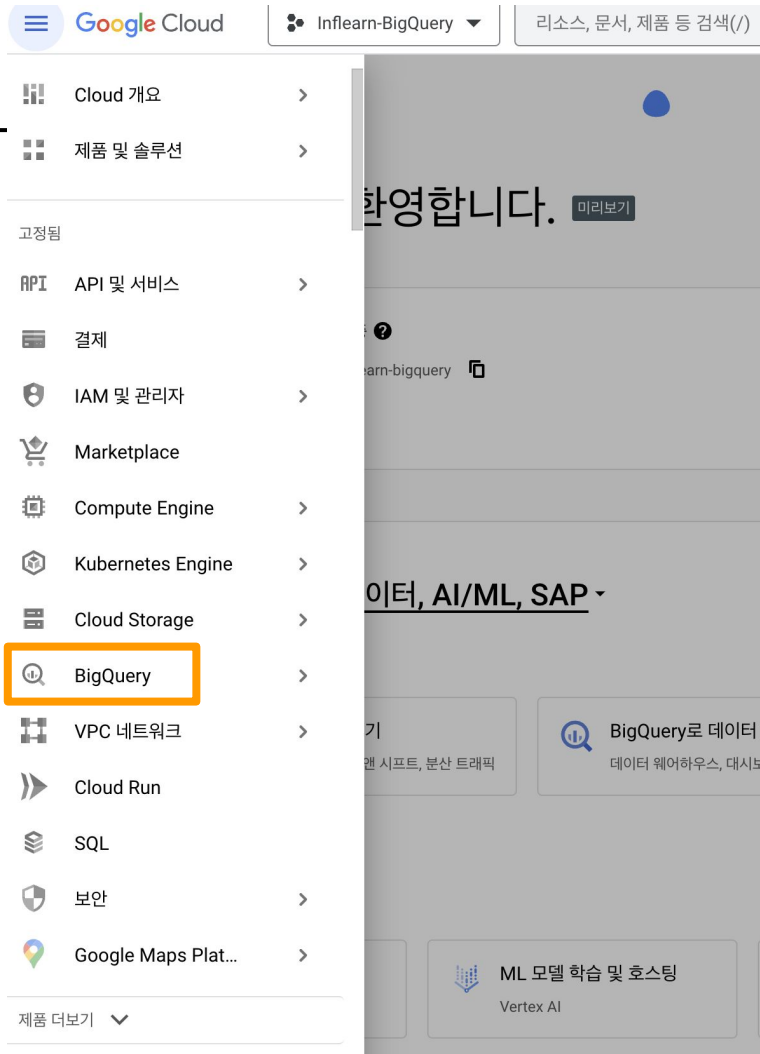
번호: 2042 ID: inflearn-bigquery

[프로젝트에 사용자 추가](#)

첫 프로젝트를 빌드하고 배포하는 데 도움이 되는 선별된 리소스 탐색 →

BigQuery 설정

- 좌측 최상단 버튼 클릭
- BigQuery 클릭



BigQuery 설정

- 샌드박스 문구가 나오면 업그레이드 클릭

BigQuery

샌드박스 완전형 BigQuery 환경으로 업그레이드하려면 결제를 설정하세요. [자세히 알아보기](#)

닫기 업그레이드

분석

SQL 작업공간

데이터 전송

예약된 쿼리

Analytics Hub

Dataform

파트너 센터

마이그레이션

평가

SQL 변환

관리

모니터링

용량 관리

BI Engine

정책 태그

탐색기

+ 추가

Q 입력하여 검색

작업공간 리소스를 보는 중입니다.
[별표표시된 리소스만 표시](#)

inlearn-bigquery

☆

SQL Workspace에 오신 것을 환영합니다

시작하기

+ 새 쿼리 작성 + 추가

샘플 데이터로 사용해 보기

Google 트렌드 데모 쿼리 사용해 보기

이 간단한 쿼리는 Google 트렌드 공개 데이터 세트의 미국 내 인기 검색어를 생성합니다.

이 쿼리 열기 데이터 세트 보기

자체 데이터 추가

문서 파일

로컬 파일 업로드

가이드 시작하기

Google Drive

Google 스토리지 서비스

가이드 시작하기

BigQuery 설정

- 앞서 설정한 결제 계정 추가하면 끝

'Infllearn-BigQuery' 프로젝트의 결제 계정 설정

이 프로젝트를 지원할 **Google Cloud Platform** 결제 계정을 선택하세요. 일부 결제 계정은 사용하지 못할 수 있습니다.[자세히 알아보기](#)

결제 계정 *

≡ 필터 | 입력하여 필터링

kyleschool.kr

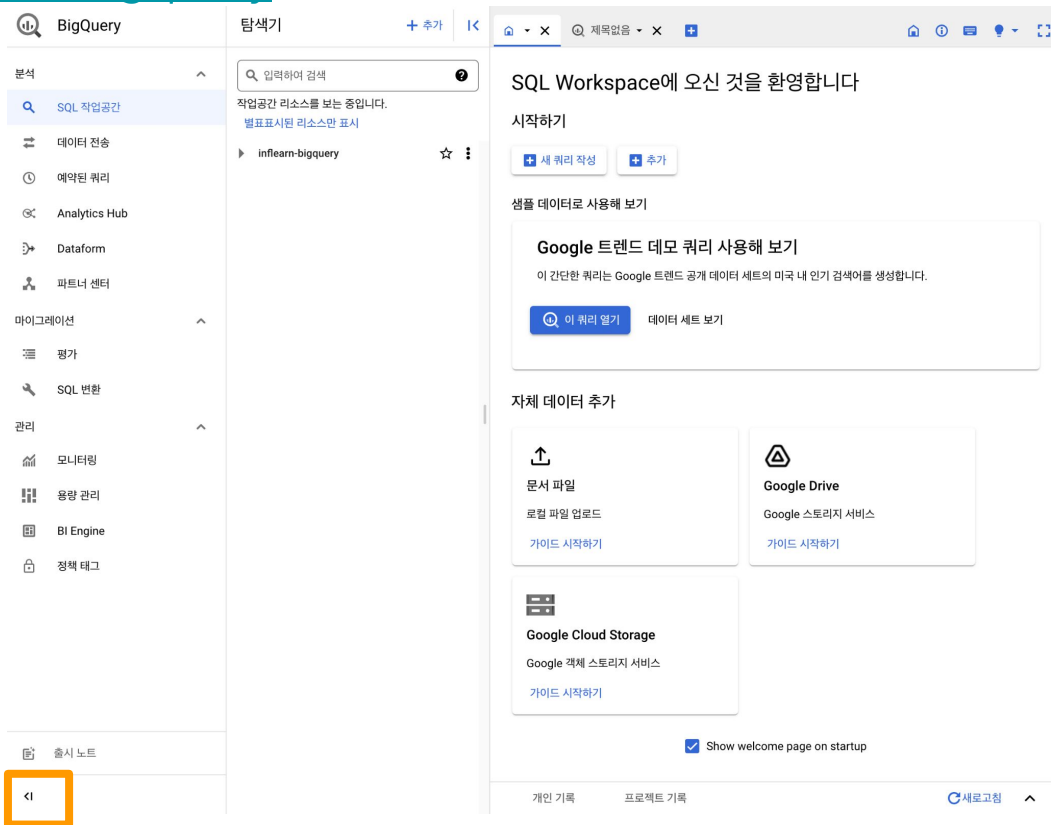
내 결제 계정

취소 계정 설정

BigQuery 메인 화면

<https://console.cloud.google.com/bigquery>

- 대부분 SQL 작업 공간에서 진행
- 아래 버튼 클릭하면 숨김



BigQuery의 환경 구성 요소

1) 프로젝트(Project)

2) 데이터셋(Dataset)

3) 테이블(Table)

BigQuery의 환경 구성 요소



1) 프로젝트(Project)

- 하나의 큰 건물. 건물 안에는 여러 목적을 가진 창고가 존재
- 하나의 프로젝트에 여러 데이터셋이 존재할 수 있음

2) 데이터셋(Dataset)

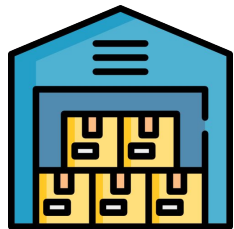
3) 테이블(Table)

BigQuery의 환경 구성 요소



1) 프로젝트(Project)

- 하나의 큰 건물. 건물 안에는 여러 목적을 가진 창고가 존재
- 하나의 프로젝트에 여러 데이터셋이 존재할 수 있음



2) 데이터셋(Dataset)

- **프로젝트에 있는 창고**. 각 창고 공간에 데이터를 저장
- 판매 데이터, 고객 데이터 등 별도의 데이터를 저장할 수 있음
- 하나의 데이터셋에 다양한 테이블이 존재할 수 있음

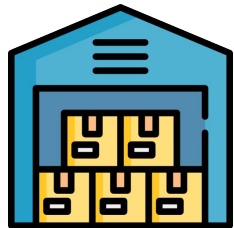
3) 테이블(Table)

BigQuery의 환경 구성 요소



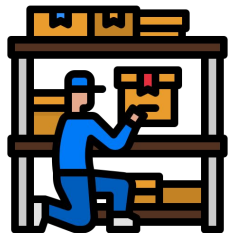
1) 프로젝트(Project)

- 하나의 큰 건물. 건물 안에는 여러 목적을 가진 창고가 존재
- 하나의 프로젝트에 여러 데이터셋이 존재할 수 있음



2) 데이터셋(Dataset)

- 프로젝트에 있는 창고. 각 창고 공간에 데이터를 저장
- 판매 데이터, 고객 데이터 등 별도의 데이터를 저장할 수 있음
- 하나의 데이터셋에 다양한 테이블이 존재할 수 있음

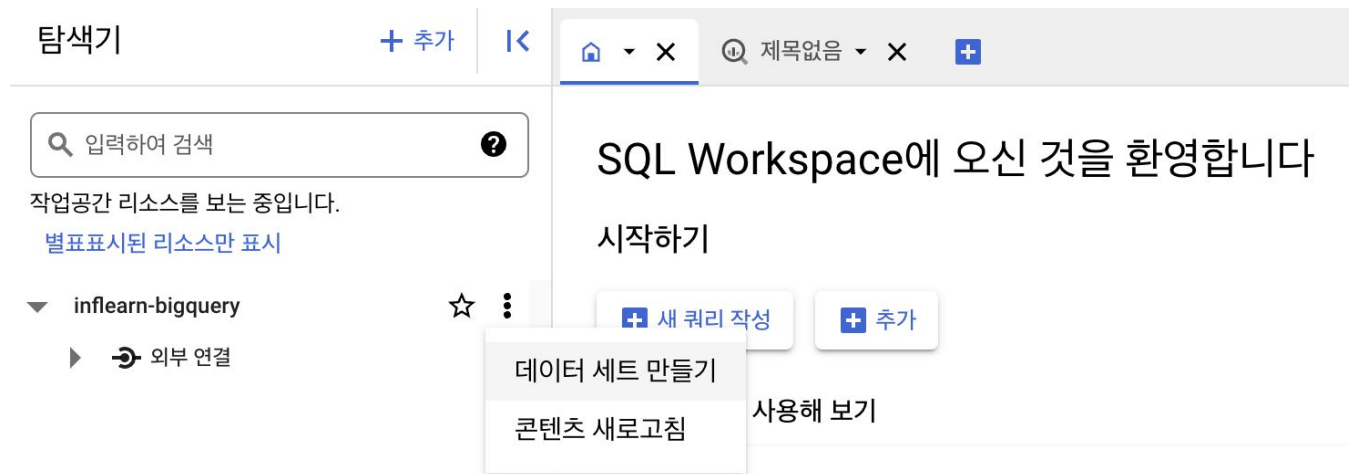


3) 테이블(Table)

- 창고에 있는 선반
- 테이블 안엔 상품의 세부 정보가 저장
- 테이블 안엔 행과 열로 이루어진 데이터들이 저장

Google Cloud 설정 - 데이터셋 생성

- 데이터셋 만들기 클릭



Google Cloud 설정 - 데이터셋 생성

- 데이터셋 만들기 클릭
- 데이터셋 ID : basic

데이터셋 만들기

프로젝트 ID
inflern-bigquery

[변경](#)

데이터셋 ID *

basic

문자, 숫자, 밑줄이 허용됩니다.

위치 유형 ?

☐ 리전

단일 리전 내에서 짧은 지연 시간

☒ 멀티 리전

더 높은 할당량 한도를 달성하기 위해 BigQuery에서 그룹 내의 리전을 선택하도록 허용합니다.

멀티 리전 *

US (미국의 멀티 리전)

기본 테이블 만료

☐ 테이블 만료 사용 설정 ?

기본 최대 테이블 기간

Days

고급 옵션



데이터셋 만들기

[취소](#)

Google Cloud 설정 - 데이터셋 생성

- 이제 basic이 보임

탐색기

+ 추가

<

입력하여 검색

?

작업공간 리소스를 보는 중입니다.

[별표표시된 리소스만 표시](#)

▼ inflearn-bigquery

☆ ⋮

▶ 외부 연결

☆ ⋮

basic

☆ ⋮

제목없음

+

SQL Workspace에 오신 것을 환영합니다

시작하기

+ 새 쿼리 작성

+ 추가

샘플 데이터로 사용해 보기



Google 트렌드 데모 쿼리 사용해 보기

이 간단한 쿼리는 Google 트렌드 공개 데이터 세트의 미국 내 인기 검색어를 생성합니다.

이 쿼리 열기

데이터 세트 보기

Google Cloud 설정 - 테이블 생성

- 수업 자료(첨부 파일)에 있는 데이터 다운로드

 학습 알림

 수업 자료

 수강평 쓰기

Google Cloud 설정 - 테이블 생성

- 데이터셋의 ... 클릭 후 테이블 만들기
- 총 4개의 테이블 생성 예정

?

작업공간 리소스를 보는 중입니다.
[별표표시된 리소스만 표시](#)

▼ inflearn-bigquery ☆ ⋮

▶ 외부 연결 ⋮

basic ☆ ⋮

열기

다음 항목에서 열기 ▶

콘텐츠 새로그침

링크 복사

테이블 만들기

공유

ID 복사

삭제

SQL Worksp

시작하기

+ 새 쿼리 작성

샘플 데이터로 사용해

Google Cloud 설정 - 테이블 생성(pokemon)

소스

테이블을 만들 소스

업로드

파일 선택 *

초보자를 위한 BigQuery(SQL) 입문 - 데이터 - [포켓몬] pokemon.csv X [찾아보기](#) ?

파일 형식

CSV

대상

프로젝트 *

inlearn-bigquery [찾아보기](#)

데이터 세트 *

basic

테이블 *

pokemon

유니코드 문자, 표시, 숫자, 커넥터, 대시, 공백이 허용됩니다.

테이블 유형

기본 테이블

스키마

☒ 자동 감지

?

스키마가 자동으로 생성됩니다.

파티션 및 클러스터 설정

파티션 나누기

파티션 나누기 없음

클러스터링 수식

테이블 만들기

취소

- 테이블을 만들 소스 : 업로드
- 파일 선택 : 다운받은 파일 선택
- 테이블 : 파일 이름(pokemon)
- 스키마 : 자동 감지 체크

테이블 만들기 클릭

Google Cloud 설정 - 테이블 생성(trainer)

소스

테이블을 만들 소스

업로드

파일 선택 *
초보자를 위한 BigQuery(SQL) 입문 - 데이터 - [포켓몬] trainer.csv X [찾아보기](#) ?

파일 형식
CSV

대상

프로젝트 *
inlearn-bigquery [찾아보기](#)

데이터 세트 *
basic

테이블 *
trainer
유니코드 문자, 표시, 숫자, 커넥터, 대시, 공백이 허용됩니다.

테이블 유형
기본 테이블

스키마

☒ 자동 감지

!

스키마가 자동으로 생성됩니다.

파티션 및 클러스터 설정

파티션 나누기
파티션 나누기 없음

클러스터링 수식

테이블 만들기

취소

- 테이블을 만들 소스 : 업로드
- 파일 선택 : 다운받은 파일 선택
- 테이블 : 파일 이름(trainer)
- 스키마 : 자동 감지 체크

테이블 만들기 클릭

Google Cloud 설정 - 테이블 생성(trainer_pokemon)

소스

테이블을 만들 소스
업로드

파일 선택 *
초보자를 위한 BigQuery(SQL) 입문 - 데이터 - [포켓몬] trainer_pokemon.csv X [찾아보기](#) ?

파일 형식
CSV

대상

프로젝트 *
inlearn-bigquery [찾아보기](#)

데이터 세트 *
basic

테이블 *
trainer_pokemon
유니코드 문자, 표시, 숫자, 커넥터, 대시, 공백이 허용됩니다.

테이블 유형
기본 테이블

스키마

☒ 자동 감지

?

스키마가 자동으로 생성됩니다.

파티션 및 클러스터 설정

파티션 나누기
파티션 나누기 없음

클러스터링 수식

[테이블 만들기](#) [취소](#)

- 테이블을 만들 소스 : 업로드
- 파일 선택 : 다운받은 파일 선택
- 테이블 : 파일 이름(trainer_pokemon)
- 스키마 : 자동 감지 체크

테이블 만들기 클릭

Google Cloud 설정 - 테이블 생성(battle)

테이블을 만들 소스

업로드

파일 선택 *

초보자를 위한 BigQuery(SQL) 입문 - 데이터 - [포켓몬] battle.csv

X

찾아보기

파일 형식

CSV

대상

프로젝트 *

inlearn-biqquery

찾아보기

데이터 세트 *

basic

테이블 *

battle

유니코드 문자, 표시, 숫자, 커넥터, 대시, 공백이 허용됩니다.

테이블 유형

기본 테이블

- 테이블을 만들 소스 : 업로드
- 파일 선택 : 다운받은 파일 선택
- 테이블 : 파일 이름(battle)

과거 테이블과 다르게 저장

- 스키마 : 자동 감지 X

Google Cloud 설정 - 테이블 생성(battle) : 스키마 정의

스키마

☐ 자동 감지

☒ 텍스트로 편집

필드 이름 *	유형 *	모드
1 id	INTEGER	NULLABLE
필드 이름 *	유형 *	모드
2 player1_id	INTEGER	NULLABLE
필드 이름 *	유형 *	모드
3 player2_id	INTEGER	NULLABLE
필드 이름 *	유형 *	모드
4 winner_id	INTEGER	NULLABLE
필드 이름 *	유형 *	모드
5 battle_date	DATE	NULLABLE
필드 이름 *	유형 *	모드
6 battle_datetime	DATETIME	NULLABLE
필드 이름 *	유형 *	모드
7 battle_timestamp	TIMESTAMP	NULLABLE
8 +		

스키마

☐ 자동 감지

☒ 텍스트로 편집

접근성 옵션을 보려면 Option+F1 키를 누르세요.

```
1 id:INTEGER,  
2 player1_id:INTEGER,  
3 player2_id:INTEGER,  
4 winner_id:INTEGER,  
5 battle_date:DATE,  
6 battle_datetime:DATETIME,  
7 battle_timestamp:TIMESTAMP
```

복사 붙여넣기용

```
id:INTEGER,  
player1_id:INTEGER,  
player2_id:INTEGER,  
winner_id:INTEGER,  
battle_date:DATE,  
battle_datetime:DATETIME,  
battle_timestamp:TIMESTAMP
```

Google Cloud 설정 - 테이블 생성(battle) : 파티션 및 고급 옵션

파티션 및 클러스터 설정

파티션 나누기

battle_datetime

파티션 나누기 필터 ?

☐ 데이터를 쿼리하려면 WHERE 절 필요

파티션 나누기 유형 ?

☒ 날짜별

☐ 시간별

☐ 월별

☐ 연도별

고급 옵션

쓰기 환경설정

비어 있으면 쓰기

허용되는 오류 개수

0

☐ 알 수 없는 값 ?

필드 구분 기호

쉼표

따옴표 문자

큰따옴표

건너뛰 헤더 행

1

☐ 따옴표 내 줄바꿈 ?

☐ 불균일 행 ?

Google Cloud 설정 - 생성된 테이블 확인

▼ inflearn-bigquery	☆	⋮
▶ 외부 연결		⋮
▼ basic	☆	⋮
battle	☆	⋮
pokemon	☆	⋮
trainer	☆	⋮
trainer_pokemon	☆	⋮

4개의 테이블 생성

battle의 모양이 다른 이유
= 파티션을 설정한 Table

Google Cloud 설정 - 쿼리 실행해보기

테이블을 클릭하고 쿼리 - 새 탭에서 열기 누르면 쿼리 포맷이 나옴

The screenshot shows the Google Cloud BigQuery interface. On the left, the 'inflearn-bigquery' project is expanded, showing the 'pokemon' table under the 'basic' dataset. The main panel displays the 'pokemon' table schema with columns: id, kor_name, eng_name, type1, type2, total, hp, attack, defense, special_attack, special_defense, speed, generation, and is_legendary. An orange box highlights the '새 탭에서 열기' (Open in new tab) button in the top bar. An orange arrow points from this button to the query editor at the bottom. The query editor shows a SQL query: `SELECT FROM `inflearn-bigquery.basic.pokemon` LIMIT 1000`. The query is highlighted with an orange box, and the '실행' (Run) button is visible next to it.

탐색기 + 추가 <

리소스를 보는 중입니다.
별표표시된 리소스만 표시

inflearn-bigquery ☆ ::
 외부 연결 ::
 basic ☆ ::
 battle ☆ ::
 pokemon ☆ ::
 trainer ☆ ::
 trainer_pokemon ☆ ::

요약

pokemon 쿼리 공유 복사 스냅샷 삭제 내보내기

스키마 세부정보 새 탭에서 열기 분할 탭에서 열기

필터 속성 이름 또는 값 입력

☐	필드 이름	유형	모드	키	대조	기본값	정책 태그	설명
☐	id	INTEGER	NULLABLE					
☐	kor_name	STRING	NULLABLE					
☐	eng_name	STRING	NULLABLE					
☐	type1	STRING	NULLABLE					
☐	type2	STRING	NULLABLE					
☐	total	INTEGER	NULLABLE					
☐	hp	INTEGER	NULLABLE					
☐	attack	INTEGER	NULLABLE					
☐	defense	INTEGER	NULLABLE					
☐	special_attack	INTEGER	NULLABLE					
☐	special_defense	INTEGER	NULLABLE					
☐	speed	INTEGER	NULLABLE					
☐	generation	INTEGER	NULLABLE					
☐	is_legendary	BOOLEAN	NULLABLE					

pokemon 제목없음

제목없음 실행 저장 다운로드 공유 일정 더보기

1 SELECT FROM `inflearn-bigquery.basic.pokemon` LIMIT 1000

Google Cloud 설정 - 쿼리 실행해보기

탭에서 + 버튼을 눌러도 쿼리를 실행할 수 있는 창이 나타남

The screenshot shows the Google Cloud BigQuery interface. On the left is the '탐색기' (Explorer) sidebar with a tree view containing 'inflearn-bigquery' and its sub-items: '외부 연결' (External connection), 'basic', 'battle', 'pokemon', 'trainer', and 'trainer_pokemon'. The 'pokemon' dataset is selected. The main area displays a table of query results with columns: '필드 이름' (Field name), '유형' (Type), and '모드' (Mode). The table lists various fields like 'id', 'kor_name', 'eng_name', 'type1', 'type2', 'total', 'hp', 'attack', 'defense', 'special_attack', 'special_defense', 'speed', 'generation', and 'is_legendary'.

An orange box highlights the '+ 추가' (Add) button in the top bar of the 'pokemon' dataset view. Another orange box highlights a new query execution window that appears when the '+' button is clicked. This window has a title bar with 'pokemon' and a search icon. The main area of this window shows a search bar with the text '제목 없음 2' (Untitled 2) and a '1' in the input field. Below the search bar are buttons for '실행' (Run), '저장' (Save), '다운로드' (Download), '공유' (Share), '일정' (Schedule), and '더보기' (More).

필드 이름	유형	모드
☐ id	INTEGER	NULLABLE
☐ kor_name	STRING	NULLABLE
☐ eng_name	STRING	NULLABLE
☐ type1	STRING	NULLABLE
☐ type2	STRING	NULLABLE
☐ total	INTEGER	NULLABLE
☐ hp	INTEGER	NULLABLE
☐ attack	INTEGER	NULLABLE
☐ defense	INTEGER	NULLABLE
☐ special_attack	INTEGER	NULLABLE
☐ special_defense	INTEGER	NULLABLE
☐ speed	INTEGER	NULLABLE
☐ generation	INTEGER	NULLABLE
☐ is_legendary	BOOLEAN	NULLABLE

Google Cloud 설정 - 쿼리 실행해보기

아래 쿼리 실행

SELECT

*

FROM `basic.pokemon`

ORDER BY id

탐색기 + 추가 <

Q 입력하여 검색

리소스를 보는 중입니다.
발표 표시된 리소스만 표시

▼ inflearn-bigquery ☆ ::

▶ 외부 연결 ::

▼ basic ☆ ::

battle ☆ ::

pokemon ☆ ::

trainer ☆ ::

trainer_pokemon ☆ ::

요약

현재 선택된 항목 없음

제목 없음 실행 저장 다운로드 공유 일정 더보기

1 SELECT

2 *

3 FROM `basic.pokemon`

4 ORDER BY id

쿼리 결과 결과 저장 데이터 탐색

작업 정보 결과 차트 미리보기 JSON 실행 세부정보 실행 그래프

행	id	kor_name	eng_name	type1	type2
1	1	이상해씨	Bulbasaur	Grass	Poison
2	2	이상해풀	Ivysaur	Grass	Poison
3	3	이상해꽃	Venusaur	Grass	Poison
4	4	파이어리	Charmander	Fire	null
5	5	리자드	Charmeleon	Fire	null
6	6	리자몽	Charizard	Fire	Flying
7	7	꼬부기	Squirtle	Water	null
8	8	어니부기	Wartortle	Water	null
9	9	거북왕	Blastoise	Water	null
10	10	캐터피	Caterpie	Bug	null
11	11	단테기	Metapod	Bug	null
12	12	비파리	Butterfree	Bug	Flying

페이지당 결과 수: 50 1 - 50 (전체 251행) < >

#2. 데이터 탐색 - 조건, 추출, 요약

카일스쿨



이번 파트에서 다룰 내용

- 데이터 활용 과정 Overview
- 포켓몬으로 이해하는 SQL
- 데이터 탐색 : 조건과 추출하기
 - SQL의 기본 문법
 - SELECT
 - FROM
 - WHERE
- 데이터 탐색 : 요약하기

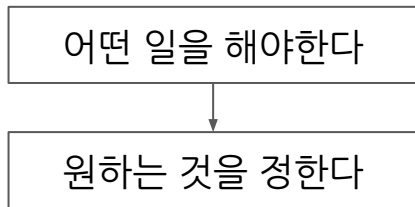
데이터 활용 과정 Overview

데이터를 활용하는 과정

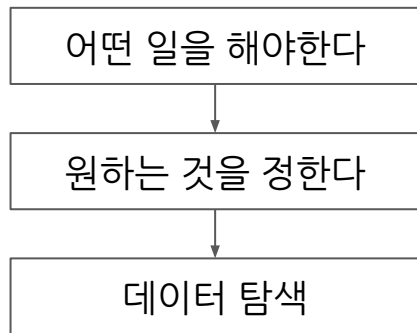
어떤 일을 해야한다



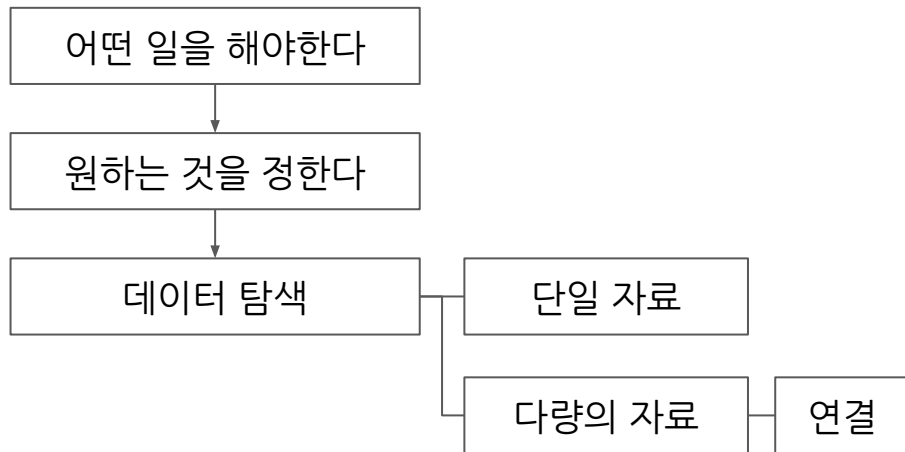
데이터를 활용하는 과정



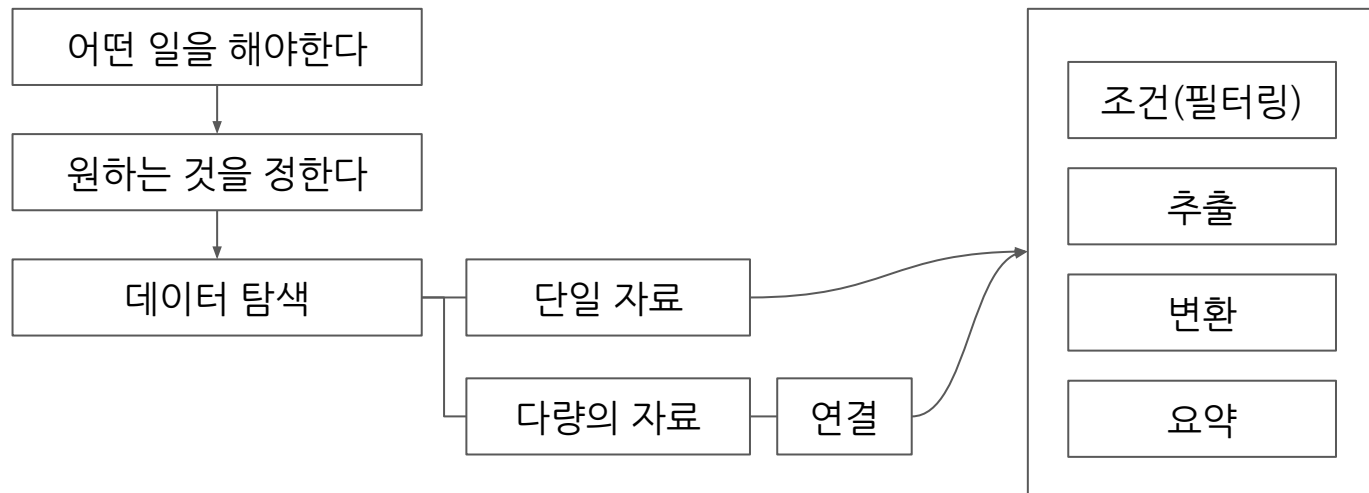
데이터를 활용하는 과정



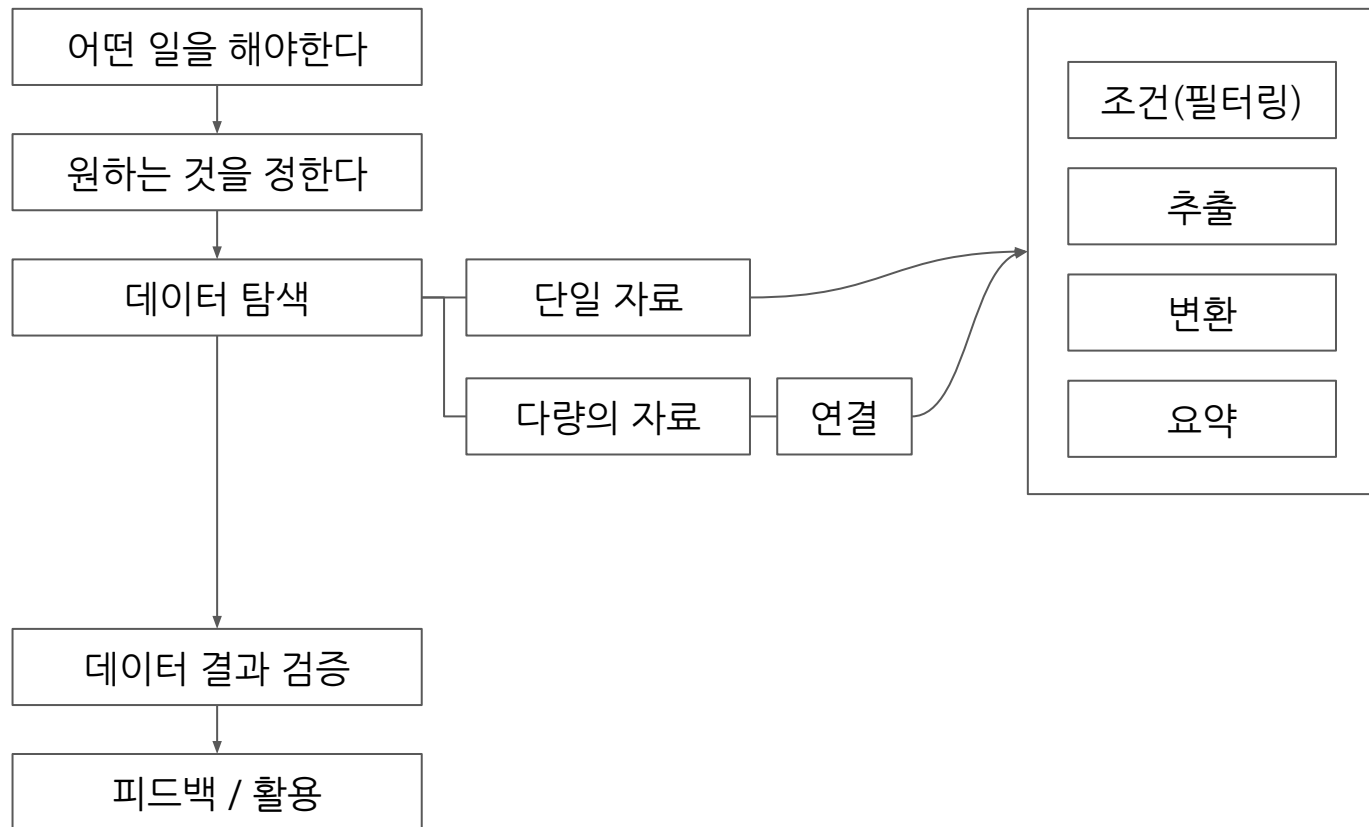
데이터를 활용하는 과정



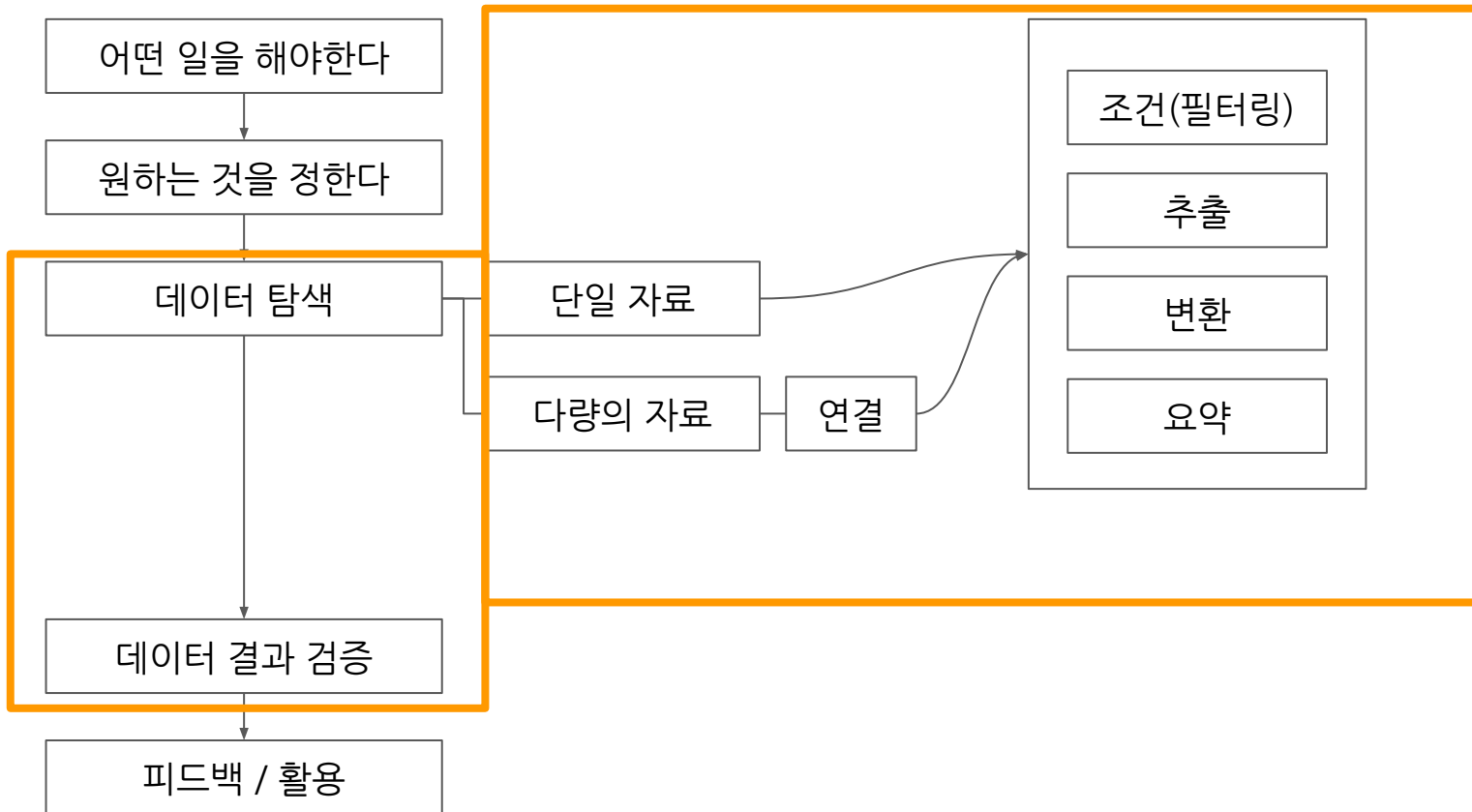
데이터를 활용하는 과정



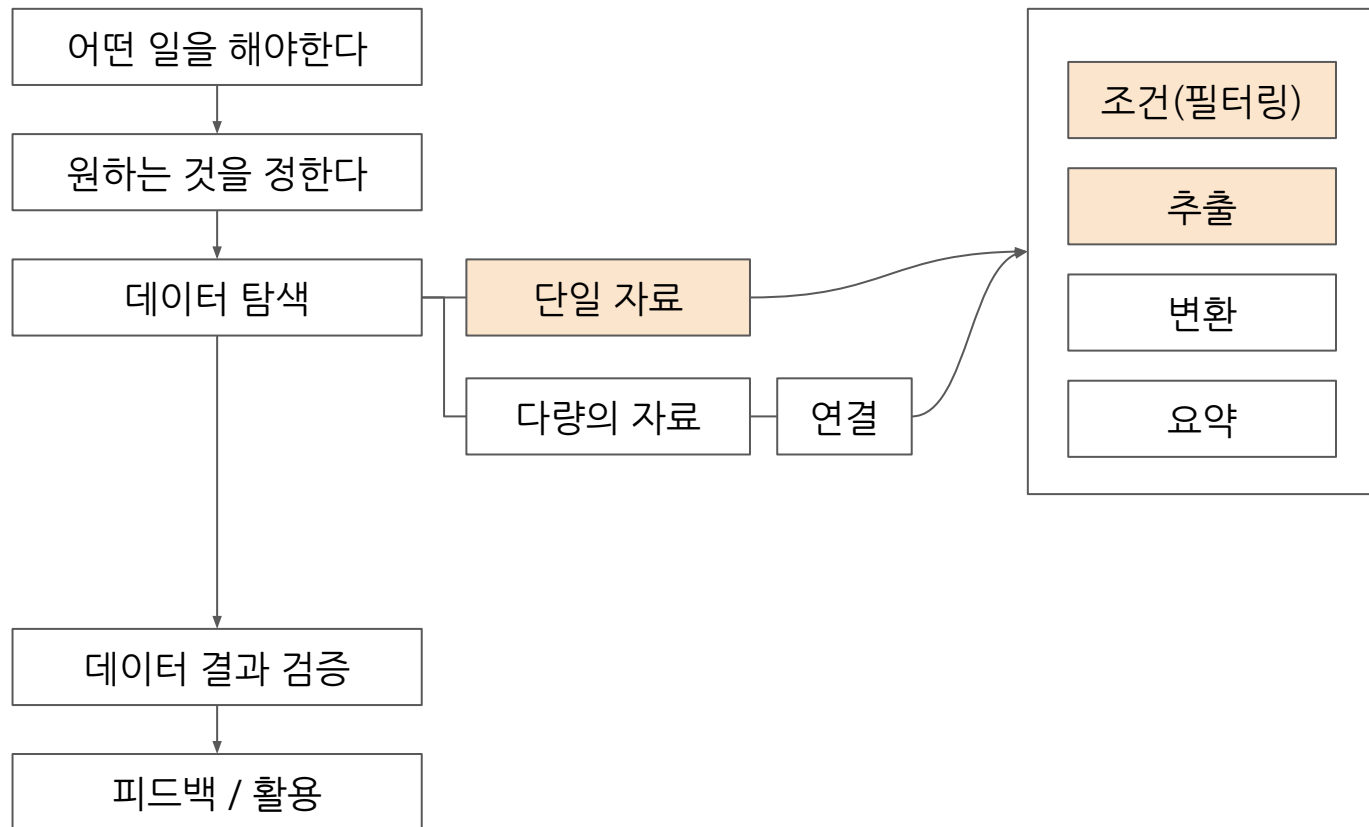
데이터를 활용하는 과정



이 부분을 SQL로 처리할 수 있음 : 이 강의에서 다루는 내용



이어지는 강의에서 학습할 내용



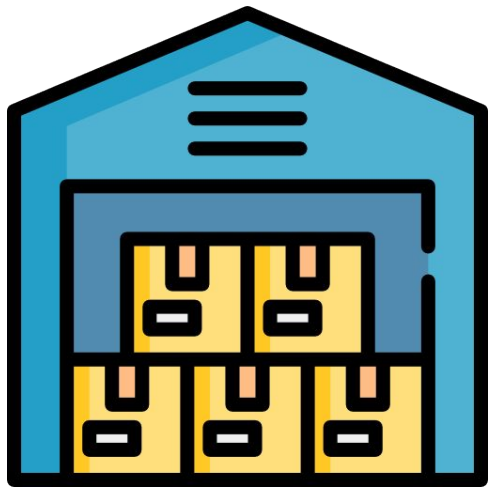
저장된 데이터 확인하기

SQL 쿼리를 작성하기 전에

- 데이터가 어떻게 저장되어 있는가?에 대해 생각해보기
- 어떤 데이터가 저장되어 있는가?
- 컬럼의 의미는 무엇인가?

데이터를 제대로 이해해야 올바른 데이터를 추출할 수 있음
(구체적인 정의를 항상 확인하면서 쿼리를 작성해야 함)

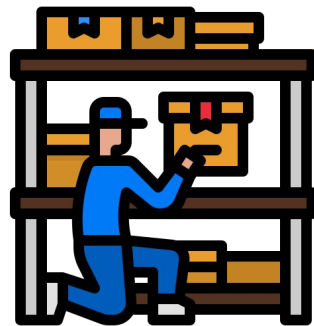
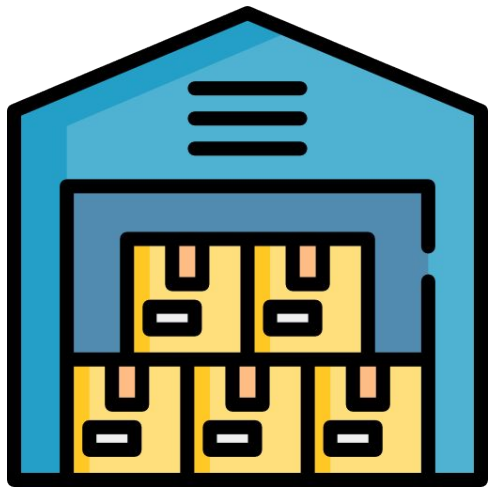
물품 창고에서 배송할 때 어떻게 해야할까요?



물품 창고에서 배송할 때 어떻게 해야할까요?

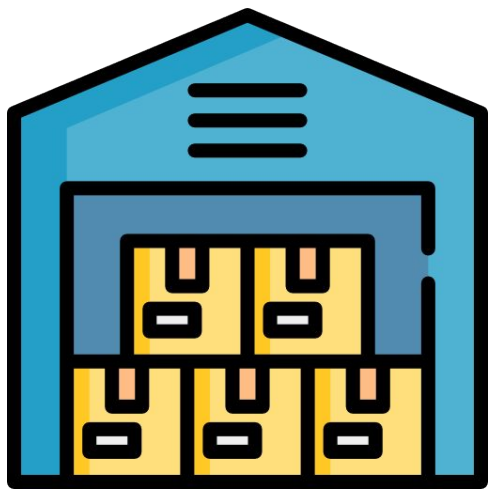
여러가지 방법이 존재

- 물품 창고가 작다면 그냥 하나씩 다 찾아보면 됨
- 물품 창고가 점점 커지면 어디에 물품이 있는지 확인해보면 좋음
- 다이소에 방문해서 물건을 찾기 어려울 때 어디에 배치되어 있는지 확인함



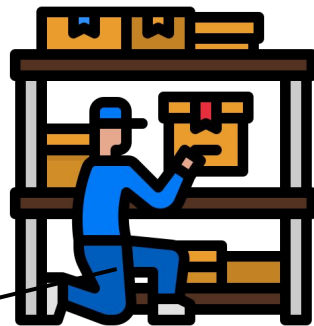
물품 창고에서 배송할 때 어떻게 해야할까요?

[데이터 웨어하우스]



데이터를 추출하기 전에
데이터 웨어하우스에 데이터가 어떻게 저장되어
있는지 확인

[데이터]

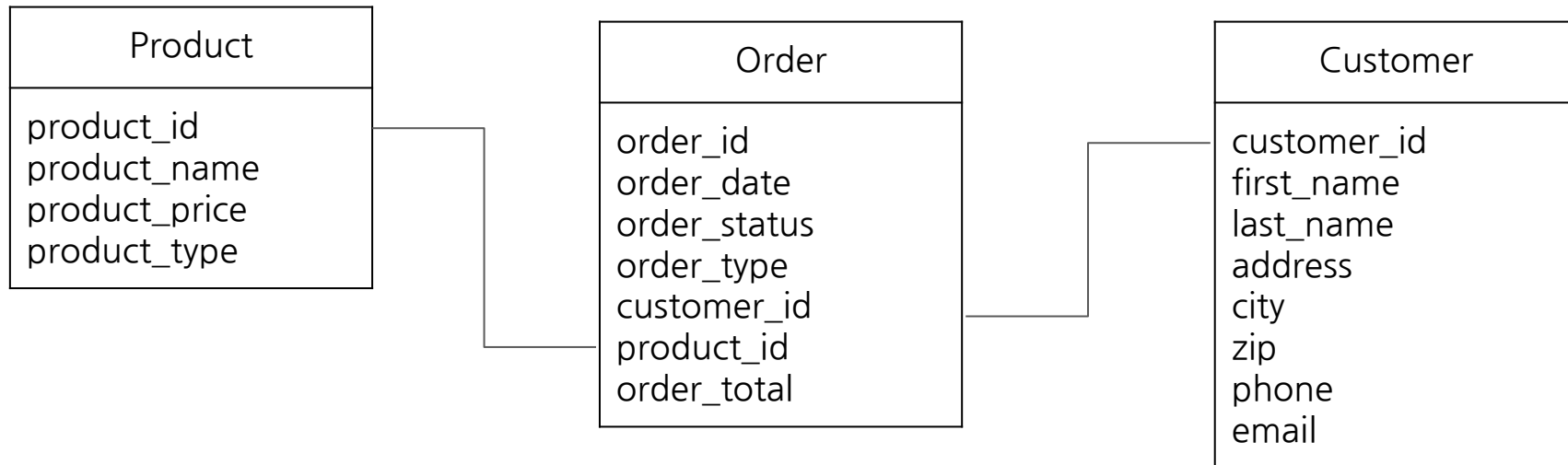


[우리]

데이터가 저장되는 형태를 알려면?

- ERD(Entity Relationship Diagram) : 데이터베이스 구조를 한눈에 알아보기 위해 사용
- 회사마다 있을수도 있고, 없을수도 있음
- 회사의 개발자, DBA 분들에게 물어보기

ERD(Entity Relationship Diagram) 예시



데이터가 저장되는 형태를 알려면?

- ERD가 없다면 모든 데이터베이스를 직접 보면서 탐색해보기
 - 어떤 **테이블**이 존재하는가?
 - 어떤 **컬럼**이 존재하는가?
 - 다른 테이블과 **연결**할 때 어떤 컬럼을 사용하는가?
 - 컬럼의 값들은 어떤 **의미**를 가지는가?
- 스프레드시트나 문서로 따로 정리해두기

회사에 존재할 수 있는 데이터(예시)

1) 서비스에 사용될 데이터베이스

- 유저 테이블
- 배송 테이블
- 물건 테이블

2) 앱/웹 로그 데이터

- 유저가 앱/웹에 들어와서 회원 가입 - 페이지 확인, 콘텐츠 확인 등등의 데이터
- "과정"을 알 수 있는 데이터

3) 공공 데이터, 서드파티 데이터

- 날씨
- 페이스북 광고 데이터

포켓몬 세상을 데이터로 생각해보기

- 포켓몬 세상엔 어떤 데이터가 존재할까요?

- 포켓몬
- 트레이너
- 트레이너가 잡은 포켓몬
- 트레이너가 도전한 유저 배틀
- 트레이너가 도전한 체육관 배틀
- NPC
- 상점
- 상점 별로 판매하는 제품

이커머스 산업의 데이터 생각하기

- 이커머스 산업엔 어떤 데이터가 존재할까요?
 - 포켓몬 : 상품
 - 트레이너 : 유저
 - 트레이너가 잡은 포켓몬 : 주문
 - 장바구니에 담은 물건
 - 웹페이지 접근 수
 - 웹페이지의 버튼 클릭 수

포켓몬 데이터 BigQuery에서 확인하기

저장된 데이터를 확인하면서 데이터 정리 및 ERD 그리기

탐색기

+ 추가

🔍 입력하여 검색

리소스를 보는 중입니다.
[별표 표시된 리소스만 표시](#)

▼ inflearn-bigquery

▶ 외부 연결

▼ basic

battle

pokemon

trainer

trainer_pokemon

pokemon															
스키마		세부정보		미리보기		계보		데이터 프로필		데이터 품질					
행	id	kor_name	eng_name	type1	type2	total	hp	attack	defense	special_attack	special_defense	speed	generation	is_legendary	
1	10	캐터피	Caterpie	Bug	null	195	45	30	35	20	20	45	1	false	
2	11	단테기	Metapod	Bug	null	205	50	20	55	25	25	30	1	false	
3	127	뿌사이저	Pinsir	Bug	null	500	65	125	100	55	70	85	1	false	
4	204	피콘	Pineco	Bug	null	290	50	65	90	35	35	15	2	false	
5	197	블래키	Umbreon	Dark	null	525	95	65	110	60	130	65	2	false	
6	4	파이어리	Charmander	Fire	null	309	39	52	43	60	50	65	1	false	
7	5	리자드	Charmeleon	Fire	null	405	58	64	58	80	65	80	1	false	
8	37	식스테일	Vulpix	Fire	null	299	38	41	40	50	65	65	1	false	
9	38	나인테일	Ninetales	Fire	null	505	73	76	75	81	100	100	1	false	
10	58	가디	Growlithe	Fire	null	350	55	70	45	70	50	60	1	false	
11	59	윈디	Arcanine	Fire	null	555	90	110	80	100	80	95	1	false	
12	77	포니타	Ponyta	Fire	null	410	50	85	55	65	65	90	1	false	
13	78	날쌘마	Rapidash	Fire	null	500	65	100	70	80	80	105	1	false	
14	126	마그마	Magmar	Fire	null	495	65	95	57	100	85	93	1	false	
15	136	부스터	Flareon	Fire	null	525	65	130	60	95	110	65	1	false	
16	155	브케인	Cyndaquil	Fire	null	309	39	52	43	60	50	65	2	false	

포켓몬 데이터 BigQuery에서 확인하기

저장된 데이터를 확인하면서 데이터 정리 및 ERD 그리기

탐색기

+ 추가

IK

Q 입력하여 검색

?

리소스를 보는 중입니다.

별표 표시된 리소스만 표시

▼ inflearn-bigquery

▶ 외부 연결

▼ basic

battle

pokemon

trainer

trainer_pokemon

요약

▼

trainer

쿼리

공유

복사

스냅샷

삭제

내보내기

스키마

세부정보

미리보기

계보

데이터 프로필

신규

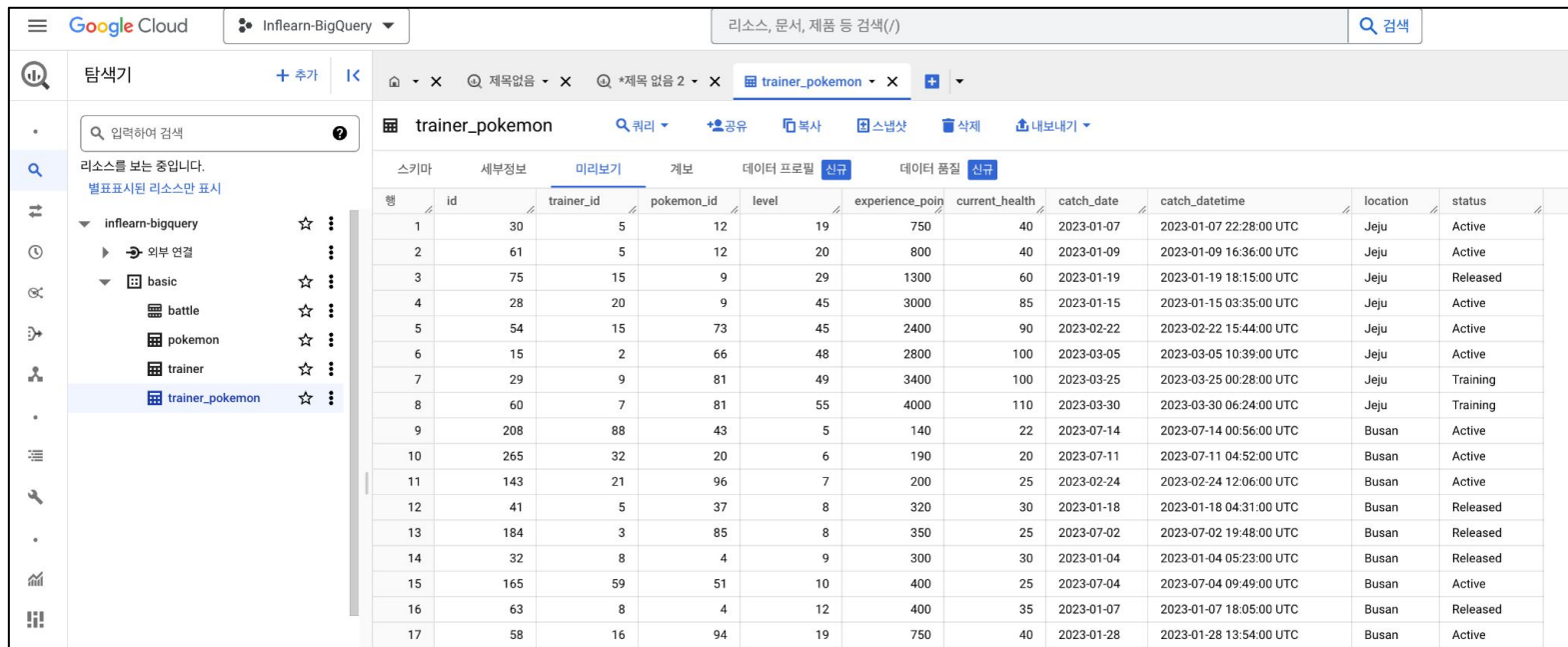
데이터 품질

신규

행	id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	13	Goh	10	Seoul	Bug	1	Beginner
2	32	Trevor	12	Seoul	Grass	1	Beginner
3	21	Iris	23	Seoul	Dragon	2	Beginner
4	30	Shauna	14	Gwangyang	Fairy	2	Beginner
5	88	Whitney	19	Gyeongsan	Ice	2	Beginner
6	9	Lilly	13	Incheon	Psychic	3	Intermediate
7	11	Lana	12	Seoul	Water	3	Intermediate
8	16	Mary	15	Seoul	Normal	3	Intermediate
9	29	Bianca	15	Seoul	Normal	3	Intermediate
10	34	Kiyo	11	Pohang	Fighting	3	Intermediate
11	57	Wally	14	Jeju	Grass	3	Intermediate
12	58	Lucas	14	Incheon	Water	3	Intermediate
13	59	Dawn	14	Seoul	Ice	3	Intermediate
14	60	Barry	14	Jeju	Fire	3	Intermediate
15	6	Dawn	14	Daejeon	Ice	4	Intermediate
16	10	Yun	20	Gangwon	Fire	4	Intermediate
17	12	KP	15	Seoul	Electric	4	Intermediate
18	31	Tierno	16	Seoul	Normal	4	Intermediate
19	33	Mallow	14	Incheon	Grass	4	Intermediate

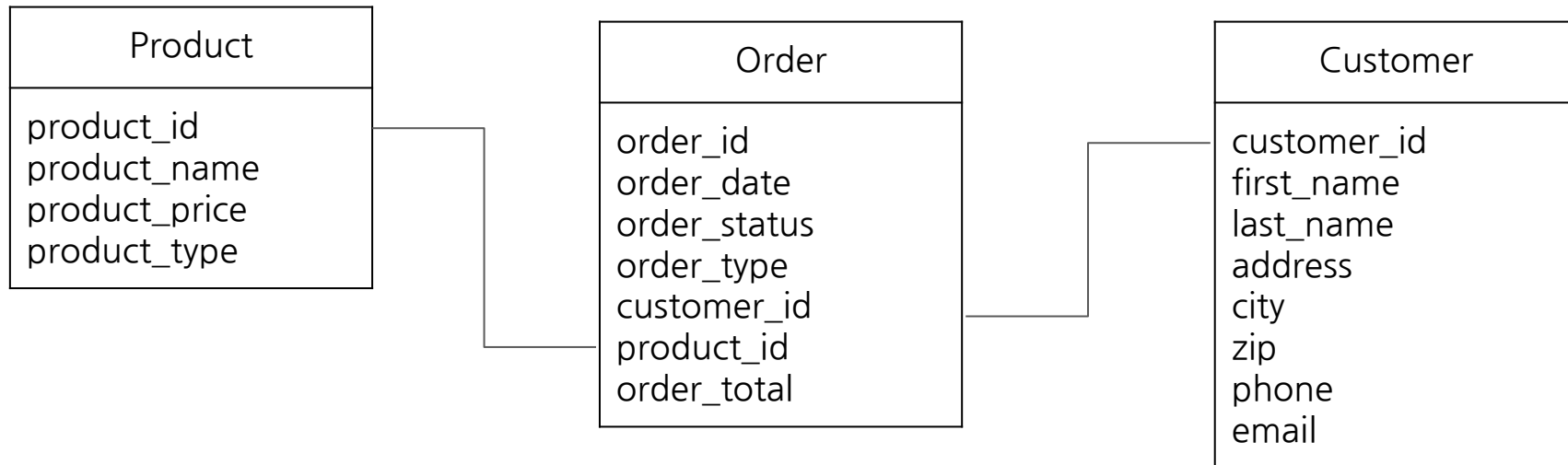
포켓몬 데이터 BigQuery에서 확인하기

저장된 데이터를 확인하면서 데이터 정리 및 ERD 그리기

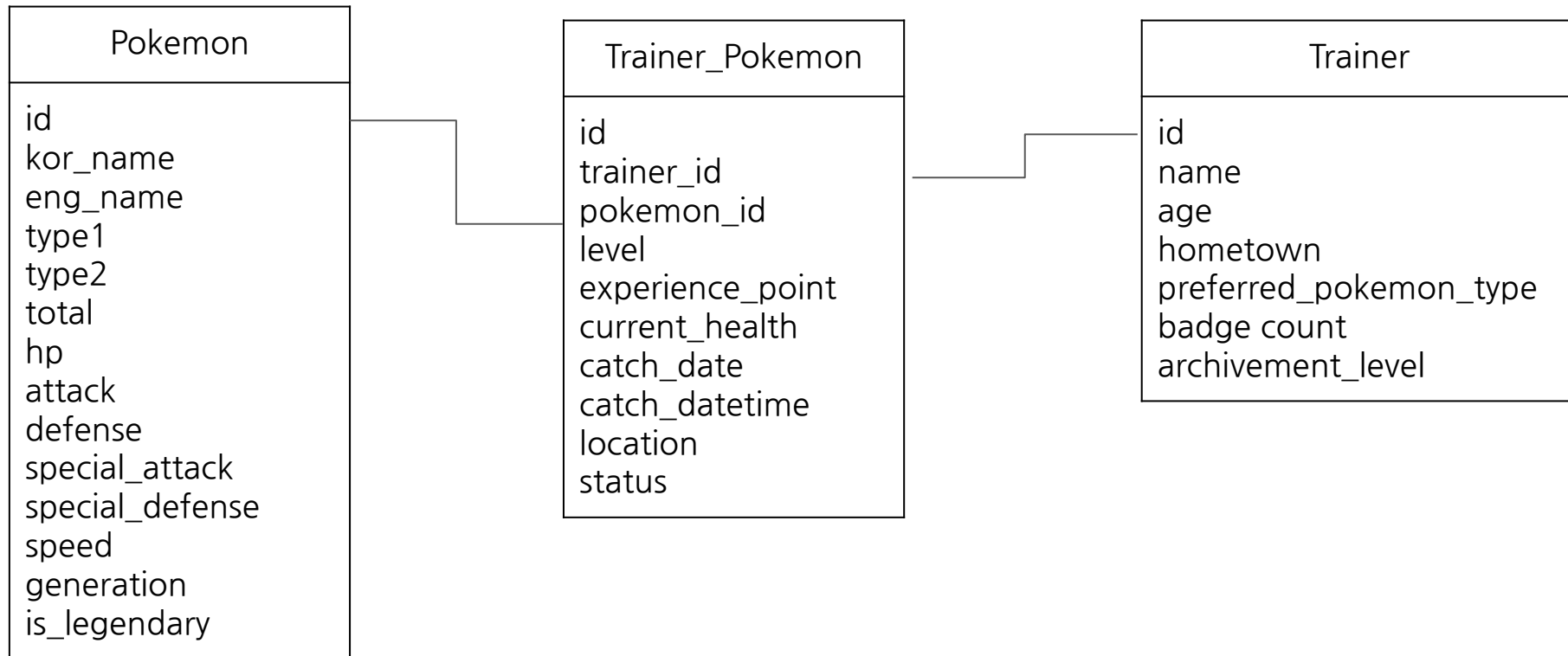


행	id	trainer_id	pokemon_id	level	experience_poin	current_health	catch_date	catch_datetime	location	status
1	30	5	12	19	750	40	2023-01-07	2023-01-07 22:28:00 UTC	Jeju	Active
2	61	5	12	20	800	40	2023-01-09	2023-01-09 16:36:00 UTC	Jeju	Active
3	75	15	9	29	1300	60	2023-01-19	2023-01-19 18:15:00 UTC	Jeju	Released
4	28	20	9	45	3000	85	2023-01-15	2023-01-15 03:35:00 UTC	Jeju	Active
5	54	15	73	45	2400	90	2023-02-22	2023-02-22 15:44:00 UTC	Jeju	Active
6	15	2	66	48	2800	100	2023-03-05	2023-03-05 10:39:00 UTC	Jeju	Active
7	29	9	81	49	3400	100	2023-03-25	2023-03-25 00:28:00 UTC	Jeju	Training
8	60	7	81	55	4000	110	2023-03-30	2023-03-30 06:24:00 UTC	Jeju	Training
9	208	88	43	5	140	22	2023-07-14	2023-07-14 00:56:00 UTC	Busan	Active
10	265	32	20	6	190	20	2023-07-11	2023-07-11 04:52:00 UTC	Busan	Active
11	143	21	96	7	200	25	2023-02-24	2023-02-24 12:06:00 UTC	Busan	Active
12	41	5	37	8	320	30	2023-01-18	2023-01-18 04:31:00 UTC	Busan	Released
13	184	3	85	8	350	25	2023-07-02	2023-07-02 19:48:00 UTC	Busan	Released
14	32	8	4	9	300	30	2023-01-04	2023-01-04 05:23:00 UTC	Busan	Released
15	165	59	51	10	400	25	2023-07-04	2023-07-04 09:49:00 UTC	Busan	Active
16	63	8	4	12	400	35	2023-01-07	2023-01-07 18:05:00 UTC	Busan	Released
17	58	16	94	19	750	40	2023-01-28	2023-01-28 13:54:00 UTC	Busan	Active

ERD(Entity Relationship Diagram) 예시



ERD(Entity Relationship Diagram) 예시 - Pokemon



데이터 탐색 : 조건과
추출
- SELECT, WHERE

포켓몬으로 SELECT 이해하기



포켓몬스터

- 포켓몬 트레이너와 포켓몬이 함께 여행을 떠남
- 체육관에 도전해 배지를 취득하기도 함
- 포켓몬의 레벨 존재
- 포켓몬 배틀을 하기도 함

포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민



포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등



포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등



이름 : 꼬부기
타입 : 물
공격력 : 48
특수 공격력 : 50

포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등

이름 : 피카츄
타입 : 전기
공격력 : 55
특수 공격력 : 50



이름 : 꼬부기
타입 : 물
공격력 : 48
특수 공격력 : 50

이름 : 이상해씨
타입 : 풀
공격력 : 49
특수 공격력 : 65

이름 : 파이어리
타입 : 불
공격력 : 52
특수 공격력 : 60

포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등
“꼬부기!!”

이름 : 피카츄
타입 : 전기
공격력 : 55
특수 공격력 : 50



이름 : 꼬부기
타입 : 물
공격력 : 48
특수 공격력 : 50

이름 : 이상해씨
타입 : 풀
공격력 : 49
특수 공격력 : 65

이름 : 파이어리
타입 : 불
공격력 : 52
특수 공격력 : 60

포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등
“공격력이 제일 높은 포켓몬”

이름 : 피카츄
타입 : 전기
공격력 : 55
특수 공격력 : 50



이름 : 꼬부기
타입 : 물
공격력 : 48
특수 공격력 : 50

이름 : 이상해씨
타입 : 풀
공격력 : 49
특수 공격력 : 65

이름 : 파이어리
타입 : 불
공격력 : 52
특수 공격력 : 60

포켓몬으로 SELECT 이해하기

포켓몬 여행을 시작할 때 함께 할 포켓몬 고민 : 포켓몬 이름, 공격력 높은 포켓몬, 타입 등
“불 타입”

이름 : 피카츄
타입 : 전기
공격력 : 55
특수 공격력 : 50



이름 : 꼬부기
타입 : 물
공격력 : 48
특수 공격력 : 50

이름 : 이상해씨
타입 : 풀
공격력 : 49
특수 공격력 : 65

이름 : 파이어리
타입 : 불
공격력 : 52
특수 공격력 : 60

앞에서 말한 데이터를 데이터베이스 테이블 형태로 생각하면

각각의 데이터 소스를 아래와 같이 표현. Row와 Column!

이미지	이름	타입	공격력	특수 공격력	...
	피카츄	전기	55	50	
	꼬부기	물	48	50	
	이상해씨	풀	49	65	
	파이리	불	52	60	

SQL 쿼리 구조

SQL 쿼리문은 아래와 같은 문법으로 작성

우선 SELECT, FROM, WHERE

SELECT

Col1 AS new_name,
Col2,
Col3

FROM Dataset.Table

WHERE

Col1 = 1

SQL 쿼리 구조

SQL 쿼리문은 아래와 같은 문법으로 작성

우선 SELECT, FROM, WHERE

SELECT

Col1 AS new_name,
Col2,
Col3

FROM Dataset.Table : 어떤 테이블에서 데이터를 확인할 것인가?

WHERE

Col1 = 1

SQL 쿼리 구조

SQL 쿼리문은 아래와 같은 문법으로 작성

우선 SELECT, FROM, WHERE

SELECT

Col1 AS new_name,
Col2,
Col3

FROM Dataset.Table : 어떤 테이블에서 데이터를 확인할 것인가?

WHERE : 만약 원하는 조건이 있다면 어떤 조건인가?

Col1 = 1 : 조건문

SQL 쿼리 구조

SQL 쿼리문은 아래와 같은 문법으로 작성

우선 SELECT, FROM, WHERE

SELECT : 테이블의 어떤 컬럼을 선택(출력)할 것인가?

Col1 AS new_name,
Col2,
Col3

FROM Dataset.Table : 어떤 테이블에서 데이터를 확인할 것인가?

WHERE : 만약 원하는 조건이 있다면 어떤 조건인가?

Col1 = 1 : 조건문

포켓몬 타입이 불인 포켓몬을 찾는 SQL 쿼리

```
SELECT  
  *  
FROM basic.pokemon  
WHERE  
  type1= "Fire"
```

포켓몬 타입이 불인 포켓몬을 찾는 SQL 쿼리

```
SELECT
  *
FROM basic.pokemon
WHERE
  type1= "Fire"
```

* : 모든 컬럼을 출력하겠다

```
SELECT
  * EXCEPT(제외할 컬럼)
이런 형태도 가능
```

집합처럼 생각해보기

특정 Table에 있는 데이터를 추출

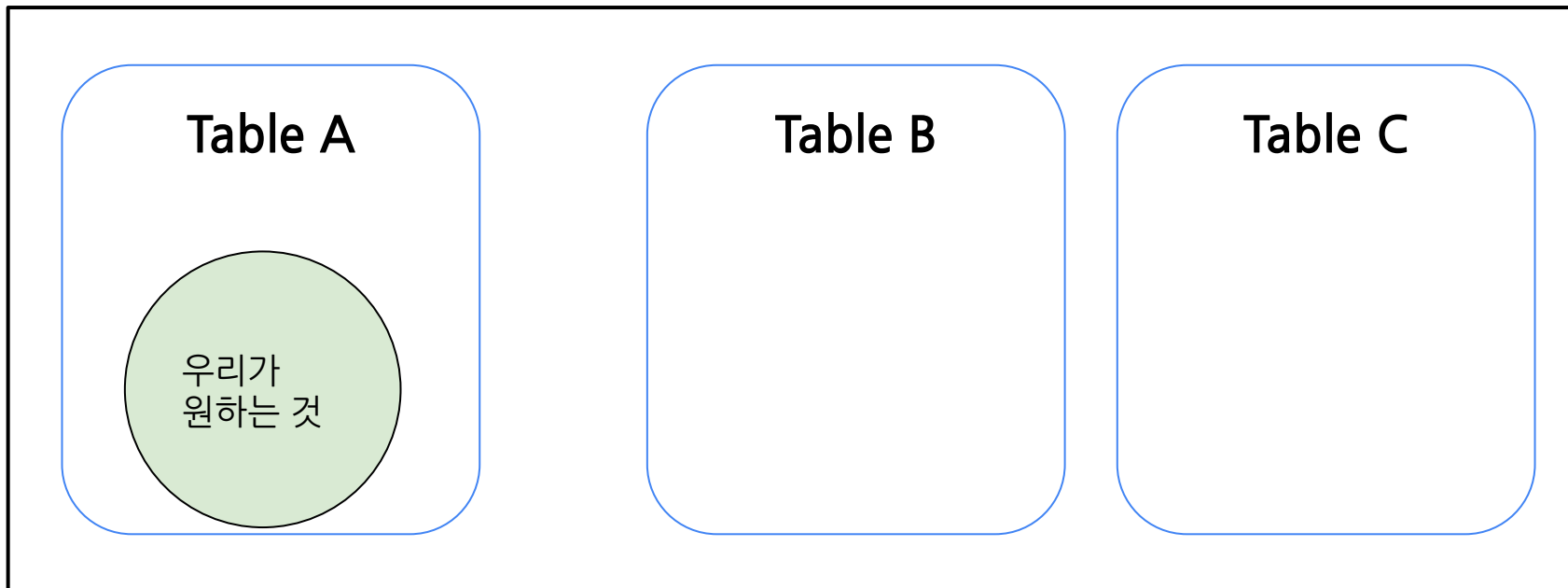
Table A

Table B

Table C

집합처럼 생각해보기

특정 Table에 있는 데이터를 추출



집합처럼 생각해보기

특정 Table에 있는 데이터를 추출

Table A

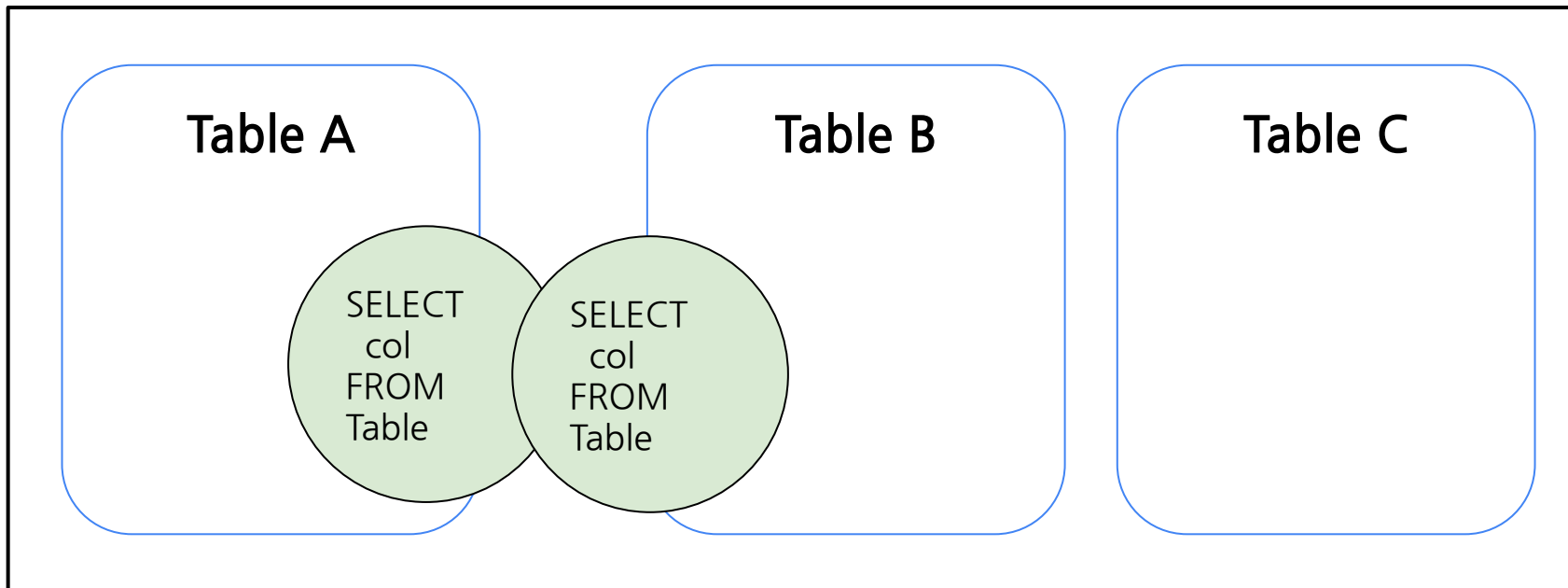
SELECT
col
FROM
Table

Table B

Table C

데이터가 여러 장소에 저장되어 있는 경우

특정 Table에 있는 데이터를 각각 추출 후, 연결하기(추후에 배울 JOIN)



SQL 문법 핵심 정리

	설명
FROM	데이터를 확인할 Table 명시 이름이 너무 길다면 AS “별칭”으로 별칭 지정 가능 FROM Table1 AS t1
WHERE	FROM에 명시된 Table에 저장된 데이터를 필터링(조건 설정) Table에 있는 컬럼을 조건 설정
SELECT	Table에 저장되어 있는 컬럼 선택 여러 컬럼 명시 가능 col1 AS “별칭”으로 컬럼의 이름도 별칭 지정 가능

SQL 쿼리 구조

```
SELECT  
  컬럼1,  
  컬럼2,  
  컬럼3  
FROM 테이블  
WHERE  
  <조건문>
```

이 순서를 꼭 기억하기!(SELECT - FROM - WHERE)

SQL 쿼리 구조

```
SELECT  
  컬럼1,  
  컬럼2,  
  컬럼3  
FROM 테이블  
WHERE  
  <조건문>
```

이 **순서**를 꼭 기억하기!(SELECT - FROM - WHERE)

```
SELECT  
WHERE  
FROM 불가능
```

SELECT 연습 문제

포켓몬 데이터 저장 형태 떠올리기

이미지	이름	타입	공격력	특수 공격력	...
	피카츄	전기	55	50	
	꼬부기	물	48	50	
	이상해씨	풀	49	65	
	파이리	불	52	60	

SELECT 연습 문제

1. trainer 테이블에 있는 모든 데이터를 보여주는 SQL 쿼리를 작성해주세요

<https://console.cloud.google.com/bigquery>

위 BigQuery Console로 접근해 문제 풀이!

SELECT 연습 문제

2. trainer 테이블에 있는 트레이너의 name을 출력하는 쿼리를 작성해주세요

SELECT 연습 문제

3. trainer 테이블에 있는 트레이너의 name, age를 출력하는 쿼리를 작성해주세요

SELECT 연습 문제

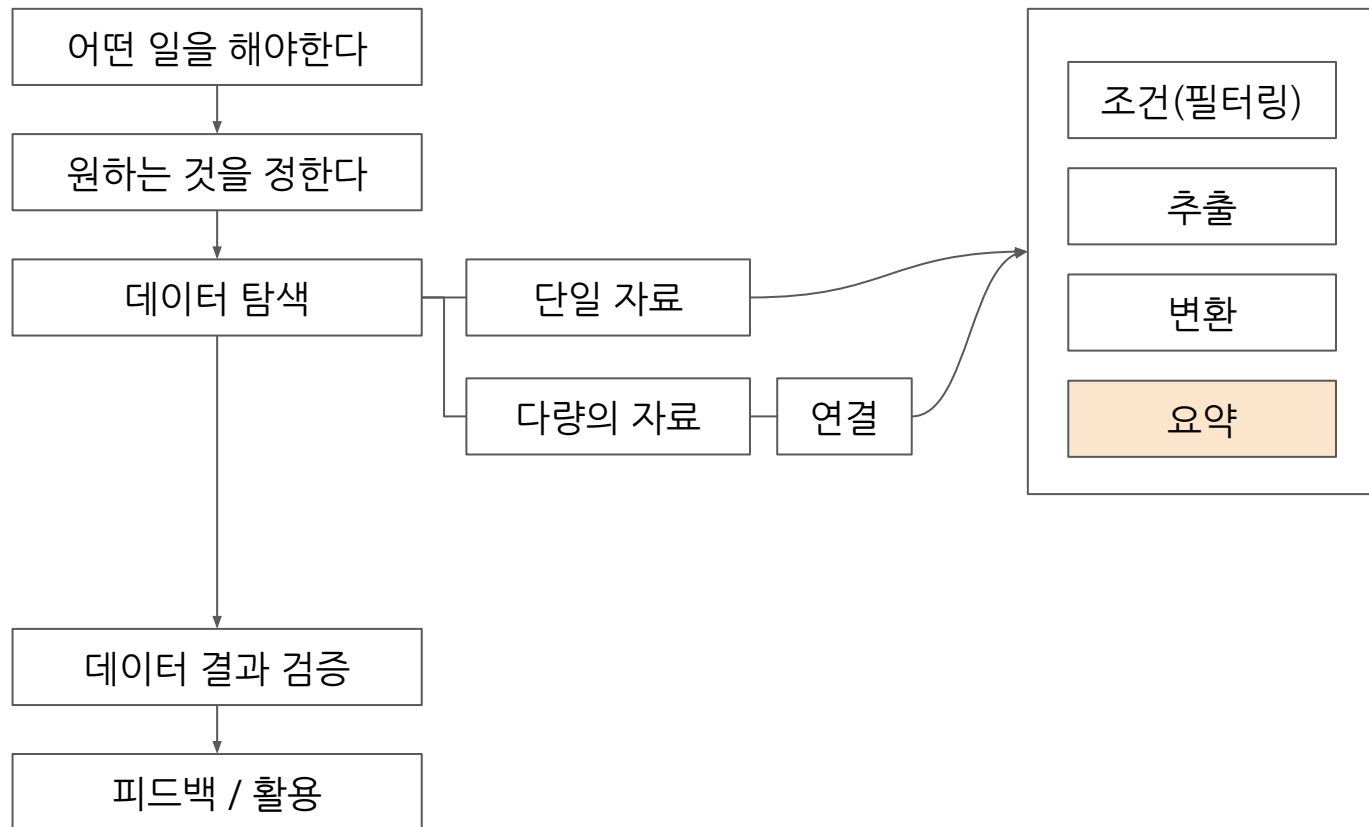
4. trainer의 테이블에서 id가 3인 트레이너의 name, age, hometown을 출력하는 쿼리를 작성해주세요

SELECT 연습 문제

5. pokemon 테이블에서 “피카츄”의 공격력과 체력을 확인할 수 있는 쿼리를 작성해주세요

데이터 탐색 : 요약(집계, 그룹화)
- COUNT, DISTINCT, GROUP BY

데이터를 활용하는 과정



집계와 그룹화

집계하다 : 모을 집, 계산할 계

모아서 계산하다
(=그룹화)해서 계산하다

집계와 그룹화 : 계산하다

집계하다 : 모을 집, 계산할 계

모아서 계산하다
(=그룹화)해서 계산하다

계산

- 더하기, 빼기
- 최대값, 최소값
- 평균
- 갯수 세기

집계 : GROUP BY

GROUP BY : 같은 값끼리 모아서 그룹화한다

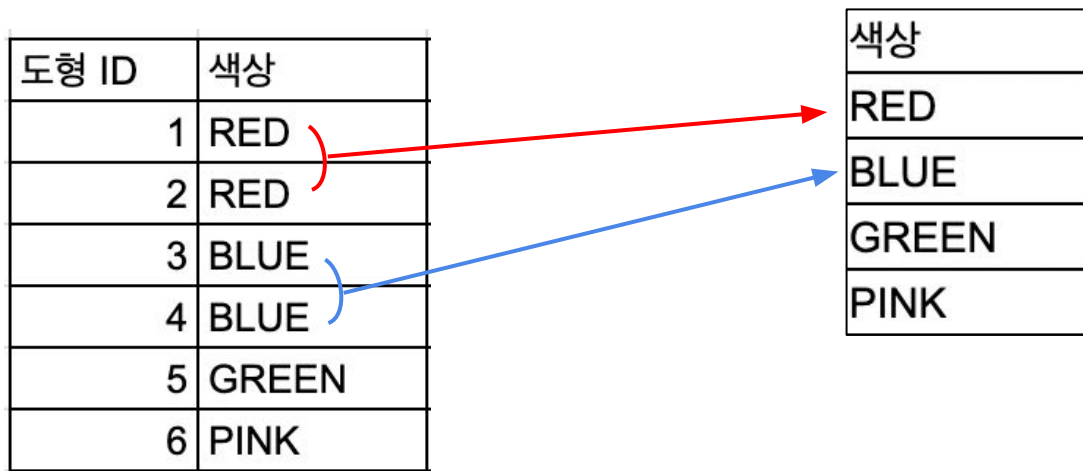
색상 기준으로 모은다! => GROUP BY

도형 ID	색상
1	RED
2	RED
3	BLUE
4	BLUE
5	GREEN
6	PINK

집계 : GROUP BY

GROUP BY : 같은 값끼리 모아서 그룹화한다

색상 기준으로 모은다! => GROUP BY



집계 : GROUP BY

GROUP BY : 같은 값끼리 모아서 그룹화한다

특정 컬럼을 기준으로 모으면서 다른 컬럼에선 집계 가능(합, 평균, MAX, MIN 등)

도형 ID	색상	가로	세로
1	RED	10	7
2	RED	13	21
3	BLUE	30	21
4	BLUE	5	3
5	GREEN	5	3
6	PINK	7	7

색상	가로의 합	세로의 합
RED	23	28
BLUE	35	24
GREEN	5	3
PINK	7	7

집계와 그룹화 : 집계

“타입”을 기준으로 그룹화해서 “평균 공격력” 집계하기

이미지	이름	타입	공격력	특수 공격력	...
	피카츄	전기	55	50	
	꼬부기	물	48	50	
	이상해씨	풀	49	65	
	파이리	불	52	60	
...					

집계와 그룹화 : 집계

“타입”을 기준으로 그룹화해서 “평균 공격력” 집계하기

타입	평균 공격력
전기	61
물	69
플	65
불	81

집계와 그룹화 : 집계

“타입”을 기준으로 그룹화해서 “평균 공격력” 집계하기
+ “타입”을 기준으로 그룹화해서 “타입 별 포켓몬 수” 집계하기

타입	평균 공격력
전기	61
물	69
풀	65
불	81

집계와 그룹화 : 집계

“타입”을 기준으로 그룹화해서 “평균 공격력” 집계하기
+ “타입”을 기준으로 그룹화해서 “타입 별 포켓몬 수” 집계하기

타입	평균 공격력	포켓몬 수
전기	61	15
물	69	46
풀	65	21
불	81	20

집계와 그룹화 : 집계

평균 공격력이 제일 높은 타입이 궁금한 경우

- 평균 공격력 내림차순(큰 것부터 작은 것으로 내려감)으로 정렬

타입	평균 공격력	포켓몬 수
전기	61	15
물	69	46
풀	65	21
불	81	20

정렬하기

평균 공격력이 제일 높은 타입이 궁금한 경우

- 평균 공격력 내림차순(큰 것부터 작은 것으로 내려감)으로 정렬

타입	평균 공격력	포켓몬 수
비행	94	9
드래곤	94	3
바위	86	13
땅	82	11

그룹화 한 값에 조건 설정하기

타입 당 포켓몬 수가 10마리 이상인 데이터만 추출하고 싶은 경우

타입	평균 공격력	포켓몬 수
물	69	46
노말	65	37
벌레	66	22
풀	65	21

앞에서 나온 것을 SQL로 표현해보기

```
SELECT  
  집계할_컬럼1,  
  집계 함수(COUNT, MAX, MIN 등)  
FROM Table  
GROUP BY  
  집계할_컬럼1
```

집계할 컬럼을 SELECT에 명시하고
그 컬럼을 꼭 GROUP BY에 작성

집계 함수 종류

https://cloud.google.com/bigquery/docs/reference/standard-sql/aggregate_functions

Function list

Name	Summary
ANY_VALUE	Gets an expression for some row.
ARRAY_AGG	Gets an array of values.
ARRAY_CONCAT_AGG	Concatenates arrays and returns a single array as a result.
AVG	Gets the average of non-NULL values.
BIT_AND	Performs a bitwise AND operation on an expression.
BIT_OR	Performs a bitwise OR operation on an expression.
BIT_XOR	Performs a bitwise XOR operation on an expression.
COUNT	Gets the number of rows in the input, or the number of rows with an expression evaluated to any value other than NULL.
COUNTIF	Gets the count of TRUE values for an expression.
GROUPING	Checks if a groupable value in the <code>GROUP BY</code> clause is aggregated.
LOGICAL_AND	Gets the logical AND of all non-NULL expressions.
LOGICAL_OR	Gets the logical OR of all non-NULL expressions.
MAX	Gets the maximum non-NULL value.
MAX_BY	Synonym for <code>ANY_VALUE(x HAVING MAX y)</code> .
MIN	Gets the minimum non-NULL value.
MIN_BY	Synonym for <code>ANY_VALUE(x HAVING MIN y)</code> .
STRING_AGG	Concatenates non-NULL <code>STRING</code> or <code>BYTES</code> values.
SUM	Gets the sum of non-NULL values.

DISTINCT : 고유값을 알고 싶은 경우

DISTINCT의 뜻 : 별개의
여러 값 중에 Unique한 것만 보고 싶은 경우 사용

DISTINCT : 고유값을 알고 싶은 경우

DISTINCT의 뜻 : 별개의
여러 값 중에 Unique한 것만 보고 싶은 경우 사용

예시

- 1, 2, 3, 3, 4 => DISTINCT하면?

DISTINCT : 고유값을 알고 싶은 경우

DISTINCT의 뜻 : 별개의
여러 값 중에 Unique한 것만 보고 싶은 경우 사용

예시

- 1, 2, 3, 3, 4 => DISTINCT하면? => 1, 2, 3, 4

DISTINCT는 중복을 제거하는 것

DISTINCT : 고유값을 알고 싶은 경우

SELECT
집계할 컬럼,
COUNT(DISTINCT count할-컬럼)
FROM table
GROUP BY
집계할 컬럼

메인 페이지 VIEW 수는? :

메인 페이지 VIEW 한 유저의 수는? :

user_id	event	event_date
1	view_main_page	2024-04-02
1	view_main_page	2024-04-02
2	view_main_page	2024-04-02
3	view_main_page	2024-04-02

DISTINCT : 고유값을 알고 싶은 경우

SELECT
집계할 컬럼,
COUNT(DISTINCT count할-컬럼)
FROM table
GROUP BY
집계할 컬럼

메인 페이지 VIEW 수는? : 4번

COUNT(user_id)

메인 페이지 VIEW 한 유저의 수는? : 3명

COUNT(DISTINCT user_id)

user_id	event	event_date
1	view_main_page	2024-04-02
1	view_main_page	2024-04-02
2	view_main_page	2024-04-02
3	view_main_page	2024-04-02

GROUP BY 연습 문제

1. pokemon 테이블에 있는 포켓몬 수를 구하는 쿼리를 작성해주세요

GROUP BY 연습 문제

2. 포켓몬의 수가 세대별로 얼마나 있는지 알 수 있는 쿼리를 작성해주세요

그룹화(집계) 활용 포인트

데이터 분석하다가 그룹화하는 경우

- 일자별 집계(원본 데이터는 특정 시간에 어떤 유저가 한 행동이 기록, 일자별로 집계)
- 연령대별 집계(특정 연령대에서 더 많이 구매했는가?)
- 특정 타입별 집계(특정 제품 타입을 많이 구매했는가?)
- 앱 화면별 집계(어떤 화면에 유저가 많이 접근했는가?)

... 무궁무진!

조건을 설정하고 싶은 경우 : WHERE

WHERE

- Table에 바로 조건을 설정하고 싶은 경우 사용

Raw Data인 테이블 데이터에서 조건 설정

SELECT

컬럼1, 컬럼2,

COUNT(컬럼1) AS col1_count

FROM <table>

WHERE

컬럼1 >= 3

조건을 설정하고 싶은 경우 : HAVING

HAVING

- **GROUP BY**한 후 조건을 설정하고 싶은 경우 사용

SELECT

컬럼1, 컬럼2,

COUNT(컬럼1) AS col1_count

FROM <table>

GROUP BY 컬럼1, 컬럼2

HAVING

col1_count >3

조건을 설정하고 싶은 경우 : WHERE이랑 HAVING의 차이는?

WHERE

이미지	이름	타입	공격력	특수 공격력	...
	피카츄	전기	55	50	
	꼬부기	물	48	50	
	이상해씨	풀	49	65	
	파이리	불	52	60	



GROUP BY + HAVING

타입	평균 공격력	포켓몬 수
전기	61	15
물	69	46
풀	65	21
불	81	20

조건을 설정하고 싶은 경우 : WHERE이랑 HAVING의 차이는?

WHERE

- Table에 바로 조건을 설정하고 싶은 경우 사용

HAVING

- GROUP BY한 후 조건을 설정하고 싶은 경우 사용

SELECT

컬럼1, 컬럼2,

COUNT(컬럼1) AS col1_count

FROM <table>

WHERE

컬럼1 >= 3

GROUP BY 컬럼1, 컬럼2

HAVING

col1_count >3

서브 쿼리

서브 쿼리

- SELECT 문 안에 존재하는 SELECT 쿼리
- FROM 절에 또 다른 SELECT 문을 넣을 수 있음
- 괄호로 묶어서 사용

서브 쿼리를 작성하고, 서브 쿼리 바깥에서 WHERE 조건 설정하는 것
= 서브 쿼리에서 HAVING으로 하는 것

정렬하기 : ORDER BY

```
SELECT  
  col  
FROM  
ORDER BY <컬럼> <순서>
```

순서 : DESC(내림차순), OSC(오름차순 - 보통 Default)

ORDER BY는 쿼리의 맨 마지막(아래)에 두고,
쿼리의 맨 마지막에만 작성하면 됨(중간에 필요없음)

출력 개수 제한하기 : LIMIT

- 쿼리문의 결과 Row 수를 제한하고 싶은 경우 LIMIT 사용

쿼리문의 제일 마지막에 작성

```
SELECT  
  col  
FROM Table  
LIMIT 10
```


GROUP BY 연습 문제

3. 포켓몬의 수를 타입 별로 집계하고, 포켓몬의 수가 10 이상인 타입만 남기는 쿼리를 작성해주세요. 포켓몬의 수가 많은 순으로 정렬해주세요

요약, 집계, 그룹화 정리

집계하고 싶은 경우 :

고유값을 알고 싶은 경우 :

조건을 설정하고 싶은 경우 :

정렬하고 싶은 경우 :

출력 개수를 제한하고 싶은 경우 :

요약, 집계, 그룹화 정리

집계하고 싶은 경우 : GROUP BY + 집계 함수(AVG, MAX 등)

고유값을 알고 싶은 경우 : DISTINCT

조건을 설정하고 싶은 경우 : WHERE / HAVING

정렬하고 싶은 경우 : ORDER BY

출력 개수를 제한하고 싶은 경우 : LIMIT

데이터 탐색 - 조건, 추출, 요약 연습 문제(17개)

데이터 탐색 - 조건, 추출, 요약 연습 문제(1)

1. 포켓몬 중에 type2가 없는 포켓몬의 수를 작성하는 쿼리를 작성해주세요

(힌트) ~가 없다 : 컬럼 IS NULL

데이터 탐색 - 조건, 추출, 요약 연습 문제(2)

2. type2가 없는 포켓몬의 type1과 type1의 포켓몬 수를 알려주는 쿼리를 작성해주세요.
단, type1의 포켓몬 수가 큰 순으로 정렬해주세요

데이터 탐색 - 조건, 추출, 요약 연습 문제(3)

3. type2 상관없이 type1의 포켓몬 수를 알 수 있는 쿼리를 작성해주세요

데이터 탐색 - 조건, 추출, 요약 연습 문제(4)

4. 전설 여부에 따른 포켓몬 수를 알 수 있는 쿼리를 작성해주세요

데이터 탐색 - 조건, 추출, 요약 연습 문제(5)

5. 동명 이인이 있는 이름은 무엇일까요? (한번에 찾으려고 하지 않고 단계적으로 가도 괜찮아요)

데이터 탐색 - 조건, 추출, 요약 연습 문제(6)

6. trainer 테이블에서 “Iris” 트레이너의 정보를 알 수 있는 쿼리를 작성해주세요

데이터 탐색 - 조건, 추출, 요약 연습 문제(7)

7. trainer 테이블에서 “Iris”, “Whitney”, “Cynthia” 트레이너의 정보를 알 수 있는 쿼리를 작성해주세요

(힌트) 컬럼 IN ("Iris", "Whitney", "Cynthia")

데이터 탐색 - 조건, 추출, 요약 연습 문제(8)

8. 전체 포켓몬 수는 얼마나 되나요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(9)

9. 세대(generation) 별로 포켓몬 수가 얼마나 되는지 알 수 있는 쿼리를 작성해주세요

데이터 탐색 - 조건, 추출, 요약 연습 문제(10)

10. type2가 존재하는 포켓몬의 수는 얼마나 되나요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(11)

11. type2가 있는 포켓몬 중에 제일 많은 type1은 무엇인가요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(12)

12. 단일(하나의 타입만 있는) 타입 포켓몬 중 많은 type1은 무엇일까요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(13)

13. 포켓몬의 이름에 "파"가 들어가는 포켓몬은 어떤 포켓몬이 있을까요?

(힌트) 컬럼 LIKE "파%"

데이터 탐색 - 조건, 추출, 요약 연습 문제(14)

14. 뱃지가 6개 이상인 트레이너는 몇 명이 있나요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(15)

15. 트레이너가 보유한 포켓몬(trainer_pokemon)이 제일 많은 트레이너는 누구일까요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(16)

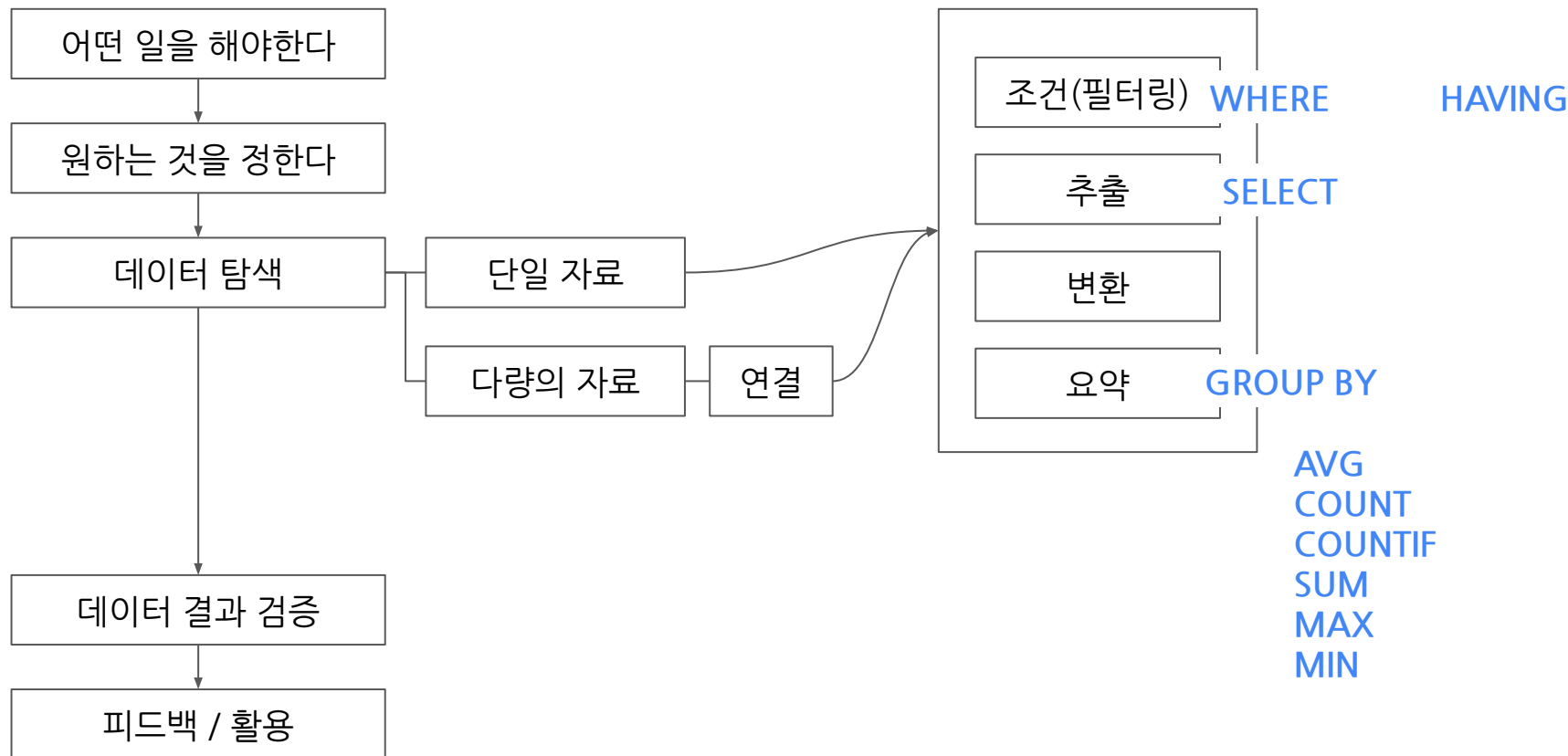
16. 포켓몬을 많이 풀어준 트레이너는 누구일까요?

데이터 탐색 - 조건, 추출, 요약 연습 문제(17)

17. 트레이너 별로 풀어준 포켓몬의 비율이 20%가 넘는 포켓몬 트레이너는 누구일까요?
풀어준 포켓몬의 비율 = (풀어준 포켓몬 수/전체 포켓몬의 수)

(힌트) COUNTIF(조건)

데이터 탐색 - 조건, 추출, 요약 정리



#3. SQL 쿼리 잘 작성하기

카일스쿨



이번 파트에서 다룰 내용

- SQL 쿼리 작성하는 흐름
- 쿼리 작성 템플릿과 생산성 도구
- 오류를 디버깅하는 방법

SQL 쿼리 작성하는 흐름

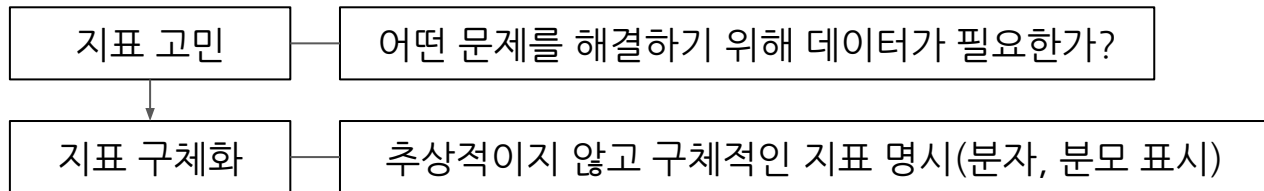
쿼리를 작성하는 흐름

지표 고민

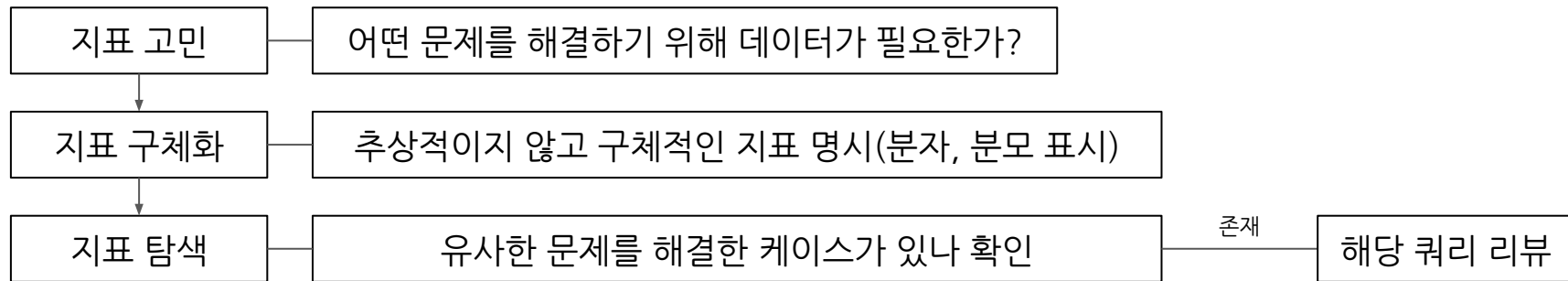
어떤 문제를 해결하기 위해 데이터가 필요한가?



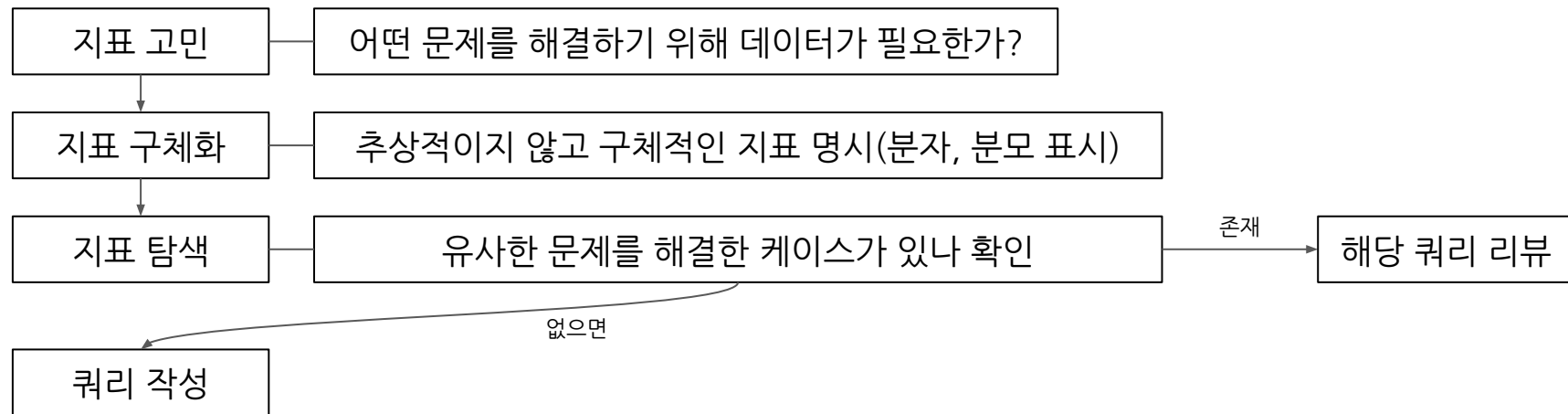
쿼리를 작성하는 흐름



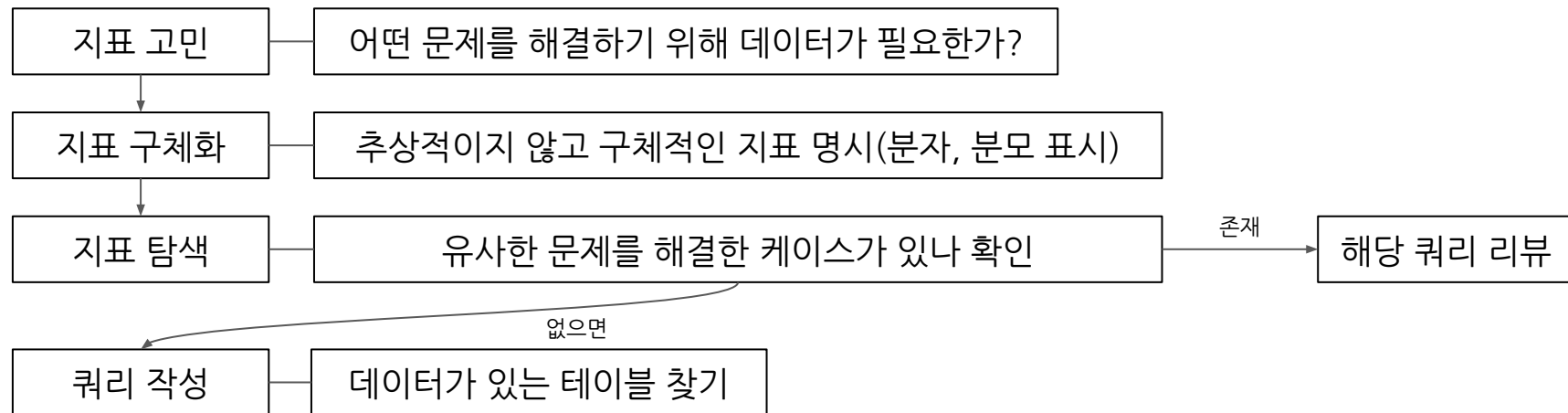
쿼리를 작성하는 흐름



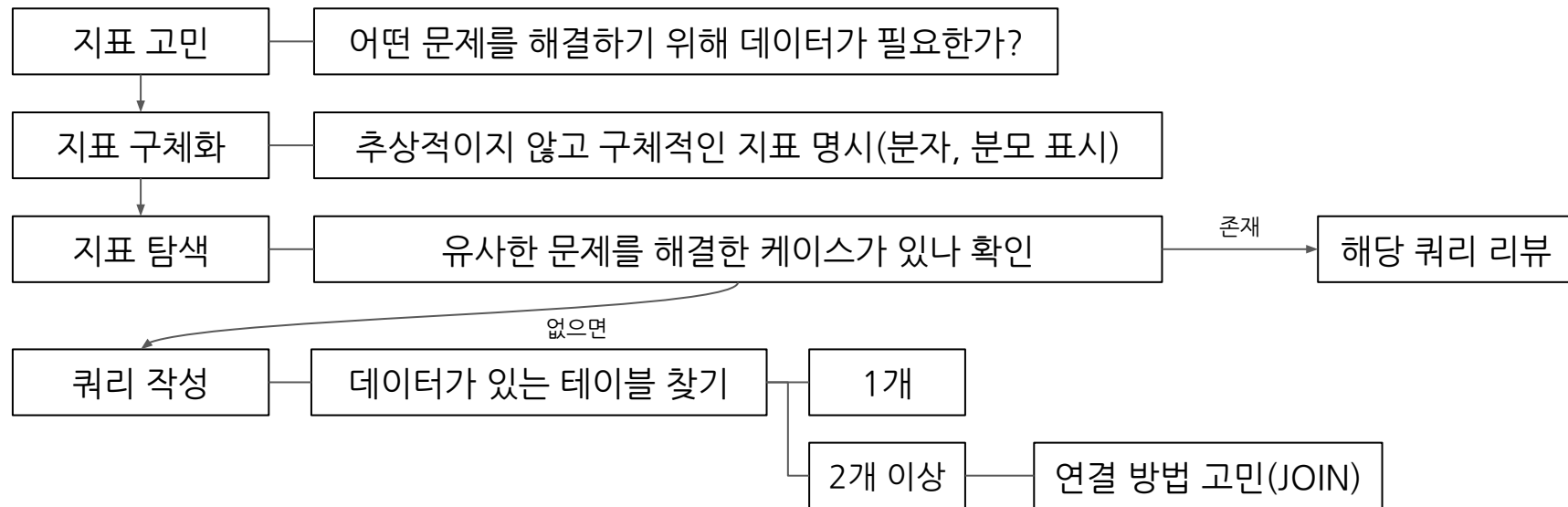
쿼리를 작성하는 흐름



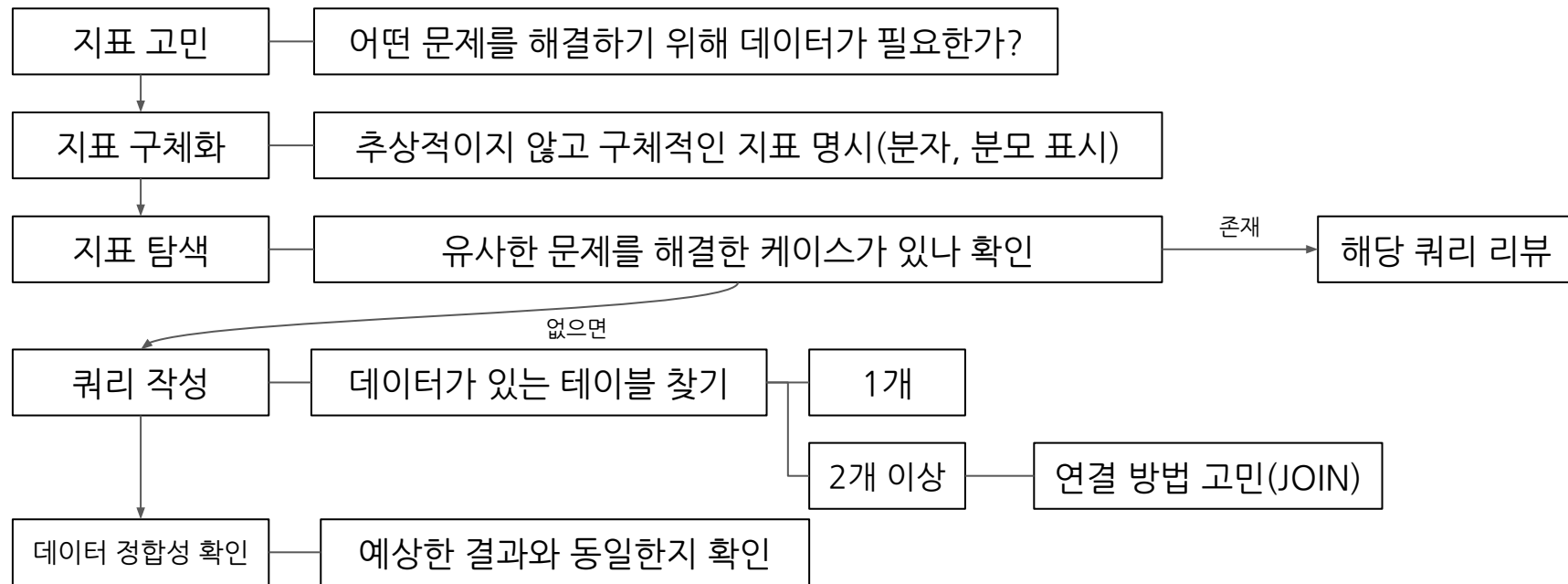
쿼리를 작성하는 흐름



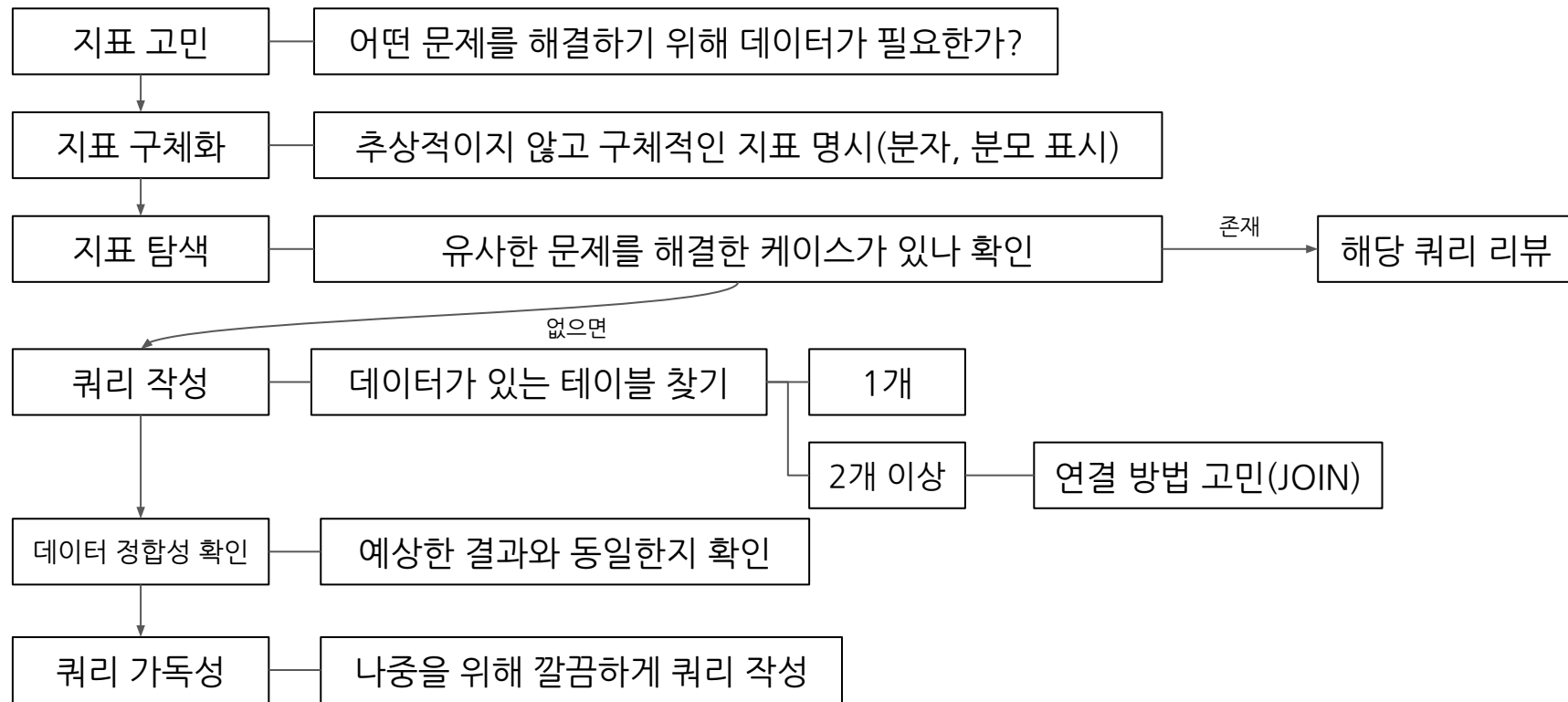
쿼리를 작성하는 흐름



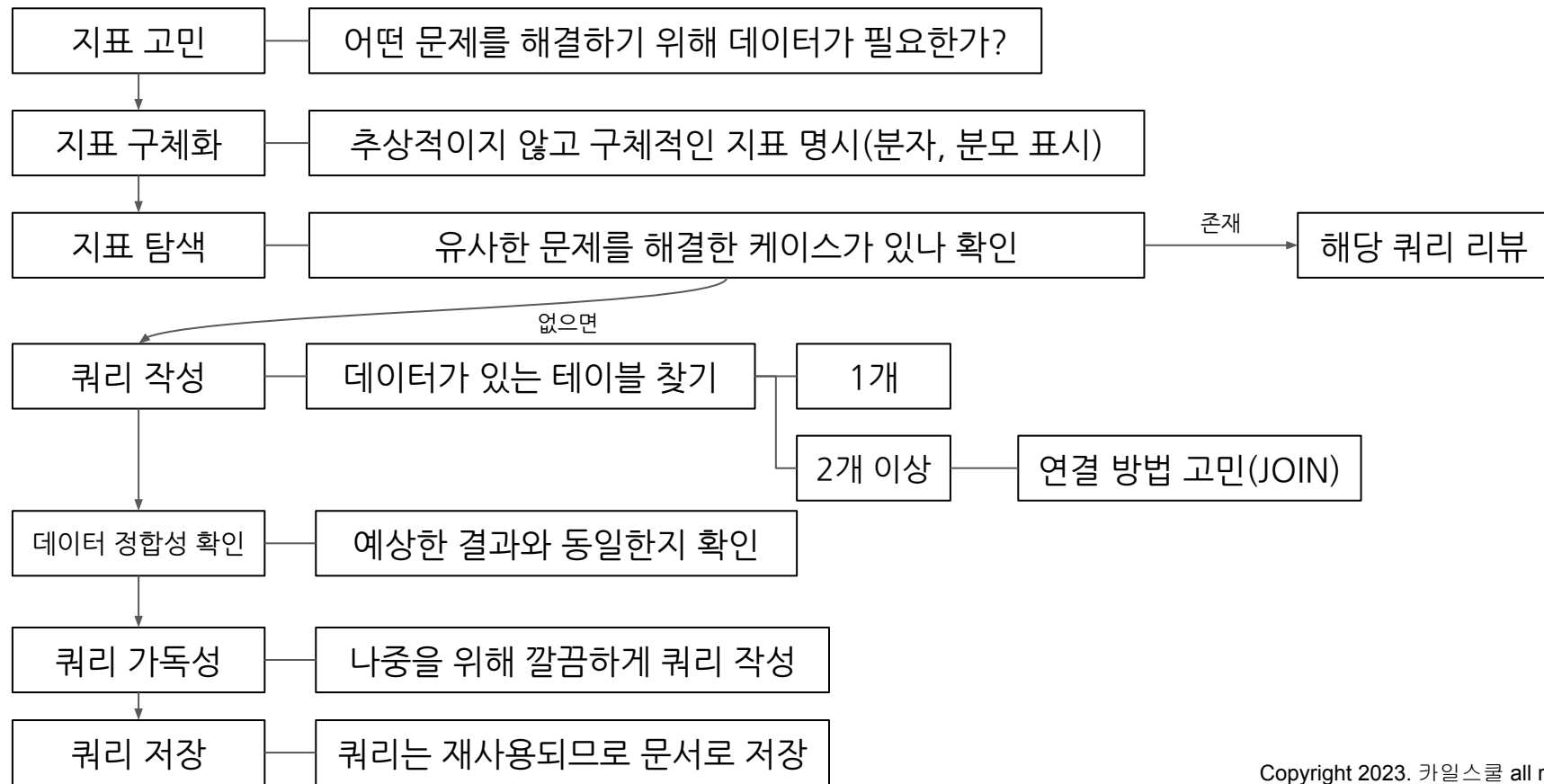
쿼리를 작성하는 흐름



쿼리를 작성하는 흐름



쿼리를 작성하는 흐름



쿼리 작성 템플릿과 생산성 도구

쿼리 작성 템플릿

쿼리를 작성하는 목표, 확인할 지표 :

쿼리 계산 방법 :

데이터의 기간 :

사용할 테이블 :

Join KEY :

데이터 특징 :

SELECT

FROM

WHERE

이렇게 글로 작성하면 쿼리 작성이 더 수월

생산성 도구 : 템플릿 쉽게 사용하기

템플릿을 사용하자!라고 제시하면 생기는 일

생산성 도구 : 템플릿 쉽게 사용하기

템플릿을 사용하자!라고 제시하면 생기는 일

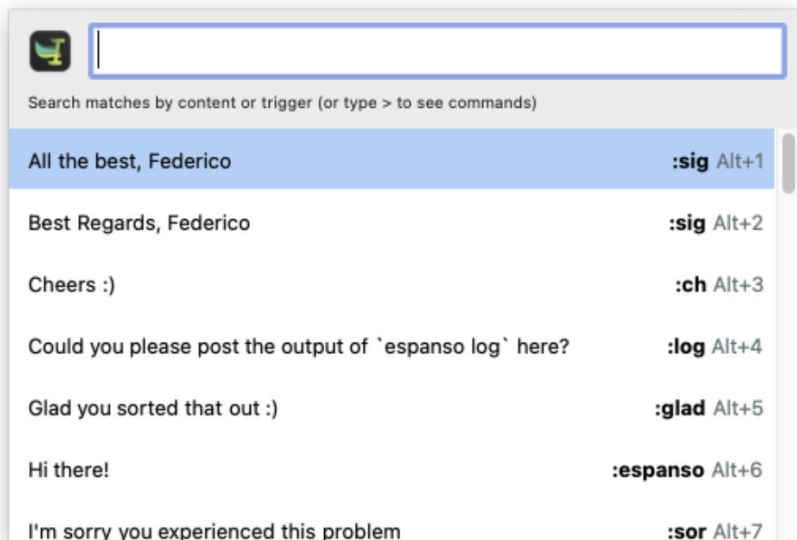
템플릿을 사용하는 것을 까먹음 => 습관 형성이 되지 않음
이 부분을 개선하기 위해 **생산성 도구**를 활용



생산성 도구 : Espanso

<https://espanso.org/>

- 윈도우10, Mac, Linux 모두 사용 가능한 프로그램(무료)
- 특정 단어를 입력하면 원하는 문장(템플릿)으로 변경



<https://espanso.org/docs/get-started/>

생산성 도구 : Espanso 설치

<https://espanso.org/install/>

- 자신의 OS에 맞게 설치
- 권한 설정 등 기본 설정으로 진행



Windows

Installer (64-bit)

Portable zip (64-bit)

Version v2.1.8



macOS

Intel

M1 (Apple Silicon)

Version v2.1.8



Linux

X11

Wayland

Which version should you choose?

Version v2.1.8

생산성 도구 : Espanso 핵심 로직

특정 단어가 감지되면 정의된 것으로 바꾼다!

Match

trigger “:date”
replace “October 11, 2021”

생산성 도구 : Espanso 경로 확인하기

Espanso 설정 파일 경로 확인(설치 방식에 따라 다를 수 있음)

- Mac : /Users/user/Library/Application Support/espanso/match
- 윈도우 : C:\Users\user\AppData\Roaming\espanso/match

- 터미널, 명령 프롬프트, Powershell 등에서 espanso path를 확인하면 정확한 주소를 알 수 있음

- 터미널에서 “espanso edit”을 입력하면 바로 수정 파일(base.yml)로 접근

생산성 도구 : Espanso 설정 수정하기(base.yml)

espanso edit를 터미널에서 입력해서 base.yml 파일 수정(텍스트 편집기에서 수정 추천)

```
Users > kyle > Library > Application Support > expanso > match > ! base.yml
1  # expanso match file
2
3  # For a complete introduction, visit the official docs at: https://espanso.org/docs/
4
5  # You can use this file to define the base matches (aka snippets)
6  # that will be available in every application when using expanso.
7
8  # Matches are substitution rules: when you type the "trigger" string
9  # it gets replaced by the "replace" string.
10 matches:
11   # Simple text replacement
12   - trigger: ":expanso"
13     | replace: "Hi there!"
14
15   # NOTE: expanso uses YAML to define matches, so pay attention to the indentation!
16
17   # But matches can also be dynamic:
18
19   # Print the current date
20   - trigger: ":date"
21     | replace: "{{mydate}}"
22     | vars:
23       - name: mydate
24         | type: date
25         | params:
26           | format: "%m/%d/%Y"
27
28   # Print the output of a shell command
29   - trigger: ":shell"
30     | replace: "{{output}}"
31     | vars:
32       - name: output
33         | type: shell
34         | params:
35           | cmd: "echo 'Hello from your shell'"
36
37   # And much more! For more information, visit the docs: https://espanso.org/docs/
```

생산성 도구 : Espanso 설정 수정하기(base.yml)

아래 내용 복사 붙여넣기(공백 포함)

```
- trigger: ":sql"
  replace: |
    # 쿼리를 작성하는 목표, 확인할 지표 :
    # 쿼리 계산 방법 :
    # 데이터의 기간 :
    # 사용할 테이블 :
    # Join KEY :
    # 데이터 특징 :

SELECT

FROM

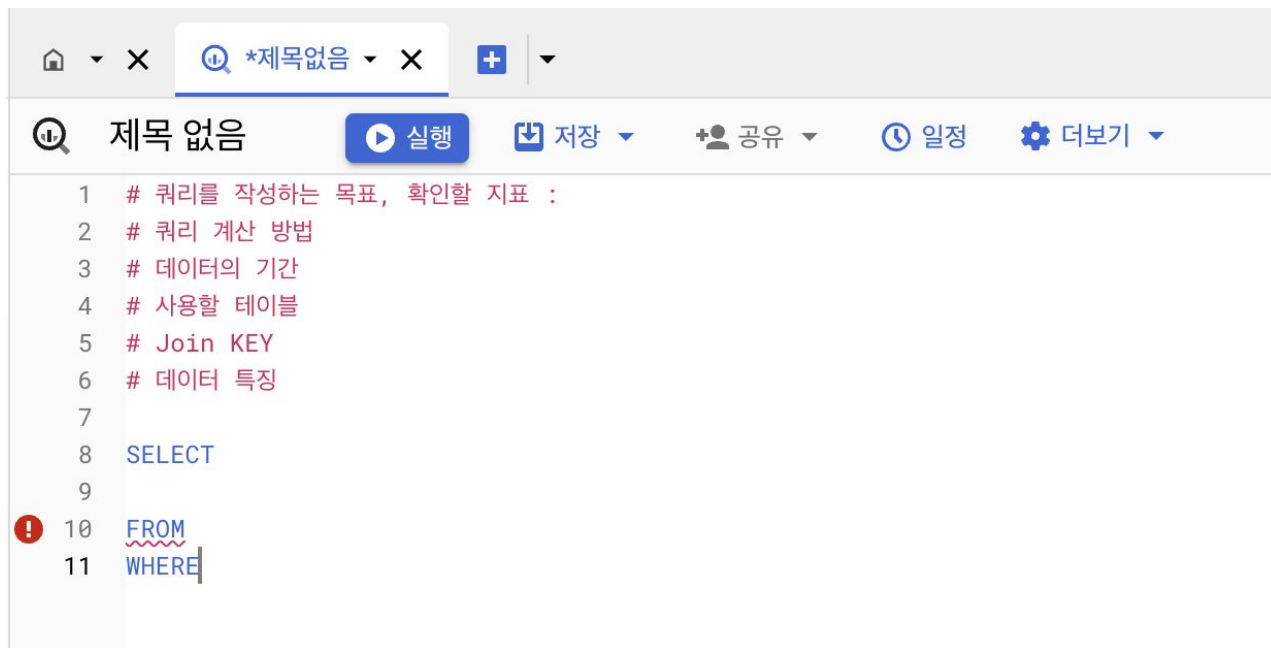
WHERE
```

```
8 # Matches are substitution rules: when you type the
9 # it gets replaced by the "replace" string.
10 ∨ matches:
11   # Simple text replacement
12   ∨ - trigger: ":espanso"
13     |   replace: "Hi there!"
14
15   ∨ - trigger: ":sql"
16     ∨   replace: |
17         |   # 쿼리를 작성하는 목표, 확인할 지표 :
18         |   # 쿼리 계산 방법
19         |   # 데이터의 기간
20         |   # 사용할 테이블
21         |   # Join KEY
22         |   # 데이터 특징
23
24         SELECT
25
26         FROM
27         WHERE
```


생산성 도구 : Espanso 확인 BigQuery Console에서 확인

trigger의 단어를 입력하면 자동으로 변경되는 것을 볼 수 있음

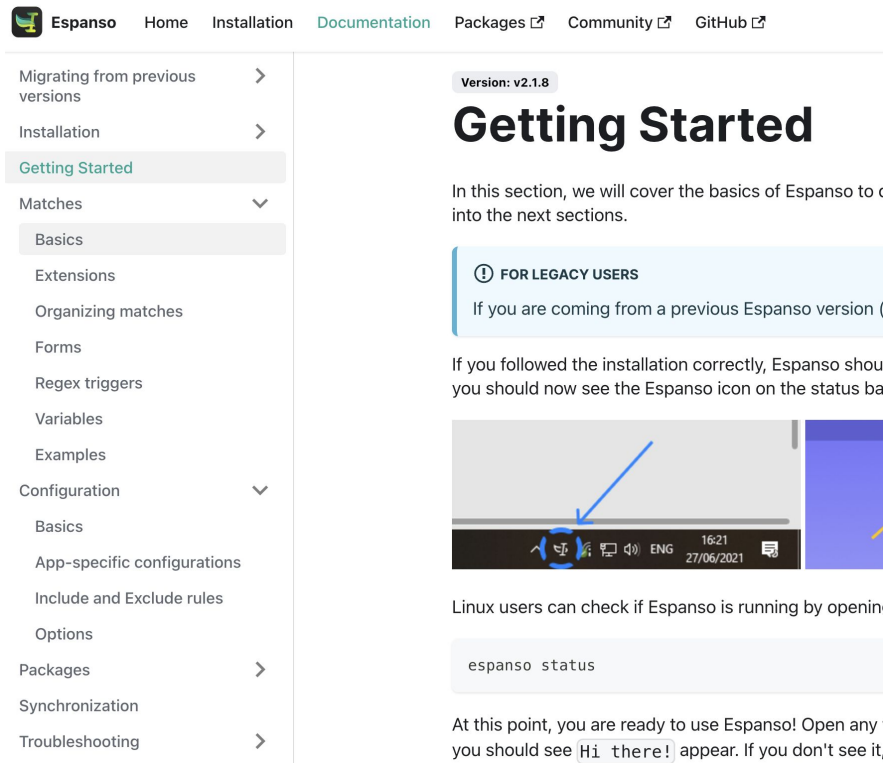
앞으로 쿼리 작성은 trigger 단어 입력, 테이블 FROM 채우기로 시작해보기!



생산성 도구 : Espanso 확인 BigQuery Console에서 확인

본 강의는 Espanso에 대한 강의는 아니므로 이정도만!

자세한 옵션은 아래 문서에서 확인 가능
<https://espanso.org/docs/get-started/>



Version: v2.1.8

Getting Started

In this section, we will cover the basics of Espanso to c into the next sections.

! FOR LEGACY USERS
If you are coming from a previous Espanso version (

If you followed the installation correctly, Espanso should now you should now see the Espanso icon on the status bar



Linux users can check if Espanso is running by opening

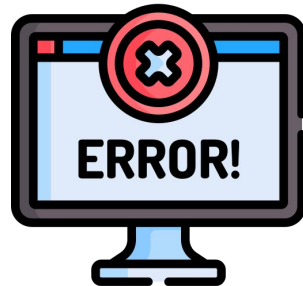
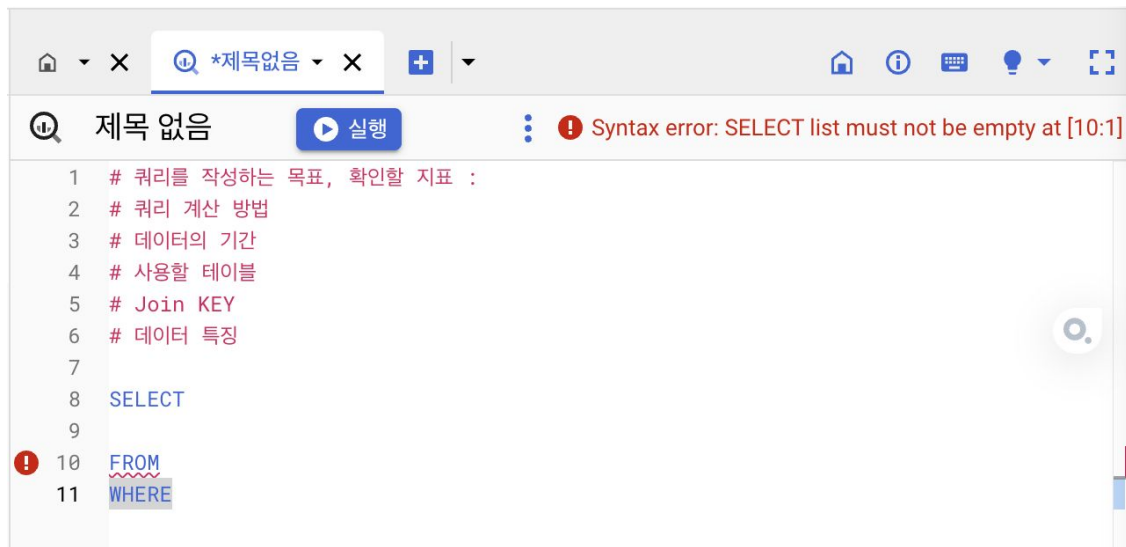
```
espanso status
```

At this point, you are ready to use Espanso! Open any you should see `Hi there!` appear. If you don't see it,

오류를 디버깅하는 방법

SQL 쿼리를 작성하다가 만나는 다양한 오류

- 쿼리를 작성하면서 실행이 안되는 경우를 경험할 것



오류(Error)의 정의

(위키피디아)

- “방황하다, 길을 잃다”라는 뜻의 라틴어(Errare)에서 유래한 단어
- 부정확하거나 잘못된 행동을 의미
- 실수와 동의어인 경우도 있음

오류(Error)의 정의

(위키피디아)

- “방황하다, 길을 잃다”라는 뜻의 라틴어(Errare)에서 유래한 단어
- 부정확하거나 잘못된 행동을 의미
- 실수와 동의어인 경우도 있음

오류 메시지가 알려주고자 하는 것

- 현재 작성한 방식으로 하면 답을 얻을 수 없어요 (길잡이 역할)
- 이 부분에 문제가 되어요 (문제 진단)

오류를 바라보는 관점

오류가 발생하면 : 아 길잡이가 나를 더 좋은 길로 나아가게 하려는구나!

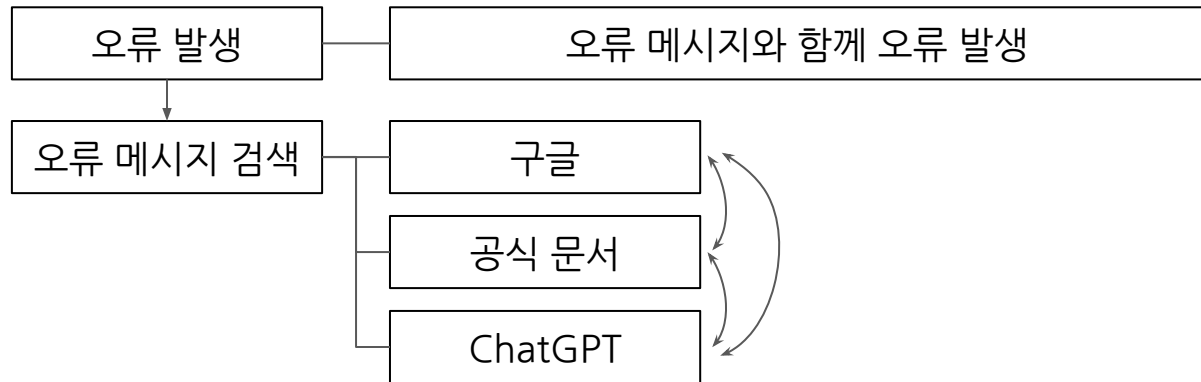
좋은 길로 어떻게 갈까?

- 오류 메시지를 보면서 더 좋은 길로 나가자

BigQuery Error

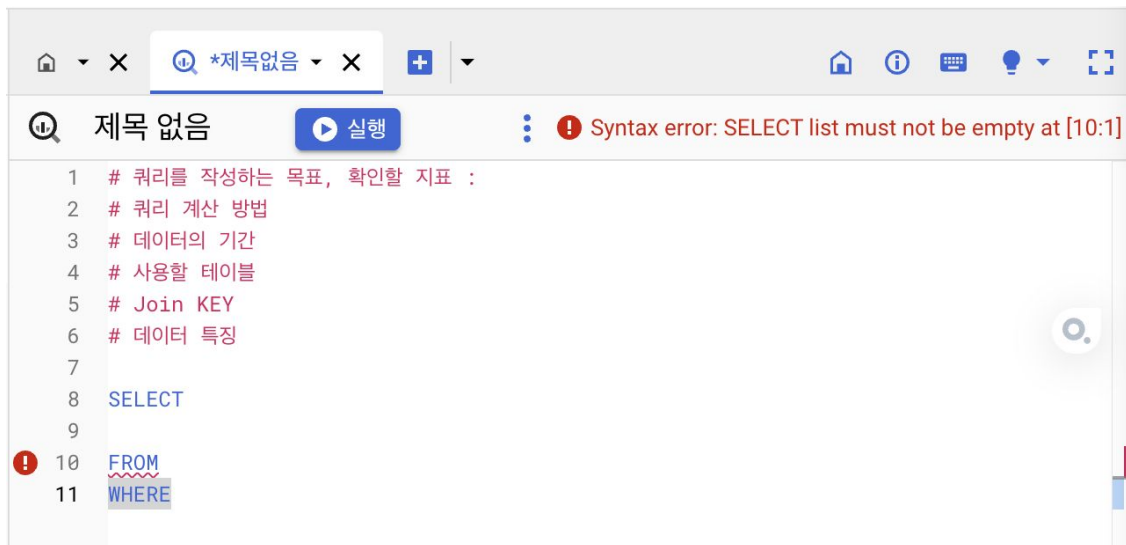
대표적인 오류 카테고리 : Syntax Error (문법 오류)

- 문법을 지키지 않아 생기는 오류
- Error Message를 보고 번역 또는 해석한 후, 해결 방법 찾아보기
 - 구글에 검색
 - ChatGPT에 질문
 - 지인에게 질문(커뮤니티 등)



BigQuery Error : SELECT list must not be empty at [10:1]

- 오류 메시지 번역 : SELECT 목록은 [10:1]에서 비어 있으면 안 됩니다



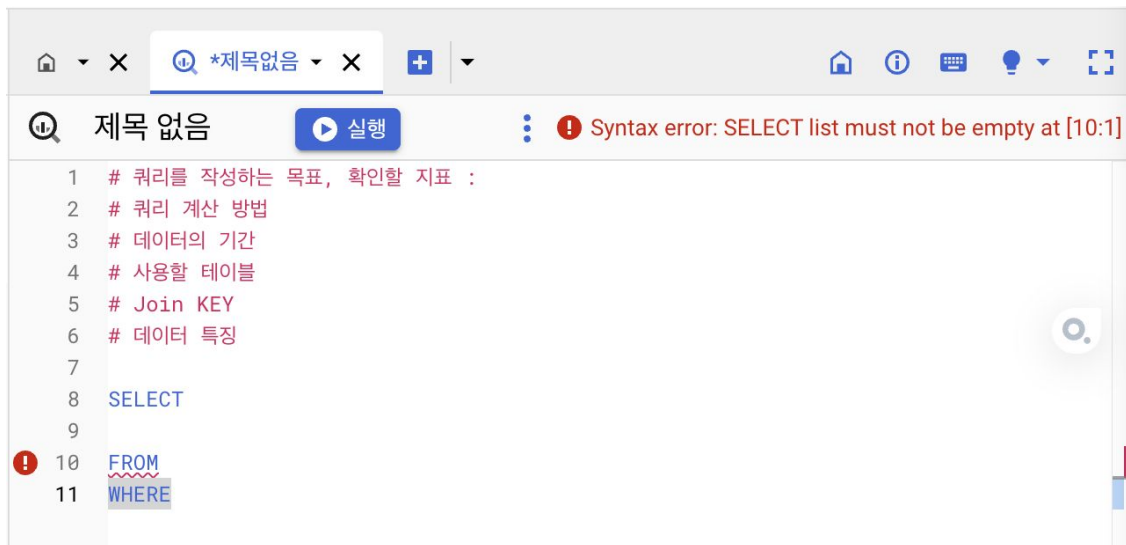
BigQuery Error : SELECT list must not be empty at [10:1]

- 오류 메시지 번역 : SELECT 목록은 [10:1]에서 비어 있으면 안 됩니다

SELECT

col => 이 부분이 비어있기에 생기는 오류

FROM



ChatGPT와 함께 진행하는 오류 디버깅

- 데이터 예시나 쿼리를 제공하고, 오류가 발생한 것을 말해보기
- 그 후에 또 다른 시도 후, 오류가 발생한다고 말하기



BigQuery에서 SELECT list must not be empty at [10:1] 라는 오류를 어떻게 해결할 수 있을까?



쿼리문은 다음과 같아요

SELECT

FROM

WHERE



‘SELECT’ 문이 빈 채로 남아 있어서 발생하는 오류로 보입니다. BigQuery에서는 ‘SELECT’ 문에 적어도 하나의 열을 지정해야 합니다.

예를 들어, 특정 테이블에서 열을 선택하여 데이터를 가져오고자 한다면, 다음과 같이 작성할 수 있습니다:

sql

Copy code

```
SELECT column1, column2, column3
FROM your_table
WHERE condition
```

여기서 ‘column1’, ‘column2’, ‘column3’는 테이블의 열 이름을 나타냅니다. ‘your_table’은 데이터를 가져올 테이블의 이름이며, ‘condition’은 선택적으로 지정된 조건입니다.

따라서 ‘SELECT’ 절에 데이터를 선택할 열을 지정하고, 데이터를 가져올 테이블을 명시해야 합니다. 이렇게 하면 오류를 해결할 수 있습니다.

Number of arguments does not match for aggregate function COUNT

```
SELECT  
  COUNT(id, kor_name)  
FROM basic.pokemon
```

Number of arguments does not match for aggregate function COUNT

```
SELECT  
  COUNT(id, kor_name)  
FROM basic.pokemon
```

해석 : 집계 함수 COUNT의 인자 수가 일치하지 않습니다.

COUNT(kor_name, eng_name) => X

SELECT list expression references column type1 a which is neither grouped nor aggregated

```
SELECT
  type1,
  COUNT(id) AS cnt
FROM basic.pokemon
```

SELECT list expression references column type1 a which is neither grouped nor aggregated

```
SELECT
  type1,
  COUNT(id) AS cnt
FROM basic.pokemon
```

해석 : SELECT 목록 식은 다음에서 그룹화되거나 집계되지 않은 열을 참조합니다.

GROUP BY에 적절한 컬럼을 명시하지 않았을 경우 발생하는 오류

Syntax error: Expected end of input but got keyword SELECT

```
SELECT
  type1,
  COUNT(id) AS cnt
FROM basic.pokemon
GROUP BY
  type1
```

```
SELECT
  *
FROM basic.trainer
```

Syntax error: Expected end of input but got keyword SELECT

```
SELECT
  type1,
  COUNT(id) AS cnt
FROM basic.pokemon
GROUP BY
  type1
```

```
SELECT
  *
FROM basic.trainer
```

해석 : 입력이 끝날 것으로 예상되었지만 SELECT 키워드가 입력되었습니다.

- SELECT 근처 확인하기
- 하나의 쿼리엔 SELECT가 1개만 있어야 함
- 혹은 쿼리가 끝나는 부분에 ; 붙이고 실행할 부분만 드래그 앤 드랍해서 실행하기

Syntax error: Expected end of input but got keyword WHERE at [5:1]

```
SELECT
```

```
  *
```

```
FROM basic.trainer LIMIT 10
```

```
WHERE
```

```
  id = 3
```

Syntax error: Expected end of input but got keyword WHERE at [5:1]

```
SELECT
```

```
  *
```

```
FROM basic.trainer LIMIT 10
```

```
WHERE
```

```
  id = 3
```

해석 : 입력이 끝날 것으로 예상되었지만 [5:1]에서 키워드 WHERE를 얻었습니다.

Syntax error: Expected ")" but got end of script at [8:11]

```
SELECT
  name,
FROM (
  SELECT
    *
  FROM basic.trainer
WHERE
  id = 3
```

Syntax error: Expected ")" but got end of script at [8:11]

```
SELECT
  name,
FROM (
  SELECT
    *
  FROM basic.trainer
WHERE
  id = 3
```

해석 : ")"가 예상되지만 [8:11]에 스크립트가 끝났습니다.

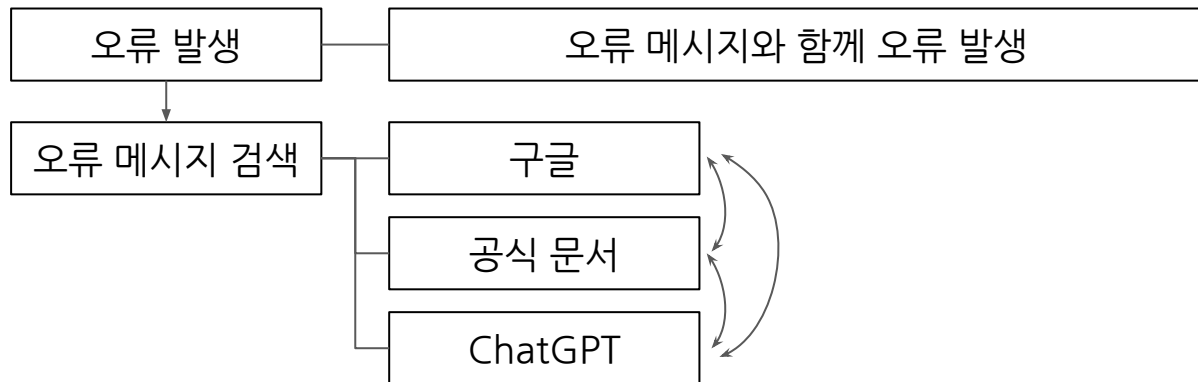
괄호를 작성하지 않은 경우

더 많은 오류 해결 방법을 알고 싶다면

<https://zzsza.github.io/bigquery>

BigQuery 가이드북 문서에 계속 업데이트 예정
(제보도 가능합니다!)

오류 메시지 해결의 핵심



#4. 데이터 탐색 - 변환

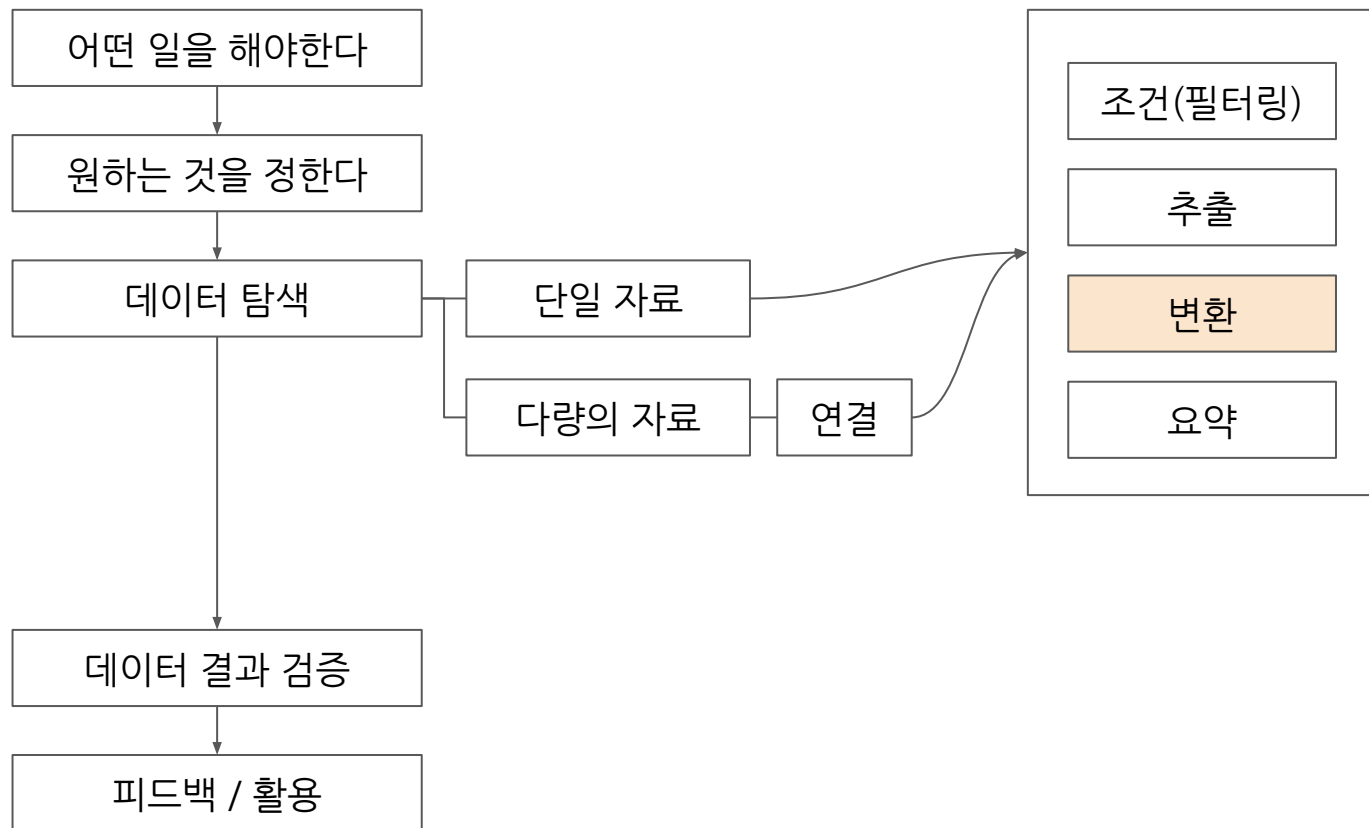
카일스쿨



이번 파트에서 다룰 내용

- 데이터 탐색 : 변환
- 자료형에 따른 여러 함수 소개
 - 문자열
 - 날짜 및 시간 데이터
- 조건문 함수
- BigQuery 공식 문서 확인하는 방법

데이터를 활용하는 과정



데이터 변환

변환을 위한 함수

- SELECT 문에서 데이터를 변환시킬 수 있음
 - 또는 WHERE의 조건문에도 사용할 수도 있음
- 데이터의 **타입**에 따라 다양한 함수가 존재

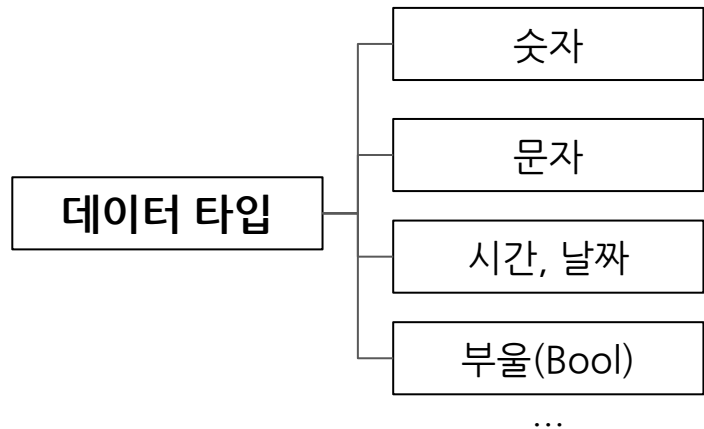
```
SELECT  
  컬럼1,  
  컬럼2,  
  컬럼3
```

```
FROM 테이블
```

```
WHERE <조건문>
```

```
GROUP BY <집계할 컬럼>
```

데이터 타입

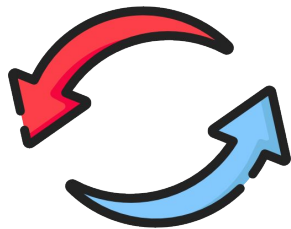


데이터 타입이 중요한 이유

- 보이는 것과 저장된 것의 차이가 존재

데이터 타입이 중요한 이유

- 보이는 것과 저장된 것의 차이가 존재
- 엑셀에서 보면 빈 값 => ""일 수도 있고, NULL일 수도 있음
- 1이라고 작성된 경우 => 숫자 1일 수도 있고, 문자 1일 수도 있음
- 2023-12-31 => DATE 2023-12-31일 수도 있고, 문자 2023-12-31일 수도 있음
- 내 생각과 다른 경우 데이터의 타입을 서로 변경해야 함



자료 타입 변경하기

- 자료 타입을 변경하는 함수 : CAST

```
SELECT  
  CAST(1 AS STRING) # 숫자 1을 문자 1로 변경
```

자료 타입 변경하기

- 자료 타입을 변경하는 함수 : CAST

```
SELECT  
  CAST(1 AS STRING) # 숫자 1을 문자 1로 변경
```

만약 아래 쿼리를 실행하면?

```
SELECT  
  CAST("카일스쿨" AS INT64)
```

자료 타입 변경하기

- 자료 타입을 변경하는 함수 : CAST

```
SELECT  
  CAST(1 AS STRING) # 숫자 1을 문자 1로 변경
```

만약 아래 쿼리를 실행하면?

```
SELECT  
  CAST("카일스쿨" AS INT64)
```

카일스쿨은 숫자로 변경하려고 해도 불가능
=> 오류 발생



The screenshot shows a web-based SQL editor interface. At the top, there's a toolbar with icons for home, close, search, and a plus sign. Below the toolbar, the query text is displayed in a monospace font: `1 SELECT`, `2 CAST("카일스쿨" AS INT64)`, and `3`. To the right of the query text are buttons for "실행" (Execute), "저장" (Save), "공유" (Share), "일정" (Schedule), and "더보기" (More). Below the query text, there's a section titled "쿼리 결과" (Query Result). Under this section, there are tabs for "작업 정보" (Job Info), "결과" (Result), "실행 세부정보" (Execution Details), and "실행 그래프" (Execution Graph). The "결과" tab is selected. At the bottom, there's a red error message box with a red exclamation mark icon and the text: "Bad int64 value: \354\271\264\354\235\274\354\212\244\354\277\250".

자료 타입 변경하기

- 더 안전하게 데이터 타입 변경하기 : **SAFE_CAST**
- **SAFE_**가 붙은 함수는 변환이 실패할 경우 **NULL** 반환

```
SELECT  
SAFE_CAST("카일스쿨" AS INT64)
```

🏠 ▾ ×

🔍 *제목없음 ▾ ×

+

🔍 제목 없음 실행 저장 ▾ 공유 ▾ 일정 더보기 ▾

```
1 SELECT  
2 | CAST("카일스쿨" AS INT64)  
3
```

쿼리 결과

작업 정보 결과 실행 세부정보 실행 그래프

❗ Bad int64 value: \354\271\264\354\235\274\354\212\244\354\277\250

🏠 ▾ ×

🔍 *제목없음 ▾ ×

+

🔍 제목 없음 실행 저장 ▾ 공유 ▾ 일정 더보기 ▾

```
1 SELECT  
2 | SAFE_CAST("카일스쿨" AS INT64)  
3
```

쿼리 결과

작업 정보 결과 JSON 실행 세부정보 차트 미리보기 실행 그래프

행	f0_ ▾	
1		null

수학 함수

- 수학 함수는 수학 연산(평균, 표준편차, 코사인 등)이 존재

Categories

Category	Functions
Trigonometric	ACOS ACOSH ASIN ASINH ATAN ATAN2 ATANH COS COSH COT COTH CSC CSCH SEC SECH SIN SINH TAN TANH
Exponential and logarithmic	EXP LN LOG LOG10
Rounding and truncation	CEIL CEILING FLOOR ROUND TRUNC
Power and root	CBRT POW POWER SQRT
Sign	ABS SIGN
Distance	COSINE_DISTANCE EUCLIDEAN_DISTANCE
Comparison	GREATEST LEAST
Random number generator	RAND
Arithmetic and error handling	DIV IEEE_DIVIDE IS_INF IS_NAN MOD SAFE_ADD SAFE_DIVIDE SAFE_MULTIPLY SAFE_NEGATE SAFE_SUBTRACT
Bucket	RANGE_BUCKET

https://cloud.google.com/bigquery/docs/reference/standard-sql/mathematical_functions

수학 함수

- 수학 함수는 수학 연산(평균, 표준편차, 코사인 등)이 존재

TIP)

나누기를 할 경우

x/y 대신 `SAFE_DIVIDE` 함수 사용하기

`SAFE_DIVIDE(x, y)`

x, y 중 하나라도 0인 경우 그냥 나누면 zero error가 발생

문자열 함수

문자열(String) 함수

문자열 : “안녕하세요”, “카일스쿨”

문자열 데이터로 할 수 있는 대표적인 연산

연산	Input	Output	함수 이름
문자열 붙이기	“안녕” + “하세요”	“안녕하세요”	CONCAT
문자열 분리하기	“가, 나, 다, 라”	“가”, “나”, “다”, “라”	SPLIT
특정 단어 수정하기	“안녕하세요”	“실천하세요”	REPLACE
문자열 자르기	“안녕하세요”	“안녕”	TRIM
영어 대문자 변환	“ab”	“AB”	UPPER

문자열(String) 함수의 예시 쿼리

```
SELECT
  CONCAT("안녕", "하세요") AS concat_example,
  SPLIT("가, 나, 다, 라", ",") AS split_example,
  REPLACE("안녕하세요", "안녕", "실천") AS replace_example,
  TRIM("안녕하세요", "하세요") AS trim_example,
  UPPER("ab") AS upper_example
```

쿼리 결과					
작업 정보 결과 차트 미리보기 JSON 실행 세부정보 실행 그래프					
행	concat_example ▼	split_example ▼	replace_example ▼	trim_example ▼	upper_example ▼
1	안녕하세요	가	실천하세요	하세요	AB
		나			
		다			
		라			

문자열(String) 함수 - CONCAT

문법

CONCAT(컬럼1, 컬럼2, ...)

쿼리 예시

SELECT

CONCAT("안녕", "하세요") AS concat_example

결과 : "안녕하세요"

문자열(String) 함수 - SPLIT

문법

SPLIT(문자열 원본, 나눌 기준이 되는 문자)

쿼리 예시

SELECT

SPLIT("가, 나, 다, 라", ",") AS split_example

결과 : "가", " 나", " 다", " 라" (배열)

문자열(String) 함수 - REPLACE

문법

REPLACE(문자열 원본, 찾을 단어, 바꿀 단어)

쿼리 예시

SELECT

REPLACE("안녕하세요", "안녕", "실천") AS replace_example

결과 : "실천하세요"

문자열(String) 함수 - TRIM

문법

TRIM(문자열 원본, 자를 단어)

쿼리 예시

SELECT

TRIM("안녕하세요", "하세요") AS trim_example

결과 : "안녕"

문자열(String) 함수 - UPPER

문법

UPPER(문자열 원본)

쿼리 예시

SELECT

UPPER("ab") AS upper_example

결과 : "AB"

문자열(String) 함수

연산	Input	Output	함수 이름
문자열 붙이기	“안녕” + “하세요”	“안녕하세요”	CONCAT
문자열 분리하기	“가, 나, 다, 라”	“가”, “나”, “다”, “라”	SPLIT
특정 단어 수정하기	“안녕하세요”	“실천하세요”	REPLACE
문자열 자르기	“안녕하세요”	“안녕”	TRIM
영어 대문자 변환	“ab”	“AB”	UPPER

날짜 및 시간 데이터 이해하기

날짜 및 시간 데이터의 핵심



- 1) 날짜 및 시간 데이터 타입 파악하기 : DATE, DATETIME, TIMESTAMP
- 2) 날짜 및 시간 데이터 관련 알면 좋은 내용 : UTC, Millisecond
- 3) 날짜 및 시간 데이터 타입 변환하기
- 4) 시간 함수(두 시간의 차이, 특정 부분 추출하기)

시간 데이터 다루기

- 시간 데이터도 세부적으로 나눌 수 있음
- DATE, DATETIME, TIMESTAMP 등
- DATE : DATE만 표시하는 데이터, 2023-12-31
- DATETIME : DATE와 TIME까지 표시하는 데이터, Time Zone 정보 없음, 2023-12-31 14:00:00

시간 데이터 다루기

- 시간 데이터도 세부적으로 나눌 수 있음
- DATE, DATETIME, TIMESTAMP 등
- DATE : DATE만 표시하는 데이터, 2023-12-31
- DATETIME : DATE와 TIME까지 표시하는 데이터, Time Zone 정보 없음, 2023-12-31 14:00:00
- TIME : 날짜와 무관하게 시간만 표시하는 데이터, 23:59:59.00

시간 데이터 다루기 - 타임존

GMT: Greenwich Mean Time(한국 시간 : GMT +9)

- 영국의 그리니치 천문대(경도 0도)를 기준으로 지역에 따른 시간의 차이를 조정하기 위해 생긴 시간의 구분선(1884년 채택)
- 영국 근처에서 자주 활용

시간 데이터 다루기 - 타임존

GMT: Greenwich Mean Time(한국 시간 : GMT +9)

- 영국의 그리니치 천문대(경도 0도)를 기준으로 지역에 따른 시간의 차이를 조정하기 위해 생긴 시간의 구분선(1884년 채택)
- 영국 근처에서 자주 활용

UTC: Universal Time Coordinated(한국 시간 : UTC+9)

- 국제적인 표준 시간
- 협정 세계시
- 타임존이 존재한다 = 특정 지역의 표준 시간대

시간 데이터 다루기 - 타임존

GMT: Greenwich Mean Time(한국 시간 : GMT +9)

- 영국의 그리니치 천문대(경도 0도)를 기준으로 지역에 따른 시간의 차이를 조정하기 위해 생긴 시간의 구분선(1884년 채택)
- 영국 근처에서 자주 활용

UTC: Universal Time Coordinated(한국 시간 : UTC+9)

- 국제적인 표준 시간
- 협정 세계시
- 타임존이 존재한다 = 특정 지역의 표준 시간대

TIMESTAMP

- 시간 도장
- UTC부터 경과한 시간을 나타내는 값
- Time Zone 정보 있음
- 2023-12-31 14:00:00 UTC

시간 데이터 다루기 - millisecond, microsecond

millisecond(ms)

- 시간의 단위, 천 분의 1초($1,000\text{ms} = 1\text{초}$)
- 우리가 아는 초보다 더 짧은 시간 단위
- 눈을 깜빡이는 시간이 약 100ms
- 빠른 반응이 필요한 분야에서 사용(초보다 더 정확하게)
- Millisecond $\Rightarrow$ TIMESTAMP $\Rightarrow$ DATETIME으로 변경

microsecond(μs)

- $1/1,000\text{ms}$, $1/1,000,000\text{초}$

시간 데이터 다루기 - millisecond, microsecond

예시 :1704176819711ms
- 2024-01-02 15:26:59(DATETIME)

```
SELECT
TIMESTAMP_MILLIS(1704176819711) AS milli_to_timestamp_value,
TIMESTAMP_MICROS(1704176819711000) AS micro_to_timestamp_value,
DATETIME(TIMESTAMP_MICROS(1704176819711000)) AS datetime_value;
```

쿼리 결과

결과 저장

작업 정보	결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	milli_to_timestamp_value	micro_to_timestamp_value	datetime_value			
1	2024-01-02 06:26:59.711000 U...	2024-01-02 06:26:59.711000 U...	2024-01-02T06:26:59.711000			

시간 데이터 다루기 - millisecond, microsecond

```
1 SELECT
2     TIMESTAMP_MILLIS(1704176819711) AS milli_to_timestamp_value,
3     TIMESTAMP_MICROS(1704176819711000) AS micro_to_timestamp_value,
4     DATETIME(TIMESTAMP_MICROS(1704176819711000)) AS datetime_value,
5     DATETIME(TIMESTAMP_MICROS(1704176819711000), 'Asia/Seoul') AS datetime_value_asia;
```

접근성 옵션을 보려면 Option+F1 키를

쿼리 결과

결과 저장 ▼

데이터 탐색 ▼

작업 정보

결과

차트

미리보기

JSON

실행 세부정보

실행 그래프

행	milli_to_timestamp_value ▼	micro_to_timestamp_value ▼	datetime_value ▼	datetime_value_asia ▼
1	2024-01-02 06:26:59.711000 U...	2024-01-02 06:26:59.711000 U...	2024-01-02T06:26:59.711000	2024-01-02T15:26:59.711000

시간 데이터끼리의 변환

많은 회사들의 Table에 시간이 TIMESTAMP로 저장된 경우가 많음(혹은 DATETIME)
(꼭 회사에 계신 백엔드 개발자에게 물어보기)

- TIMESTAMP $\Leftrightarrow$ DATETIME 변환을 해야할 수 있음

TIMESTAMP와 DATETIME 비교

- 24년 1월 18일 20시 55분(한국 시간)에 실행

SELECT

CURRENT_TIMESTAMP() AS timestamp_col,

DATETIME(CURRENT_TIMESTAMP(), 'Asia/Seoul') AS datetime_col

쿼리 결과			
작업 정보		결과	차트 미리보기
JSON		실행 세부정보	
행	timestamp_col ▼	datetime_col ▼	
1	2024-01-18 11:55:11.536268 UTC	2024-01-18T20:55:11.536268	

TIMESTAMP와 DATETIME 비교

- 24년 1월 18일 20시 55분(한국 시간)에 실행

```
SELECT
  CURRENT_TIMESTAMP() AS timestamp_col,
  DATETIME(CURRENT_TIMESTAMP(), 'Asia/Seoul') AS datetime_col
```

쿼리 결과			
작업 정보		결과	차트 미리보기 JSON 실행 세부정보
행	timestamp_col ▼	datetime_col ▼	
1	2024-01-18 11:55:11.536268 UTC	2024-01-18T20:55:11.536268	

	TIMESTAMP	DATETIME
타임존	UTC라고 나옴	T가 나옴(TIME을 의미)
시간 차이	한국 시간 -9시간	한국 Zone 사용시 한국 시간과 동일

DATETIME 함수 - CURRENT_DATETIME

CURRENT_DATETIME([time_zone]) : 현재 DATETIME 출력

SELECT

```
CURRENT_DATE() AS current_date,  
CURRENT_DATE("Asia/Seoul") AS asia_date,  
CURRENT_DATETIME() AS current_datetime,  
CURRENT_DATETIME("Asia/Seoul") AS current_datetime_asia;
```

쿼리 결과

 결과 저장 ▼

작업 정보	결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	current_date ▼	asia_date ▼	current_datetime ▼	current_datetime_asia ▼		
1	2024-01-22	2024-01-22	2024-01-22T05:01:42.052090	2024-01-22T14:01:42.052090		

DATETIME 함수 - EXTRACT

DATETIME에서 특정 부분만 추출하고 싶은 경우

2024-01-02 14:00:00

EXTRACT

```
EXTRACT(part FROM datetime_expression)
```



Description

Returns a value that corresponds to the specified `part` from a supplied `datetime_expression`.

Allowed `part` values are:

- MICROSECOND
- MILLISECOND
- SECOND
- MINUTE
- HOUR
- DAYOFWEEK : Returns values in the range [1,7] with Sunday as the first day of of the week.
- DAY
- DAYOFYEAR
- WEEK : Returns the week number of the date in the range [0, 53]. Weeks begin with Sunday, and dates prior to the first Sunday of the year are in week 0.
- WEEK(<WEEKDAY>) : Returns the week number of `datetime_expression` in the range [0, 53]. Weeks begin on `WEEKDAY`. `datetime` s prior to the first `WEEKDAY` of the year are in week 0. Valid values for `WEEKDAY` are `SUNDAY`, `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, and `SATURDAY`.
- ISOWEEK : Returns the [ISO 8601 week](#) number of the `datetime_expression`. `ISOWEEK` s begin on Monday. Return values are in the range [1, 53]. The first `ISOWEEK` of each ISO year begins on the Monday before the first Thursday of the Gregorian calendar year.
- MONTH
- QUARTER
- YEAR
- ISOYEAR : Returns the [ISO 8601](#) week-numbering year, which is the Gregorian calendar year containing the Thursday of the week to which `date_expression` belongs.
- DATE
- TIME

[Extract 공식 문서](#)

DATETIME 함수 - EXTRACT

DATETIME에서 특정 부분만 추출하고 싶은 경우

예)

SELECT

```
EXTRACT(DATE FROM DATETIME "2024-01-02 14:00:00") AS date,  
EXTRACT(YEAR FROM DATETIME "2024-01-02 14:00:00") AS year,  
EXTRACT(MONTH FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS month,  
EXTRACT(DAY FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS day,  
EXTRACT(HOUR FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS hour,  
EXTRACT(MINUTE FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS minute,
```

DATETIME 함수 - EXTRACT

DATETIME에서 특정 부분만 추출하고 싶은 경우

예)

SELECT

EXTRACT(DATE FROM DATETIME "2024-01-02 14:00:00") AS date,
EXTRACT(YEAR FROM DATETIME "2024-01-02 14:00:00") AS year,
EXTRACT(MONTH FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS month,
EXTRACT(DAY FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS day,
EXTRACT(HOUR FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS hour,
EXTRACT(MINUTE FROM DATETIME "2024-01-02 14:00:00" AS DATETIME) AS minute,

쿼리 결과

 결과 저장 ▾

 데이터 탐색 ▾

작업 정보 결과 차트 미리보기 JSON 실행 세부정보 실행 그래프							
행	date ▾	year ▾	month ▾	day ▾	hour ▾	minute ▾	
1	2024-01-02	2024	1	2	14	0	

DATETIME 함수 - EXTRACT

요일을 추출하고 싶은 경우

EXTRACT(DAYOFWEEK FROM datetime_col)

- 한 주의 첫날이 일요일인 [1,7] 범위의 값을 반환

EXTRACT 

```
EXTRACT(part FROM date_expression)
```



설명

지정된 날짜 부분에 해당하는 값을 반환합니다. `part` 는 다음 중 하나여야 합니다.

- `DAYOFWEEK` : 한 주의 첫날이 일요일인 [1,7] 범위의 값을 반환합니다.
- `DAY`
- `DAYOFYEAR`
- `WEEK` : [0, 53] 범위에서 날짜의 주 번호를 반환합니다. 주는 일요일부터 시작되며 그 해의 첫 번째 일요일 이전 날짜는 0번째 주에 속합니다.
- `WEEK(<WEEKDAY>)` : [0, 53] 범위에서 해당 날짜의 주 번호를 반환합니다. 주는 `WEEKDAY` 에 시작됩니다. 그 해의 첫 번째 `WEEKDAY` 이전의 날짜는 0번째 주에 속합니다. 유효한 `WEEKDAY` 값은 `SUNDAY`, `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, `SATURDAY` 입니다.
- `ISOWEEK` : `date_expression` 의 `ISO 8601` 주 번호를 반환합니다. `ISOWEEK` 는 월요일에 시작됩니다. 반환 값은 [1, 53] 범위에 속합니다. 각 ISO 연도의 첫 번째 `ISOWEEK` 는 태양력 연도의 첫 번째 목요일 이전의 월요일에 시작됩니다.
- `MONTH`
- `QUARTER` : [1,4] 범위의 값을 반환합니다.
- `YEAR`
- `ISOYEAR` : `date_expression` 이 속한 주의 목요일을 포함하는 태양력 연도인 `ISO 8601` 주 번호 지정 연도를 반환합니다.

DATETIME 함수 - EXTRACT

요일을 추출하고 싶은 경우

EXTRACT(DAYOFWEEK FROM datetime_col)

- 한 주의 첫날이 일요일인 [1,7] 범위의 값을 반환

```
1 SELECT
2   EXTRACT(DAYOFWEEK FROM DATETIME "2024-01-21 14:00:00") AS day_of_week_sun,
3   EXTRACT(DAYOFWEEK FROM DATETIME "2024-01-22 14:00:00") AS day_of_week_mon,
4   EXTRACT(DAYOFWEEK FROM DATETIME "2024-01-23 14:00:00") AS day_of_week_tues,
5   EXTRACT(DAYOFWEEK FROM DATETIME "2024-01-27 14:00:00") AS day_of_week_satur;
```

쿼리 결과



작업 정보

결과

차트

미리보기

JSON

실행 세부정보

실행 그래프

행	day_of_week_sun	day_of_week_mon	day_of_week_tues	day_of_week_satur	
1	1	2	3	7	

DATETIME 함수 - DATETIME_TRUNC

DATE과 HOUR만 남기고 싶은 경우 => 시간 자르기

DATETIME_TRUNC(datetime_col, HOUR)
- "2024-01-02 14:42:13"을 HOUR로 자르면
"2024-01-02 14:00:00"

DATETIME_TRUNC

```
DATETIME_TRUNC(datetime_expression, date_time_part)
```

Description

Truncates a `DATETIME` value to the granularity of `date_time_part`. The `DATETIME` value is always rounded to the beginning of `date_time_part`, which can be one of the following:

- `MICROSECOND`: If used, nothing is truncated from the value.
- `MILLISECOND`: The nearest lessor or equal millisecond.
- `SECOND`: The nearest lessor or equal second.
- `MINUTE`: The nearest lessor or equal minute.
- `HOUR`: The nearest lessor or equal hour.
- `DAY`: The day in the Gregorian calendar year that contains the `DATETIME` value.
- `WEEK`: The first day of the week in the week that contains the `DATETIME` value. Weeks begin on Sundays. `WEEK` is equivalent to `WEEK(SUNDAY)`.
- `WEEK(WEEKDAY)`: The first day of the week in the week that contains the `DATETIME` value. Weeks begin on `WEEKDAY`. `WEEKDAY` must be one of the following: `SUNDAY`, `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, or `SATURDAY`.
- `ISOWEEK`: The first day of the [ISO 8601 week](#) in the ISO week that contains the `DATETIME` value. The ISO week begins on Monday. The first ISO week of each ISO year contains the first Thursday of the corresponding Gregorian calendar year.
- `MONTH`: The first day of the month in the month that contains the `DATETIME` value.
- `QUARTER`: The first day of the quarter in the quarter that contains the `DATETIME` value.
- `YEAR`: The first day of the year in the year that contains the `DATETIME` value.
- `ISOYEAR`: The first day of the [ISO 8601 week-numbering year](#) in the ISO year that contains the `DATETIME` value. The ISO year is the Monday of the first week whose Thursday belongs to the corresponding Gregorian calendar year.

[DATETIME_TRUNC 공식
문서](#)

DATETIME 함수 - DATETIME_TRUNC

예시
SELECT

```
DATETIME "2024-03-02 14:42:13" AS original_data,  
DATETIME_TRUNC(DATETIME "2024-03-02 14:42:13", DAY) AS day_trunc,  
DATETIME_TRUNC(DATETIME "2024-03-02 14:42:13", YEAR) AS year_trunc,  
DATETIME_TRUNC(DATETIME "2024-03-02 14:42:13", MONTH) AS month_trunc,  
DATETIME_TRUNC(DATETIME "2024-03-02 14:42:13", HOUR) AS hour_trunc;
```

쿼리 결과

 결과 저장  데

작업 정보

결과

차트

미리보기

JSON

실행 세부정보

실행 그래프

행	original_data ▼	day_trunc ▼	year_trunc ▼	month_trunc ▼	hour_trunc ▼
1	2024-03-02T14:42:13	2024-03-02T00:00:00	2024-01-01T00:00:00	2024-03-01T00:00:00	2024-03-02T14:00:00

[DATETIME_TRUNC 공식 문서](#)

DATETIME 함수 - PARSE_DATETIME

문자열로 저장된 DATETIME을 DATETIME 타입으로 바꾸고 싶은 경우

PARSE_DATETIME('문자열의 형태', 'DATETIME 문자열') AS datetime

SELECT

PARSE_DATETIME('%Y-%m-%d %H:%M:%S', '2024-01-11 12:35:35') AS parse_datetime;

작업 정보

결과

차트

미리보기

행	parse_datetime ▼	
1	2024-01-11T12:35:35	

DATETIME 함수 - PARSE_DATETIME

[Format Elements 문서](#)를 확인하면 %Y 등이 어떤 의미인지 알 수 있음

외울 필요는 없으며,
필요할 때 보고 적용

Format element	Type	Description	Example
%A	DATE DATETIME TIMESTAMP	The full weekday name (English).	Wednesday
%a	DATE DATETIME TIMESTAMP	The abbreviated weekday name (English).	Wed
%B	DATE DATETIME TIMESTAMP	The full month name (English).	January
%b	DATE DATETIME TIMESTAMP	The abbreviated month name (English).	Jan
%C	DATE DATETIME TIMESTAMP	The century (a year divided by 100 and truncated to an integer) as a decimal number (00-99).	20
%c	DATETIME TIMESTAMP	The date and time representation (English).	Wed Jan 20 21:47:00 2021
%D	DATE DATETIME TIMESTAMP	The date in the format %m/%d/%y.	01/20/21
%d	DATE DATETIME TIMESTAMP	The day of the month as a decimal number (01-31).	20
%e	DATE DATETIME TIMESTAMP	The day of month as a decimal number (1-31); single digits are preceded by a space.	20
%F	DATE DATETIME TIMESTAMP	The date in the format %Y-%m-%d.	2021-01-20

DATETIME 함수 - FORMAT_DATETIME

DATETIME 타입 데이터를 특정 형태의 문자열 데이터로 변환하고 싶은 경우

```
SELECT
  FORMAT_DATETIME("%c", DATETIME "2024-01-11 12:35:35") AS formatted;
```

Format element	Type	Description	Example
%A	DATE DATETIME TIMESTAMP	The full weekday name (English).	Wednesday
%a	DATE DATETIME TIMESTAMP	The abbreviated weekday name (English).	Wed
%B	DATE DATETIME TIMESTAMP	The full month name (English).	January
%b	DATE DATETIME TIMESTAMP	The abbreviated month name (English).	Jan
%C	DATE DATETIME TIMESTAMP	The century (a year divided by 100 and truncated to an integer) as a decimal number (00-99).	20
%c	DATETIME TIMESTAMP	The date and time representation (English).	Wed Jan 20 21:47:00 2021
%D	DATE DATETIME TIMESTAMP	The date in the format %m/%d/%y.	01/20/21
%d	DATE DATETIME TIMESTAMP	The day of the month as a decimal number (01-31).	20
%e	DATE DATETIME TIMESTAMP	The day of month as a decimal number (1-31); single digits are preceded by a space.	20
%F	DATE DATETIME TIMESTAMP	The date in the format %Y-%m-%d.	2021-01-20

DATETIME 함수 - FORMAT_DATETIME

DATETIME 타입 데이터를 특정 형태의 문자열 데이터로 변환하고 싶은 경우

SELECT

FORMAT_DATETIME("%c", DATETIME "2024-01-11 12:35:35") AS formatted;

문자열 => DATETIME : PARSE_DATETIME
DATETIME => 문자열 : FORMAT_DATETIME

Format element	Type	Description	Example
%A	DATE DATETIME TIMESTAMP	The full weekday name (English).	Wednesday
%a	DATE DATETIME TIMESTAMP	The abbreviated weekday name (English).	Wed
%B	DATE DATETIME TIMESTAMP	The full month name (English).	January
%b	DATE DATETIME TIMESTAMP	The abbreviated month name (English).	Jan
%C	DATE DATETIME TIMESTAMP	The century (a year divided by 100 and truncated to an integer) as a decimal number (00-99).	20
%c	DATETIME TIMESTAMP	The date and time representation (English).	Wed Jan 20 21:47:00 2021
%D	DATE DATETIME TIMESTAMP	The date in the format %m/%d/%y.	01/20/21
%d	DATE DATETIME TIMESTAMP	The day of the month as a decimal number (01-31).	20
%e	DATE DATETIME TIMESTAMP	The day of month as a decimal number (1-31); single digits are preceded by a space.	20
%F	DATE DATETIME TIMESTAMP	The date in the format %Y-%m-%d.	2021-01-20

DATETIME 함수 - LAST_DAY

마지막 날을 알고 싶은 경우 : 자동으로 월의 마지막 값을 계산해서 특정 연산을 할 경우

LAST_DAY(DATETIME) : 월의 마지막 값을 반환

```
SELECT
  LAST_DAY(DATETIME '2024-01-03 15:30:00') AS last_day,
  LAST_DAY(DATETIME '2024-01-03 15:30:00', MONTH) AS last_day_month,
  LAST_DAY(DATETIME '2024-01-03 15:30:00', WEEK) AS last_day_week,
  LAST_DAY(DATETIME '2024-01-03 15:30:00', WEEK(SUNDAY)) AS last_day_week_sun,
  LAST_DAY(DATETIME '2024-01-03 15:30:00', WEEK(MONDAY)) AS last_day_week_mon
```

쿼리 결과

결과 저장

작업 정보		결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	last_day ▼	last_day_month ▼	last_day_week ▼	last_day_week_sun	last_day_week_mon		
1	2024-01-31	2024-01-31	2024-01-06	2024-01-06	2024-01-07		

DATETIME 함수 - DATETIME_DIFF

두 DATETIME의 차이를 알고 싶은 경우

DATETIME_DIFF(첫 DATETIME, 두번째 DATETIME, 궁금한 차이)

```
SELECT
  DATETIME_DIFF(first_datetime, second_datetime, DAY) AS day_diff1,
  DATETIME_DIFF(second_datetime, first_datetime, DAY) AS day_diff2,
  DATETIME_DIFF(first_datetime, second_datetime, MONTH) AS month_diff,
  DATETIME_DIFF(first_datetime, second_datetime, WEEK) AS week_diff,
FROM (
  SELECT
    DATETIME "2024-04-02 10:20:00" AS first_datetime,
    DATETIME "2021-01-01 15:30:00" AS second_datetime,
  )
```

DATETIME 함수 - DATETIME_DIFF

두 DATETIME의 차이를 알고 싶은 경우
DATETIME_DIFF(첫 DATETIME, 두번째 DATETIME, 궁금한 차이)

```
SELECT
  DATETIME_DIFF(first_datetime, second_datetime, DAY) AS day_diff1,
  DATETIME_DIFF(second_datetime, first_datetime, DAY) AS day_diff2,
  DATETIME_DIFF(first_datetime, second_datetime, MONTH) AS month_diff,
  DATETIME_DIFF(first_datetime, second_datetime, WEEK) AS week_diff,
FROM (
  SELECT
    DATETIME "2024-04-02 10:20:00" AS first_datetime,
    DATETIME "2021-01-01 15" AS second_datetime
)
```

작업 정보					
결과					
차트 미리보기					
JSON					
실행 세부정보					
실행 그래프					
행	day_diff1 ▼	day_diff2 ▼	month_diff ▼	week_diff ▼	
1	1187	-1187	39	170	

정리

날짜 및 시간 데이터 타입

- DATE
- DATETIME : DATE + TIME. 타임존 정보 X
- TIMESTAMP : 특정 시점에 도장찍은 값. 타임존 정보 O
- UTC : 국제적인 표준 시간. 한국은 UTC+9
- Millisecond : 1/1000초
- Microsecond : 1/1000ms

정리

시간 데이터 타입 변환하기

TIMESTAMP_MILLIS

TIMESTAMP_MICROS

DATETIME

- 문자열 => DATETIME : PARSE_DATETIME
- DATETIME => 문자열 : FORMAT_DATETIME

정리

현재 DATETIME : CURRENT_DATETIME

DATETIME의 특정 부분 추출 : EXTRACT

DATETIME 특정 부분 자르기 : DATETIME_TRUNC

DATETIME 차이 구하기 : DATETIME_DIFF

정리

현재 DATETIME : CURRENT_DATETIME

DATETIME의 특정 부분 추출 : EXTRACT

DATETIME 특정 부분 자르기 : DATETIME_TRUNC

DATETIME 차이 구하기 : DATETIME_DIFF

EXTRACT vs DATETIME_TRUNC

언제 무엇을 사용해야 할까?

이후 데이터 분석에서 어떻게 활용할지에 따라 다름

2024-01-01T00:00:00 => 00:00:00이 필요한지 생각해보기

DATETIME_TRUNC은 시간 자를 때 자주 활용했음(1시간 단위로 수요 집계)

정리

DATETIME 함수 위주로 다루었으나, 대부분 TIMESTAMP, DATE에도 사용할 수 있는 함수

대표적인 함수만 기억하고 필요할 때 찾아서 사용하기

자주 사용하는 함수는 따로 저장해두기

- Millisecond to Datetime
- Parse Datetime

시간 데이터 연습 문제

시간 데이터 연습 문제

1. 트레이너가 포켓몬을 포획한 날짜(catch_date)를 기준으로, 2023년 1월에 포획한 포켓몬의 수를 계산해주세요.

시간 데이터 연습 문제

2. 배틀이 일어난 시간(battle_datetime)을 기준으로, 오전 6시에서 오후 6시 사이에 일어난 배틀의 수를 계산해주세요.

시간 데이터 연습 문제

3. 각 트레이너별로 그들이 포켓몬을 포획한 첫 날(catch_date)을 찾고, 그 날짜를 'DD/MM/YYYY' 형식으로 출력해주세요.
(2024-01-01 => 01/01/2024)

시간 데이터 연습 문제

4. 배틀이 일어난 날짜(battle_date)를 기준으로, 요일별로 배틀이 얼마나 자주 일어났는지 계산해주세요.

시간 데이터 연습 문제

5. 트레이너가 포켓몬을 처음으로 포획한 날짜와 마지막으로 포획한 날짜의 간격이 큰 순으로 정렬하는 쿼리를 작성해주세요.

조건문 함수

조건문 함수

조건문

- 만약 특정 조건이 충족되면, 어떤 행동을 하자
- 특정 조건이 참일때 A, 아니면 B
 - 조건에 따른 분기 처리가 필요한 경우
- 조건에 따라 다른 값을 표시하고 싶을 때 사용

조건문을 사용하는 방법

- 1) CASE WHEN
- 2) IF

조건문 함수가 사용되는 이유

- 데이터 분석을 하다보면, 특정 카테고리를 하나로 합치는 전처리가 필요할 수 있음
- 이런 일이 발생하는 이유
 - 데이터를 저장하는 쪽과 데이터를 분석하는 쪽이 나뉘고(팀 등으로),
 - 분석할 때 필요한 부분에서 조건 설정해서 변경하는 것이 더 유용
 - 저장할 때부터 특정 카테고리를 합쳐서 저장하면, 쪼개서 보고 싶을 때 볼 수 없음

조건문 : 1) CASE WHEN

여러 조건이 있을 경우 유용

문법

SELECT

CASE

WHEN 조건1 THEN 조건1이 참일 경우 결과

WHEN 조건2 THEN 조건2가 참일 경우 결과

ELSE 그 외 조건일 경우 결과

END AS 새로운_컬럼_이름

조건문 : 1) CASE WHEN 예시

포켓몬 데이터 : type1, type2

Rock(바위) 타입과 Ground(땅) 타입이 결국 비슷
“Rock&Ground”라는 타입을 새로 만들면 어떨까?



조건문 : 1) CASE WHEN 예시

포켓몬 데이터 : type1, type2

Rock(바위) 타입과 Ground(땅) 타입이 결국 비슷
“Rock&Ground”라는 타입을 새로 만들면 어떨까?



작업 정보			결과
행	new_type1	cnt	
1	Bug	21	
2	Dark	5	
3	Fire	19	
4	Rock&Ground	34	
5	Fairy	7	
6	Ghost	4	
7	Grass	21	
8	Water	43	
9	Dragon	3	
10	Normal	37	
11	Poison	13	
12	Psychic	15	

조건문 : 1) CASE WHEN 예시

포켓몬 데이터 : type1, type2

Rock(바위) 타입과 Ground(땅) 타입이 결국 비슷
“Rock&Ground”라는 타입을 새로 만들면 어떨까?

```
SELECT
  new_type1,
  COUNT(DISTINCT id) AS cnt
FROM (
  SELECT
    *,
    CASE
      WHEN (type1 IN ("Rock", "Ground")) OR (type2 IN ("Rock", "Ground"))
      THEN "Rock&Ground"
      ELSE type1
    END AS new_type1
  FROM basic.pokemon
)
GROUP BY
  new_type1
```

작업 정보			결과
행	new_type1	cnt	
1	Bug	21	
2	Dark	5	
3	Fire	19	
4	Rock&Ground	34	
5	Fairy	7	
6	Ghost	4	
7	Grass	21	
8	Water	43	
9	Dragon	3	
10	Normal	37	
11	Poison	13	
12	Psychic	15	

조건문 : 1) CASE WHEN 순서

각 포켓몬의 공격력(attack)을 기준으로, 50 이상이면 'Strong', 100 이상이면 'Very Strong', 그 이하면 'Weak'으로 분류해주세요

조건문 : 1) CASE WHEN 순서

각 포켓몬의 공격력(attack)을 기준으로, 50 이상이면 'Strong', 100 이상이면 'Very Strong', 그 이하면 'Weak'으로 분류해주세요

```
SELECT
  eng_name,
  attack,
  CASE
    WHEN attack >= 100 THEN 'Very Strong'
    WHEN attack >= 50 THEN 'Strong'
    ELSE 'Weak'
  END AS attack_level
FROM basic.pokemon
```

조건문 : 1) CASE WHEN 순서

각 포켓몬의 공격력(attack)을 기준으로, 50 이상이면 'Strong', 100 이상이면 'Very Strong', 그 이하면 'Weak'으로 분류해주세요

```
SELECT
  eng_name,
  attack,
  CASE
    WHEN attack >= 100 THEN 'Very Strong'
    WHEN attack >= 50 THEN 'Strong'
    ELSE 'Weak'
  END AS attack_level
FROM basic.pokemon
```

조건문 : 1) CASE WHEN 순서

각 포켓몬의 공격력(attack)을 기준으로, 50 이상이면 'Strong', 100 이상이면 'Very Strong', 그 이하면 'Weak'으로 분류해주세요

```
SELECT
  eng_name,
  attack,
  CASE
    WHEN attack >= 50 THEN 'Strong'
    WHEN attack >= 100 THEN 'Very Strong'
    ELSE 'Weak'
  END AS attack_level
FROM basic.pokemon
```

조건문 : 1) CASE WHEN

🔍 제목 없음 실행 쿼리 완료됨

```
1 SELECT
2   eng_name,
3   attack,
4   CASE
5     WHEN attack >= 100 THEN 'Very Strong'
6     WHEN attack >= 50 THEN 'Strong'
7     ELSE 'Weak'
8   END AS attack_level
9 FROM basic.pokemon
10 ORDER BY attack DESC
```

접근성 옵션을 보려면 Option+F1 키를 누르세요.

쿼리 결과

결과 저장

데이터 탐색



< 작업 정보 결과 차트 미리보기 JSON 실행 세부 >

행	eng_name	attack	attack_level
1	Tyranitar	134	Very Strong
2	Dragonite	134	Very Strong
3	Flareon	130	Very Strong
4	Kingler	130	Very Strong
5	Ursaring	130	Very Strong

🔍 제목 없음 실행 쿼리 완료됨

```
1 SELECT
2   eng_name,
3   attack,
4   CASE
5     WHEN attack >= 50 THEN 'Strong'
6     WHEN attack >= 100 THEN 'Very Strong'
7     ELSE 'Weak'
8   END AS attack_level
9 FROM basic.pokemon
10 ORDER BY attack DESC
```

접근성 옵션을 보려면 Option+F1 키를 누르세요.

쿼리 결과

결과 저장

데이터 탐색



< 작업 정보 결과 차트 미리보기 JSON 실행 세부 >

행	eng_name	attack	attack_level
1	Tyranitar	134	Strong
2	Dragonite	134	Strong
3	Flareon	130	Strong
4	Kingler	130	Strong
5	Ursaring	130	Strong

조건문 : 1) CASE WHEN

```
SELECT  
  CASE  
    WHEN 조건1 THEN 조건1이 참일 경우 결과  
    WHEN 조건2 THEN 조건2가 참일 경우 결과  
    ELSE 그 외 조건일 경우 결과  
  END AS 새로운_컬럼_이름
```

- 조건1, 조건2에 둘 다 해당하면 하면 앞선 순서를 따름
- 문자열 함수(특정 단어 추출)에서 이슈가 자주 발생

조건문 : 2) IF

단일 조건일 경우 유용

문법

- IF(조건문, True일 때의 값, False일 때의 값) AS 새로운_컬럼_이름

SELECT

```
IF(1=1, '동일한 결과', '동일하지 않은 결과') AS result1,  
IF(1=2, '동일한 결과', '동일하지 않은 결과') AS result2
```

작업 정보		결과	차트	미리보기	JSON	실행 세부정보
행		result1 ▼			result2 ▼	
1		동일한 결과			동일하지 않은 결과	

조건문 정리

- CASE WHEN : 여러 조건이 있을 경우 사용. 조건의 순서에 주의
- IF : 단일 조건일 경우 사용

조건문 함수 연습 문제

조건문 함수 연습 문제

1. 포켓몬의 'speed'가 70 이상이면 '빠름', 그렇지 않으면 '느림'으로 표시하는 새로운 컬럼 'Speed_Category'를 만들어 주세요

조건문 함수 연습 문제

2. 포켓몬의 'type1'에 따라 'Water', 'Fire', 'Electric' 타입은 각각 '물', '불', '전기'로, 그 외 타입은 '기타'로 분류하는 새로운 컬럼 'type_Korean'을 만들어 주세요

조건문 함수 연습 문제

3. 각 포켓몬의 총점(total)을 기준으로, 300 이하면 'Low', 301에서 500 사이면 'Medium', 501 이상이면 'High'로 분류해주세요

조건문 함수 연습 문제

4. 각 트레이너의 배지 개수(badge_count)를 기준으로, 5개 이하면 'Beginner', 6개에서 8개 사이면 'Intermediate', 그 이상이면 'Advanced'로 분류해주세요.

조건문 함수 연습 문제

5. 트레이너가 포켓몬을 포획한 날짜(catch_date)가 '2023-01-01' 이후이면 'Recent', 그렇지 않으면 'Old'로 분류해주세요.

조건문 함수 연습 문제

6. 배틀에서 승자(winner_id)가 player1_id와 같으면 'Player 1 Wins', player2_id와 같으면 'Player 2 Wins', 그렇지 않으면 'Draw'로 결과가 나오게 해주세요

데이터 탐색 - 컬럼 변환하기 정리

컬럼 변환하기 정리

```
SELECT
```

```
  컬럼1,
```

```
  컬럼2,
```

```
  컬럼3
```

```
FROM 테이블
```

```
WHERE <조건문>
```

```
GROUP BY <집계할 컬럼>
```

컬럼 변환하기 정리

```
SELECT  
  컬럼1,  
  컬럼2,  
  컬럼3
```

```
FROM 테이블  
WHERE <조건문>  
GROUP BY <집계할 컬럼>
```

데이터 타입

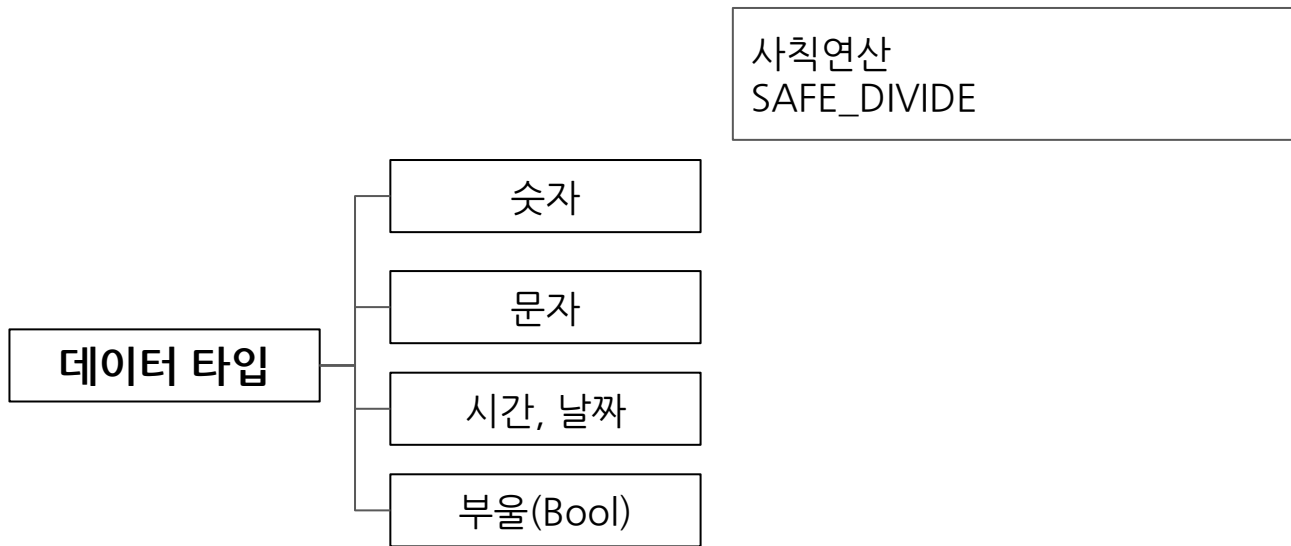
숫자

문자

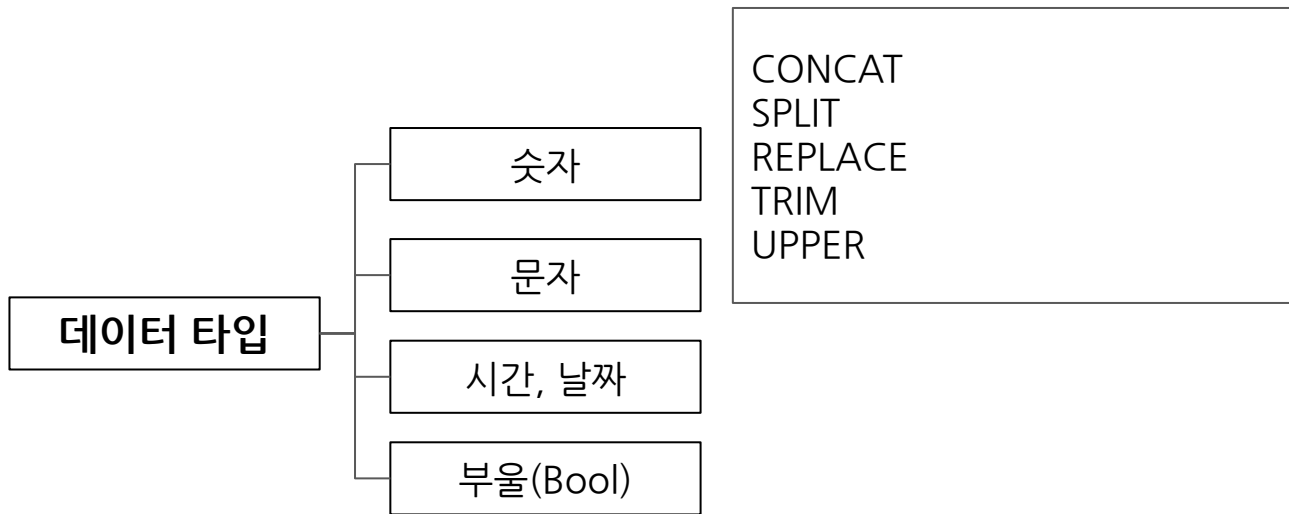
시간, 날짜

부울(Bool)

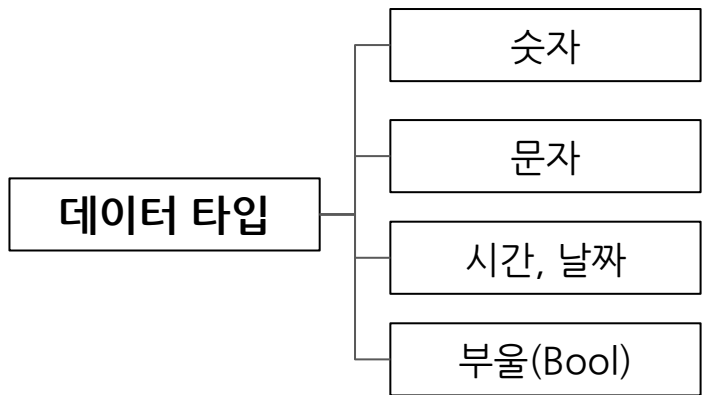
컬럼 변환하기 정리



컬럼 변환하기 정리

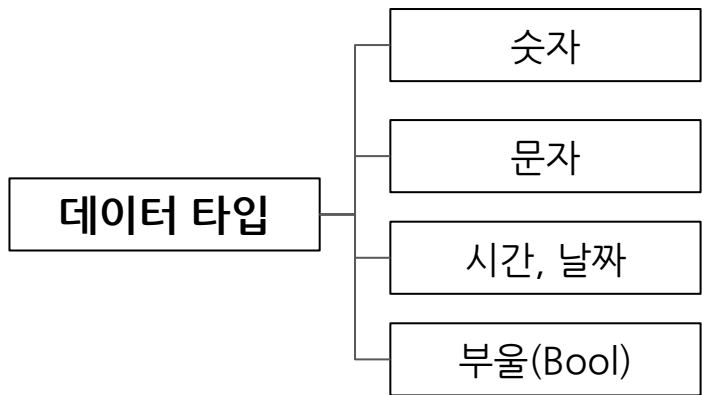


컬럼 변환하기 정리



EXTRACT(HOUR FROM datetime)
DATETIME_TRUNC
PARSE_DATETIME

컬럼 변환하기 정리



데이터 타입 변경하기
조건문 : 특정 조건이면 변경하기
- CASE WHEN
- IF

BigQuery 공식 문서 확인하는 방법

개발 공식 문서

- 프로그래밍 언어, 라이브러리 등은 해당 기술을 어떻게 사용하면 좋은지에 대해 문서를 제공함
- 완성도는 모두 다르지만, 많이 참고하는 참고서 느낌(수학의 정석)
- 처음엔 어려워 보일 수 있지만, 익숙해지면 빠르게 파악할 수 있음
- BigQuery에 해당되는 것은 아니고 대부분의 프로그래밍 관련 내용에 적용할 수 있음

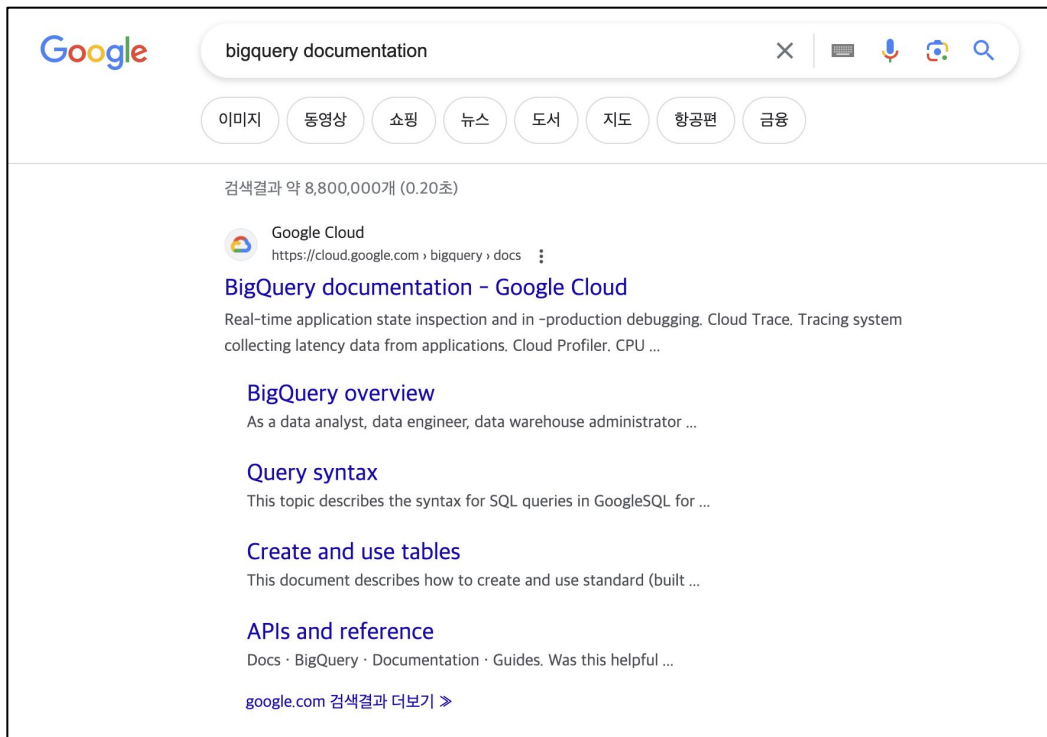
찾는 방법

“기술명 + documentation”으로 검색

예) BigQuery Documentation 검색

구글에서 BigQuery Documentation으로 검색

- BigQuery는 구글 클라우드의 제품이므로 구글 클라우드의 URL을 찾을 수 있음



구글에서 BigQuery Documentation으로 검색

- <https://cloud.google.com/bigquery/docs?hl=ko>
- BigQuery는 기능이 많고 발전이 오래된 제품이라 공식 문서에 자료가 많음
- 목적에 맞는 문서를 탐색

The screenshot shows the Google Cloud BigQuery documentation page. At the top, there's a navigation bar with 'Google Cloud' logo, links for '개요' (Overview), '솔루션' (Solutions), '제품' (Products), '가격 책정' (Pricing), and '리소스' (Resources). A search bar and 'Docs' link are also present. Below the navigation bar, the 'BigQuery' section is active, with sub-links for '개요' (Overview), '가이드' (Guides), '참조' (References), '샘플' (Samples), and '리소스' (Resources). The main content area is titled 'BigQuery 문서' (BigQuery Docs) and includes a brief description of BigQuery as a serverless data warehouse. It also features three columns of links: '가이드' (Guides) with links like '빠른 시작: Console, 명령줄, 클라이언트 라이브러리' and 'BigQuery ML 소개'; '참조' (References) with links like 'GoogleSQL의 표현식, 함수, 연산자' and 'BigQuery API 클라이언트 라이브러리'; and '리소스' (Resources) with links like '가격 책정' and 'BigQuery 활동량 오류 문제 해결'.

Google Cloud 개요 솔루션 제품 가격 책정 리소스

BigQuery 개요 가이드 참조 샘플 리소스

문의하기 무료로 시작하기

BigQuery 문서

BigQuery는 Google Cloud의 페타바이트급 규모의 경제적인 완전 관리형 분석 데이터 웨어하우스로, 거의 실시간으로 방대한 양의 데이터를 분석할 수 있습니다. BigQuery를 사용하면 설정하거나 관리할 인프라가 없으므로 GoogleSQL을 사용하여 유용한 정보를 찾는 데 집중하고 주 문형 옵션과 정역제 옵션에서 유연하게 가격 책정 모델을 활용할 수 있습니다. [자세히 알아보기](#)

개요

- 교육 및 튜토리얼
- 사용 사례
- 코드 샘플
- 동영상

가이드

- 빠른 시작: [Console](#), [명령줄](#), [클라이언트 라이브러리](#)
- 테이블 만들기 및 사용
- 파티션을 나누는 테이블 소개
- BigQuery ML 소개
- 사전 정의된 역할 및 권한
- 데이터 로드 소개
- Cloud Storage에서 CSV 데이터 로드
- 테이블 데이터 내보내기
- BigQuery ML에서 머신러닝 모델 만들기
- 외부 데이터 소스 쿼리

참조

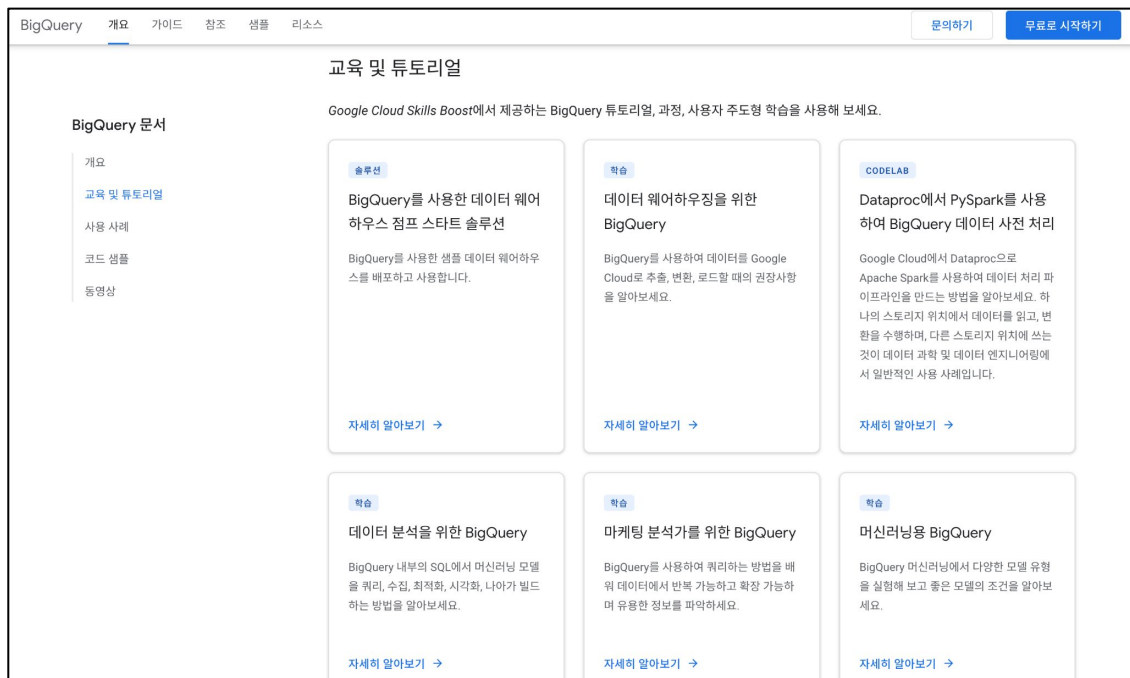
- GoogleSQL의 표현식, 함수, 연산자
- GoogleSQL의 날짜 함수
- GoogleSQL의 쿼리 구문
- GoogleSQL의 문자열 함수
- bq 명령줄 도구 사용
- 머신러닝 모델을 위한 엔드 투 엔드 여정
- BigQuery API 클라이언트 라이브러리
- 모델 만들기 및 학습
- 공개 데이터 세트
- 기능 사전 처리

리소스

- 가격 책정
- 출시 노트
- 위치
- 지원 받기
- 활동량 및 한도
- 비용 관리
- 커스텀 비용 관리 만들기
- BigQuery 활동량 오류 문제 해결
- 결재 관련 문의

구글에서 BigQuery Documentation으로 검색

- <https://cloud.google.com/bigquery/docs?hl=ko>
- BigQuery는 기능이 많고 발전이 오래된 제품이라 공식 문서에 자료가 많음
- 목적에 맞는 문서를 탐색



BigQuery Query Documentation

- <https://cloud.google.com/bigquery/docs/reference/standard-sql/query-syntax>
- 작성된 내용이 무엇이 있나 확인해보기

The screenshot shows the BigQuery Query syntax documentation page. The page has a navigation sidebar on the left with links to various BigQuery topics. The main content area is titled "Query syntax" and includes a "Send feedback" button. Below the title, there is a blue banner with the text "GoogleSQL is the new name for Google Standard SQL! New name, same great SQL dialect." followed by a paragraph explaining that query statements scan tables or expressions and return computed result rows. The section "SQL syntax notation rules" lists several rules for SQL syntax notation, including square brackets for optional clauses, curly braces with vertical bars for logical OR, ellipsis for repeating items, red square brackets for required parts, red curly braces for required parts, and red vertical bars for required parts. The section "SQL syntax" is partially visible at the bottom, showing the definition of a query statement as a query expression. The right sidebar contains a table of contents for the page, listing various SQL syntax topics such as SELECT statement, FROM clause, UNNEST operator, PIVOT operator, and Join operation.

BigQuery Overview 가이드 참조 샘플 리소스

Filter

할당량 오류 문제 해결

BigQuery 명령줄 도구
bq 명령줄 도구 참조

BigQuery의 SQL
GoogleSQL 참조
쿼리 구문

- ▶ 일반 자료실
- ▶ 표현식
- ▶ 함수
 - 모든 함수 및 연산자
 - AEAD 암호화 함수
 - 집계 함수
 - 근사치 집계 함수
 - 배열 함수
 - 비트 함수
 - 변환 함수
 - 날짜 함수
 - 날짜/시간 함수
 - 디버깅 함수
 - 차등 비교에 집계 함수
 - 통합 쿼리 함수
 - DLP 암호화 함수
 - 지리 함수
 - 해시 함수
 - HyperLogLog++ 함수

Query syntax

Send feedback

GoogleSQL is the new name for Google Standard SQL! New name, same great SQL dialect.

Query statements scan one or more tables or expressions and return the computed result rows. This topic describes the syntax for SQL queries in GoogleSQL for BigQuery.

SQL syntax notation rules

The GoogleSQL documentation commonly uses the following syntax notation rules:

- Square brackets []: Optional clause.
- Curly braces with vertical bars { a | b | c }: Logical OR. Select one option.
- Ellipsis ...: Preceding item can repeat.
- Red square brackets []: Required and part of the syntax.
- Red curly braces { }: Required and part of the syntax.
- Red vertical bar |: Required and part of the syntax.

SQL syntax

`query_statement:`
`query_expr`

이 페이지의 내용

- SQL syntax notation rules
- SQL syntax
- SELECT statement
 - SELECT *
 - SELECT expression
 - SELECT expression *
 - SELECT * EXCEPT
 - SELECT * REPLACE
 - SELECT DISTINCT
 - SELECT ALL
 - SELECT AS STRUCT
 - SELECT AS VALUE
- FROM clause
- UNNEST operator
 - UNNEST and structs
 - Explicit and implicit UNNEST
 - UNNEST and NULL values
- PIVOT operator
- UNPIVOT operator
- TABLESAMPLE operator
- Join operation
 - [INNER] JOIN
 - CROSS JOIN
 - Comma cross join (,)
 - FULL [OUTER] JOIN
 - LEFT [OUTER] JOIN
 - RIGHT [OUTER] JOIN
- Join conditions
- ON and USING equivalency

다양한 문자열(String) 함수

- 만약 CONCAT 문법을 알고 싶다면
- BigQuery CONCAT document라고 구글에 검색

https://cloud.google.com/bigquery/docs/reference/standard-sql/string_functions

Filter

DLP encryption functions

Geography functions

Hash functions

HyperLogLog++ functions

Interval functions

JSON functions

Mathematical functions

Navigation functions

Net functions

Numbering functions

Search functions

Security functions

Statistical aggregate functions

String functions

Table functions (built-in)

Text analysis functions

Time functions

Timestamp functions

Utility functions

Statements

BigQuery ML SQL reference

INFORMATION SCHEMA views

Legacy SQL reference

BigQuery > Documentation > Reference

Was this helpful?

String functions

GoogleSQL for BigQuery supports string functions. These string functions work on two different values: `STRING` and `BYTES` data types. `STRING` values must be well-formed UTF-8.

Functions that return position values, such as `STRPOS`, encode those positions as `INT64`. The value `1` refers to the first character (or byte), `2` refers to the second, and so on. The value `0` indicates an invalid position. When working on `STRING` types, the returned positions refer to character positions.

All string comparisons are done byte-by-byte, without regard to Unicode canonical equivalence.

Function list

Name	Summary
ASCII	Gets the ASCII code for the first character or byte in a <code>STRING</code> or <code>BYTES</code> value.
BYTE_LENGTH	Gets the number of <code>BYTES</code> in a <code>STRING</code> or <code>BYTES</code> value.
CHAR_LENGTH	Gets the number of characters in a <code>STRING</code> value.
CHARACTER_LENGTH	Synonym for <code>CHAR_LENGTH</code> .
CHR	Converts a Unicode code point to a character.

On this page

[Function list](#)

[ASCII](#)

[BYTE_LENGTH](#)

[CHAR_LENGTH](#)

[CHARACTER_LENGTH](#)

[CHR](#)

[CODE_POINTS_TO_BYTES](#)

[CODE_POINTS_TO_STRING](#)

[COLLATE](#)

[CONCAT](#)

[CONTAINS_SUBSTR](#)

[EDIT_DISTANCE](#)

[ENDS_WITH](#)

[FORMAT](#)

[FROM_BASE32](#)

[FROM_BASE64](#)

[FROM_HEX](#)

[INITCAP](#)

[INSTR](#)

[LEFT](#)

[LENGTH](#)

[LOWER](#)

[LPAD](#)

[LTRIM](#)

[NORMALIZE](#)

문자열(String) 함수 문법 찾는 방법

- 공식 문서에서 찾기(CTRL+F)를 사용해 문법쪽 확인

CONCAT

CONCAT(value1[, ...])



Description

Concatenates one or more values into a single result. All values must be `BYTES` or data types that can be cast to `STRING`.

The function returns `NULL` if any input argument is `NULL`.



Note: You can also use the `||` [concatenation operator](#) to concatenate values into a string.

Return type

`STRING` or `BYTES`

문자열(String) 함수 문법 찾는 방법

- 보통 예제 쿼리도 같이 제공

Examples

```
SELECT CONCAT('T.P.', ' ', 'Bar') as author;
```

```
/*-----*  
| author      |  
+-----+  
| T.P. Bar    |  
*-----*/
```

```
SELECT CONCAT('Summer', ' ', 1923) as release_date;
```

```
/*-----*  
| release_date |  
+-----+  
| Summer 1923  |  
*-----*/
```


공식 문서를 꼭 봐야 하나?

- 공식 문서"도" 볼 수 있어야 함
- ChatGPT, 블로그에도 참고할 수 있는 자료가 있을 수 있지만, 잘못된 경우나 과거 문법인 경우가 존재
- 구글에서 관리하는 문서이므로 기준으로 삼아보기
- 다만 공식 문서로 시작하면 방대한 양에 압도될 수 있으므로,
필요할 때마다(목적에 맞게)공식 문서를 보는 방식으로 학습 추천

예시)

어떤 작업을 해야 한다 => 찾아본다 => A라는 함수를 써야 함 => A 함수 공식 문서 보기

특정 작업을 하다가 어떻게 해야할지 모르겠다면

BigQuery xxxxx 라고 검색

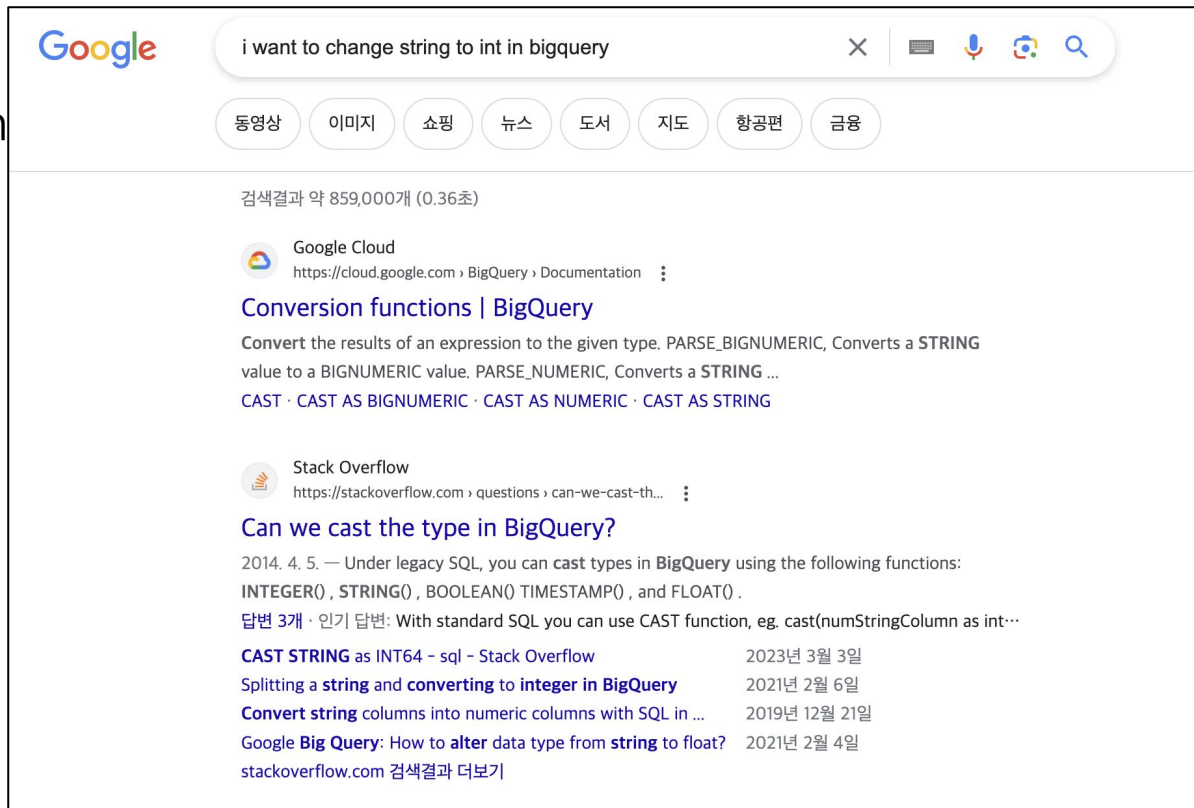
xxxx : 특정 행위

i want to change string to int **in bigquery**

특정 작업을 하다가 어떻게 해야할지 모르겠다면

BigQuery xxxxx 라고 검색
xxxx : 특정 행위

i want to change string to int



특정 작업을 하다가 어떻게 해야할지 모르겠다면

원하는 것을 잘 찾는 과정

- 검색(command / ctrl + f) 활용
- Stackoverflow, GPT 에서 키워드 힌트 찾고 검색
- 공식 문서를 여유를 가지고 읽기

Conversion functions

[Send feedback](#)

GoogleSQL for BigQuery supports conversion functions. These data type conversions are explicit, but some conversions can happen implicitly. You can learn more about implicit and explicit conversion [here](#).

Function list

Name	Summary
CAST	Convert the results of an expression to the given type.
PARSE_BIGNUMERIC	Converts a <code>STRING</code> value to a <code>BIGNUMERIC</code> value.
PARSE_NUMERIC	Converts a <code>STRING</code> value to a <code>NUMERIC</code> value.
SAFE_CAST	Similar to the <code>CAST</code> function, but returns <code>NULL</code> when a runtime error is produced.

CAST

`CAST(expression AS typename [format_clause])`  

Description

Cast syntax is used in a query to indicate that the result type of an expression should be converted to some other type.

When using `CAST`, a query can fail if GoogleSQL is unable to perform the cast. If you want to protect your queries from these types of errors, you can use [SAFE_CAST](#).

Casts between supported types that do not successfully map from the original value to the target domain produce runtime errors. For example, casting `BYTES` to `STRING` where the byte sequence is not valid UTF-8 results in a runtime error.

이 페이지의 내용

[Function list](#)

- [CAST](#)
- [CAST AS ARRAY](#)
- [CAST AS BIGNUMERIC](#)
- [CAST AS BOOL](#)
- [CAST AS BYTES](#)
- [CAST AS DATE](#)
- [CAST AS DATETIME](#)
- [CAST AS FLOAT64](#)
- [CAST AS INT64](#)
- [CAST AS INTERVAL](#)
- [CAST AS NUMERIC](#)
- [CAST AS STRING](#)
- [CAST AS STRUCT](#)
- [CAST AS TIME](#)
- [CAST AS TIMESTAMP](#)
- [PARSE_BIGNUMERIC](#)
- [PARSE_NUMERIC](#)
- [SAFE_CAST](#)

[Other conversion functions](#)

공식 문서 Slack RSS Feed 추가하기

BigQuery가 조용히(?) 새로운 기능을 출시하기도 함
이걸 빠르게 알 수 있는 방법은?

=> RSS Feed 구독

공식 문서 Slack RSS Feed 추가하기

RSS Feed(Really Simple Syndication)

- 새 기사들의 제목 또는 새 기사들 전체를 뽑아서 하나의 파일로 만들어 놓은 것

BigQuery Release Note 페이지에서 확인할 수 있음

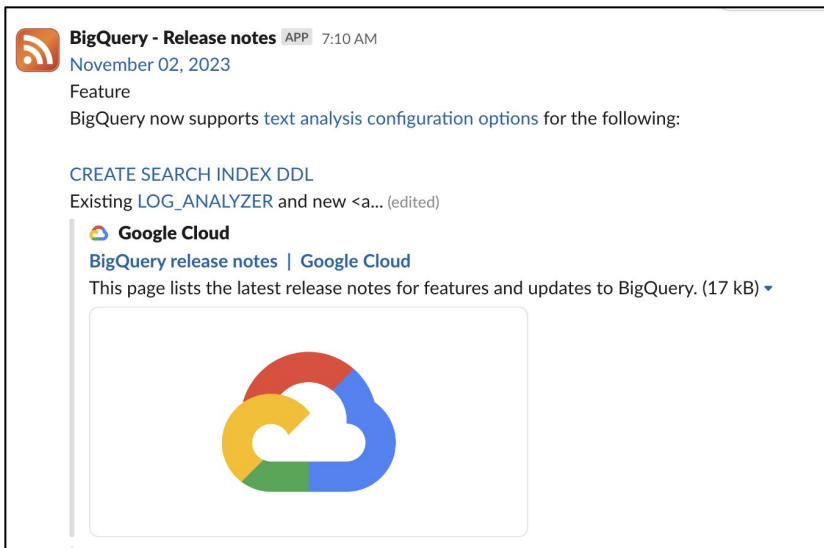
<https://cloud.google.com/bigquery/docs/release-notes>

공식 문서 Slack RSS Feed 추가하기

슬랙에서 알람 받아보기

- 개인 목적으로 사용할 슬랙 워크스페이스 생성
- 채널에 아래 명령어 입력

/feed subscribe https://cloud.google.com/feeds/bigquery-release-notes.xml

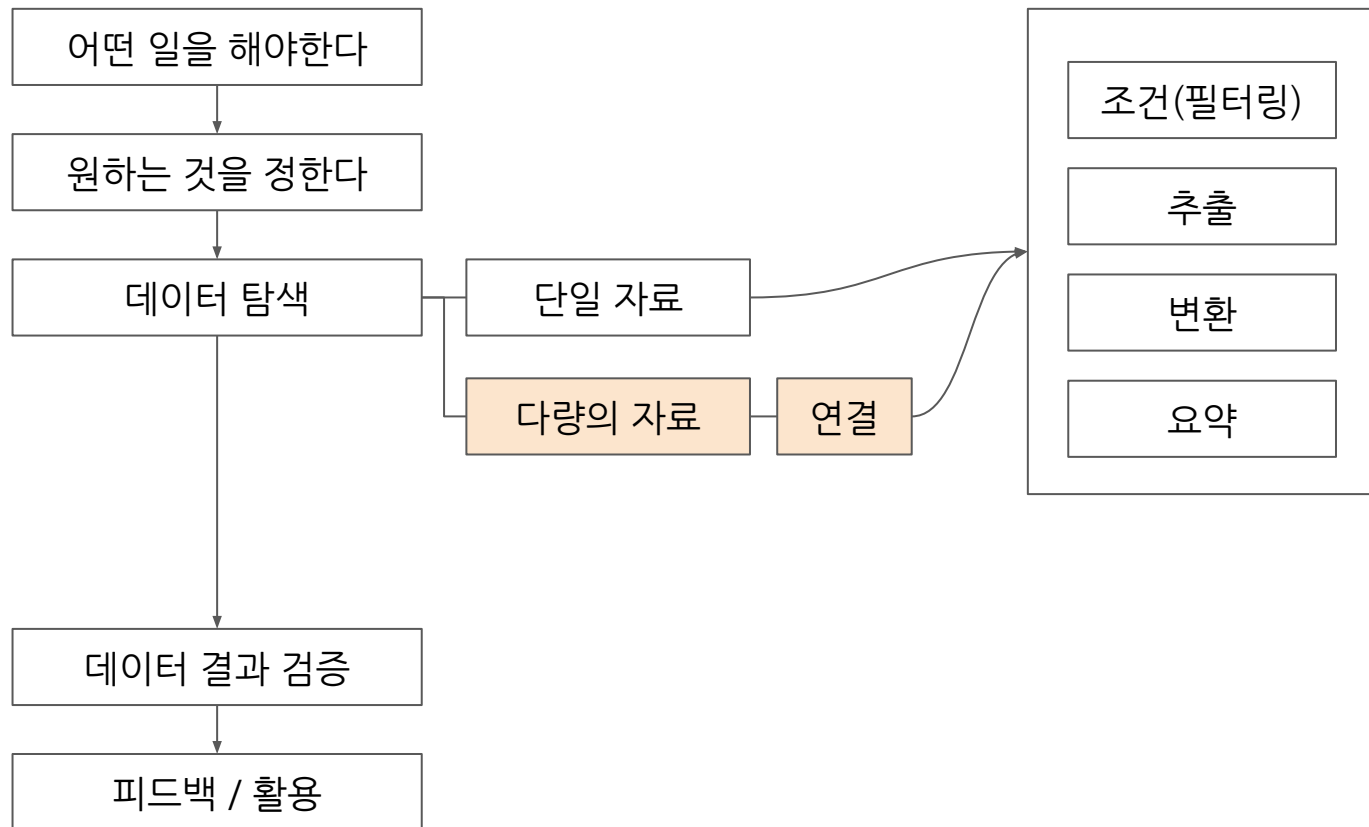


#5. 다량의 자료를 연결 : JOIN

카일스쿨



데이터를 활용하는 과정



이번 파트에서 다룰 내용

- 포켓몬으로 JOIN 이해하기
- 다양한 JOIN 방법
- JOIN 쿼리 작성하기
- JOIN을 처음 공부할 때 헷갈렸던 부분
- JOIN 연습 문제

포켓몬으로 JOIN 이해하기

SQL JOIN

- SQL을 처음 배울 때, 어려울 수 있는 부분
- 간단하게 “서로 다른 데이터 테이블을 연결하는 것”
- 많은 문제 풀이를 통한 체화

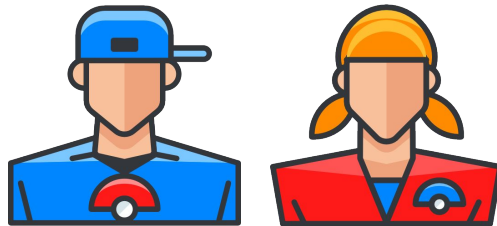
[직관적 이해]

[SQL 쿼리를 통한 실습]

포켓몬으로 JOIN 이해하기

포켓몬과

트레이너 : 포켓몬을 포획해 육성하는 사람들



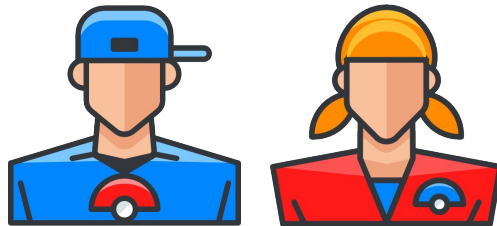
포켓몬으로 JOIN 이해하기

포켓몬과

트레이너 : 포켓몬을 포획해 육성하는 사람들

포켓몬은 다양한 곳에서 나타남

트레이너는 포획한 포켓몬을 육성할 수 있음



포켓몬으로 JOIN 이해하기

트레이너 데이터

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced

포켓몬으로 JOIN 이해하기

트레이너 데이터

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced

포켓몬 데이터

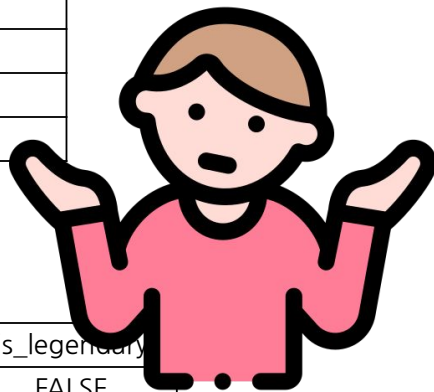
id	kor_name	type1	type2	total	hp	attack	defense	speed	generation	is_legendary
1	이상해씨	Grass	Poison	318	45	49	49	45	1	FALSE
2	이상해풀	Grass	Poison	405	60	62	63	60	1	FALSE
3	이상해꽃	Grass	Poison	525	80	82	83	80	1	FALSE
4	파이리	Fire		309	39	52	43	65	1	FALSE
5	리자드	Fire		405	58	64	58	80	1	FALSE

포켓몬으로 JOIN 이해하기

트레이너 데이터

두 데이터를 연결할 수 있는 공통 값이 없음!

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced



포켓몬 데이터

id	kor_name	type1	type2	total	hp	attack	defense	speed	generation	is_legendary
1	이상해씨	Grass	Poison	318	45	49	49	45	1	FALSE
2	이상해풀	Grass	Poison	405	60	62	63	60	1	FALSE
3	이상해꽃	Grass	Poison	525	80	82	83	80	1	FALSE
4	파이리	Fire		309	39	52	43	65	1	FALSE
5	리자드	Fire		405	58	64	58	80	1	FALSE

포켓몬으로 JOIN 이해하기

트레이너 데이터

id	name	age	
1	Ash	16	
2	K	18	
3	Peter	20	
4	Maya	15	
5	Gary	17	

포켓몬 데이터

id	kor_name	type1	
1	이상해씨	Grass	
2	이상해풀	Grass	
3	이상해꽃	Grass	
4	파이리	Fire	
5	리자드	Fire	

트레이너가 포획한 포켓몬 데이터
(=트레이너와 포켓몬을 연결할 수 있음)

id	trainer_id	pokemon_id	catch_date	location
1	1	1	2023-01-14	Seoul
2	2	4	2023-02-10	Busan
3	3	3	2023-03-05	Incheon

포켓몬으로 JOIN 이해하기

트레이너가 포획한 포켓몬 기준으로 트레이너 데이터를 연결하기(JOIN)

트레이너가 포획한 포켓몬

id	trainer_id	pokemon_id	catch_date	location
1	1	1	2023-01-14	Seoul
2	2	4	2023-02-10	Busan
3	3	3	2023-03-05	Incheon

트레이너

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced

포켓몬으로 JOIN 이해하기

트레이너가 포획한 포켓몬 기준으로 트레이너 데이터를 연결하기(JOIN)
연결할 수 있는 Key = **trainer_id, id**

트레이너가 포획한 포켓몬

id	trainer_id	pokemon_id	catch_date	location
1	1	1	2023-01-14	Seoul
2	2	4	2023-02-10	Busan
3	3	3	2023-03-05	Incheon

트레이너

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced

포켓몬으로 JOIN 이해하기

trainer_id 컬럼 기준으로 트레이너 데이터를 연결!

트레이너가 포획한 포켓몬

id	trainer_id	pokemon_id	catch_date	location	id	name	age	hometown	...	achievement_level
1	1	1	2023-01-14	Seoul	1	Ash	16	Seoul		Master
2	2	4	2023-02-10	Busan	2	K	18	Busan		Advanced
3	3	3	2023-03-05	Incheon	3	Peter	20	Gwangju		Expert

트레이너

id	name	age	hometown	preferred_pokemon_type	badge_count	achievement_level
1	Ash	16	Seoul	Electric	8	Master
2	K	18	Busan	Water	6	Advanced
3	Peter	20	Gwangju	Rock	7	Expert
4	Maya	15	Incheon	Grass	5	Advanced
5	Gary	17	Daegu	Flying	6	Advanced

포켓몬으로 JOIN 이해하기

trainer_id 컬럼 기준으로 트레이너 데이터를 연결!

트레이너가 포획한 포켓몬 + 트레이너

id	trainer_id	pokemon_id	catch_date	location	id	name	age	hometown	...	achievement_level
1	1	1	2023-01-14	Seoul	1	Ash	16	Seoul		Master
2	2	4	2023-02-10	Busan	2	K	18	Busan		Advanced
3	3	3	2023-03-05	Incheon	3	Peter	20	Gwangju		Expert

포켓몬

id	kor_name	type1	type2	total	hp	attack	defense	speed	generation	is_legendary
1	이상해씨	Grass	Poison	318	45	49	49	45	1	FALSE
2	이상해풀	Grass	Poison	405	60	62	63	60	1	FALSE
3	이상해꽃	Grass	Poison	525	80	82	83	80	1	FALSE
4	파이리	Fire		309	39	52	43	65	1	FALSE
5	리자드	Fire		405	58	64	58	80	1	FALSE

포켓몬으로 JOIN 이해하기

trainer_id 컬럼 기준으로 트레이너 데이터를 연결!

트레이너가 포획한 포켓몬 + 트레이너

id	trainer_id	pokemon_id	catch_date	location	id	id	kor_name	type1	type2	total	hp	attack	defense
1	1	1	2023-01-14	Seoul	1	1	이상해씨	Grass	Poison	318	45	49	49
2	2	4	2023-02-10	Busan	2	4	파이리	Fire		309	39	52	43
3	3	3	2023-03-05	Incheon	3	3	이상해꽃	Grass	Poison	525	80	82	83

포켓몬

id	kor_name	type1	type2	total	hp	attack	defense	speed	generation	is_legendary
1	이상해씨	Grass	Poison	318	45	49	49	45	1	FALSE
2	이상해풀	Grass	Poison	405	60	62	63	60	1	FALSE
3	이상해꽃	Grass	Poison	525	80	82	83	80	1	FALSE
4	파이리	Fire		309	39	52	43	65	1	FALSE
5	리자드	Fire		405	58	64	58	80	1	FALSE

SQL JOIN

- 간단하게 “서로 다른 데이터 테이블을 연결하는 것”
- 공통적으로 존재하는 컬럼(=Key)이 있다면, JOIN 할 수 있음

보통 id 값을 Key로 많이 사용하고, 특정 범위(예 : Date)로 JOIN도 가능함

JOIN이 어려운 것이 아니고
테이블 구조에 익숙하지 않아서 어려움을 느낄 확률이 높음

JOIN을 해야하는 이유 - 데이터 저장되는 형태에 대한 이해

- 관계형 데이터베이스(RDBMS) 설계시 정규화 과정을 거침
- 정규화는 **중복을 최소화하게 데이터를 구조화**
- User Table은 유저 데이터만, Order Table은 주문 데이터만
- 따라서 데이터를 다양한 Table에 저장해서 필요할 때 JOIN해서 사용

JOIN을 해야하는 이유 - 데이터 저장되는 형태에 대한 이해

- 관계형 데이터베이스(RDBMS) 설계시 정규화 과정을 거침
 - 정규화는 **중복을 최소화하게 데이터를 구조화**
 - User Table은 유저 데이터만, Order Table은 주문 데이터만
 - 따라서 데이터를 다양한 Table에 저장해서 필요할 때 JOIN해서 사용

데이터 분석하는 관점에선 미리 JOIN되어 있는 것이 좋을 수 있지만,
개발 관점에선 분리되어 있는 것이 좋음

대신 데이터 웨어하우스에서 JOIN + 필요한 연산을 해서
“데이터 마트”를 만들어서 활용

다양한 JOIN 방법

다양한 SQL JOIN 방법

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN

처음에 어렵다면 LEFT JOIN만 주로 사용해도 충분

다양한 SQL JOIN 방법

- (INNER) JOIN : 두 테이블의 **공통** 요소만 연결
- LEFT/RIGHT (OUTER) JOIN : **왼쪽/오른쪽** 테이블 기준으로 연결
- FULL (OUTER) JOIN : **양쪽** 기준으로 연결
- CROSS JOIN : 두 테이블의 각각의 **요소**를 곱하기

처음에 어렵다면 LEFT JOIN만 주로 사용해도 충분

다양한 SQL JOIN 방법

KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B



다양한 SQL JOIN 방법

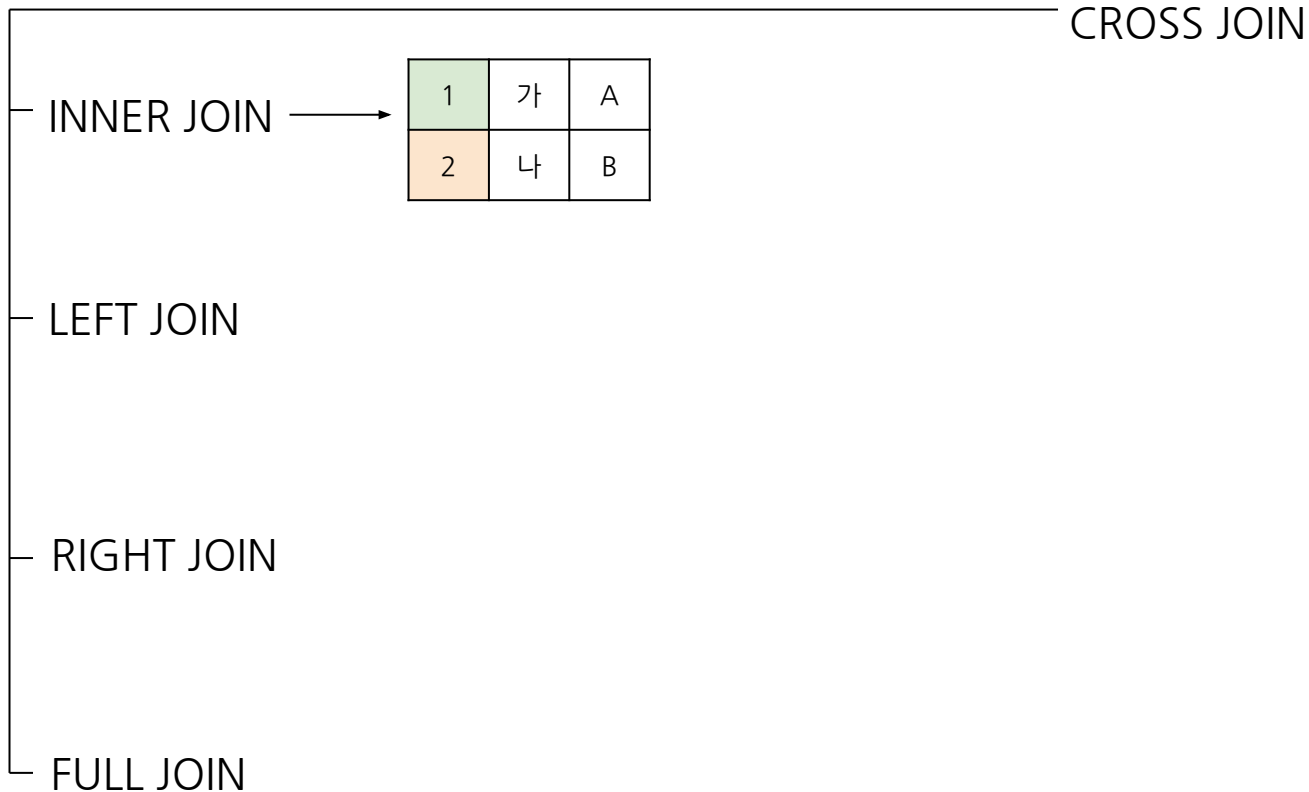
KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B



다양한 SQL JOIN 방법

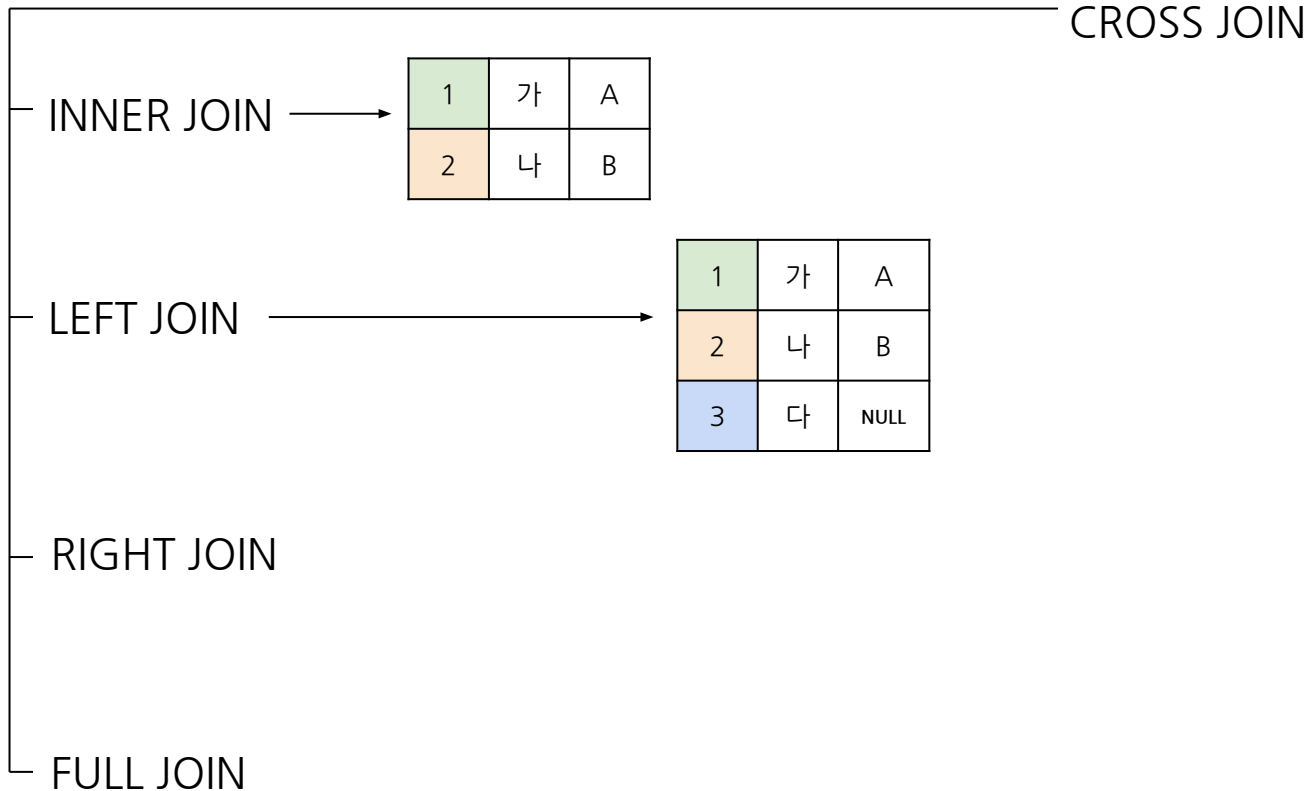
KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B



다양한 SQL JOIN 방법

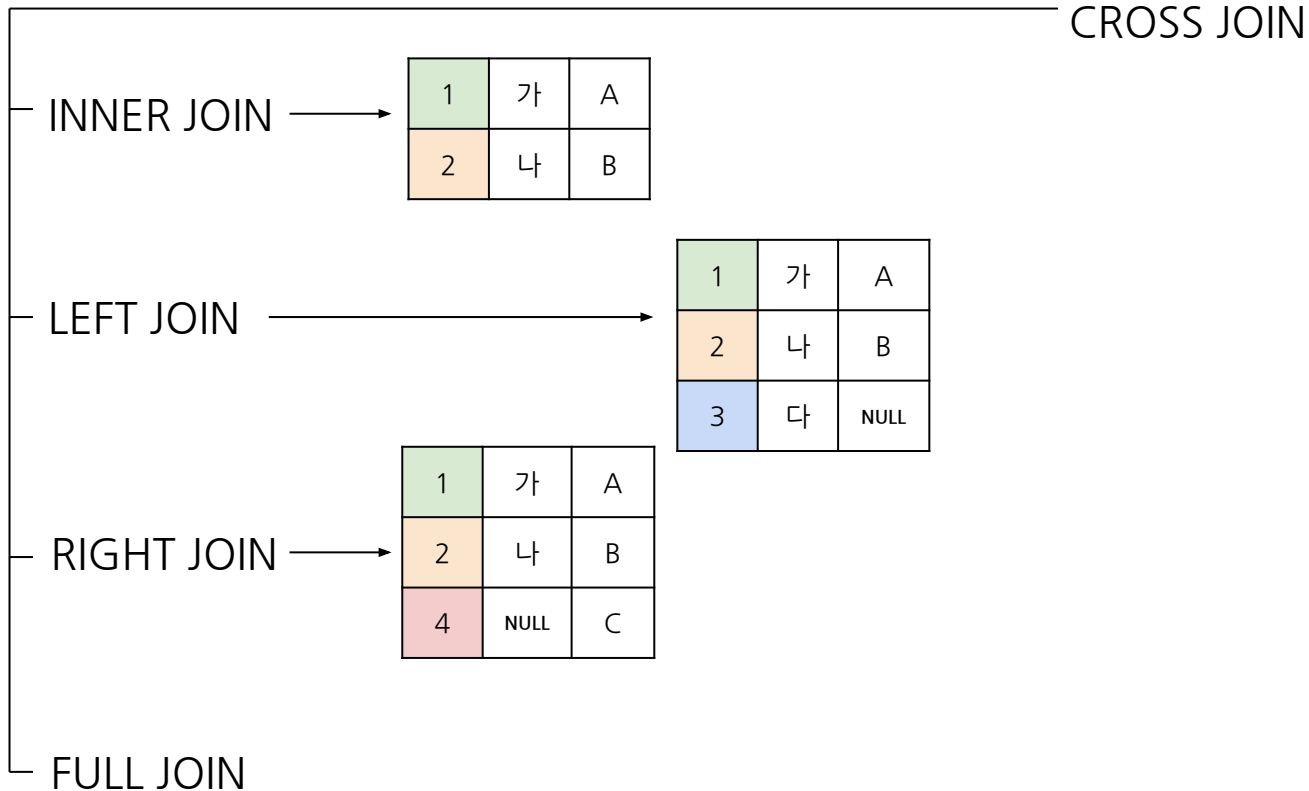
KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B



다양한 SQL JOIN 방법

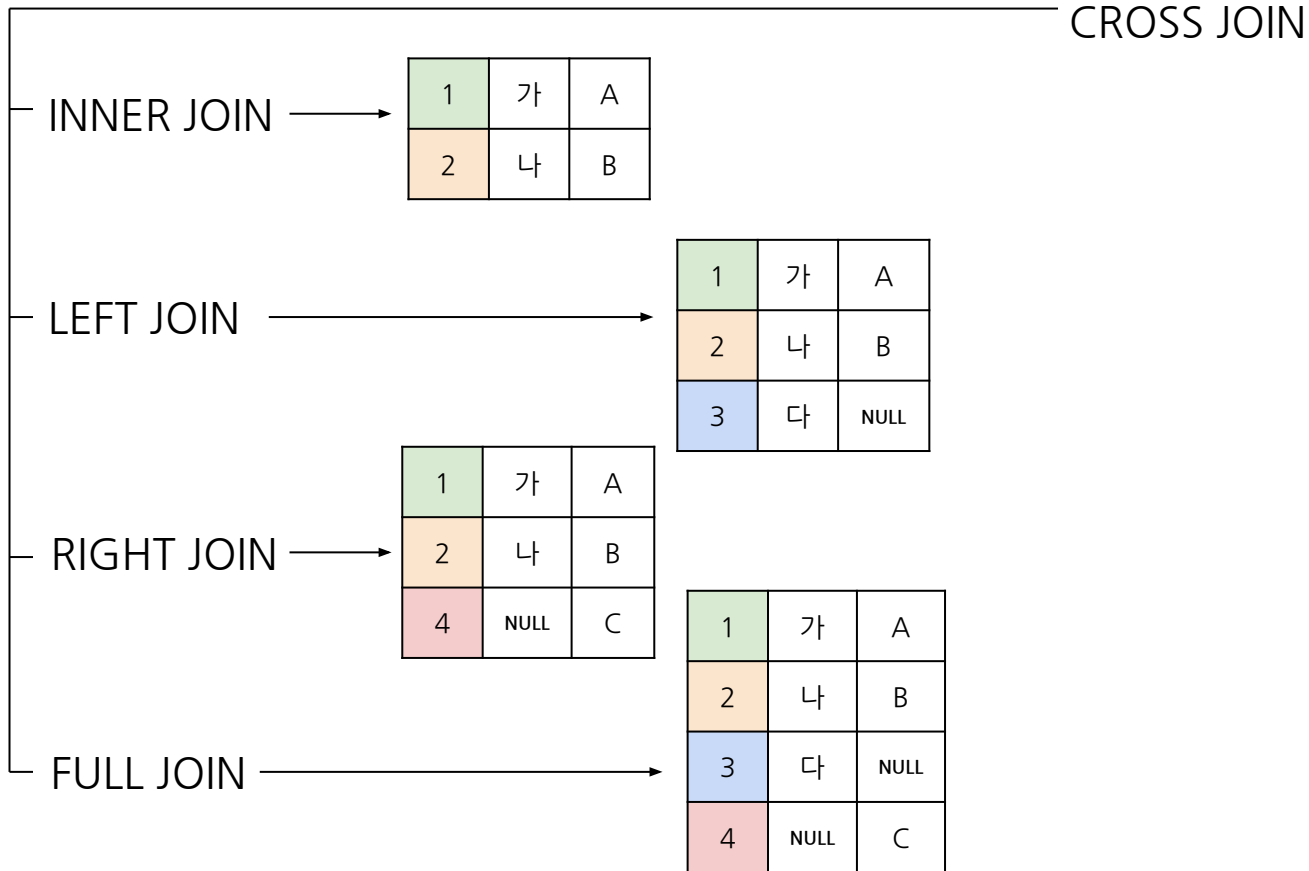
KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B



다양한 SQL JOIN 방법

KEY

1	가
2	나
3	다

Table A

X

KEY

1	A
2	B
4	C

Table B

INNER JOIN

1	가	A
2	나	B

LEFT JOIN

1	가	A
2	나	B
3	다	NULL

RIGHT JOIN

1	가	A
2	나	B
4	NULL	C

FULL JOIN

1	가	A
2	나	B
3	다	NULL
4	NULL	C

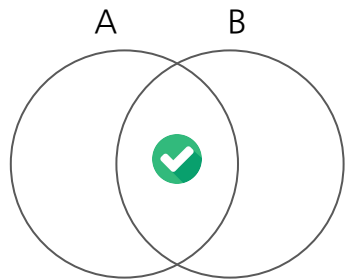
CROSS JOIN

1	가	1	A
1	가	2	B
1	가	4	C
2	나	1	A
2	나	2	B
2	나	4	C
3	다	1	A
3	다	2	B
3	다	4	C

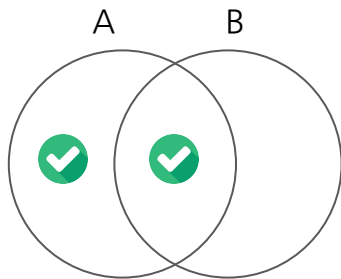


JOIN 집합 관점으로 생각하기

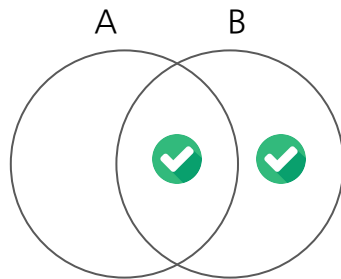
Inner Join



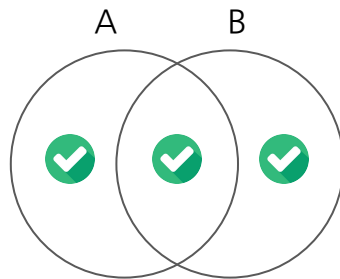
LEFT Join



RIGHT Join



FULL Join



JOIN 쿼리 작성하기

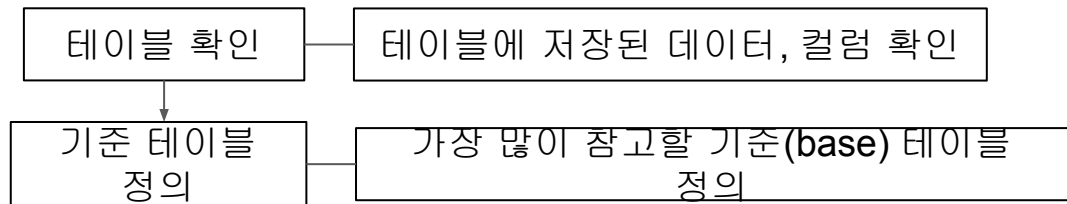
SQL JOIN 쿼리 작성하는 흐름

테이블 확인

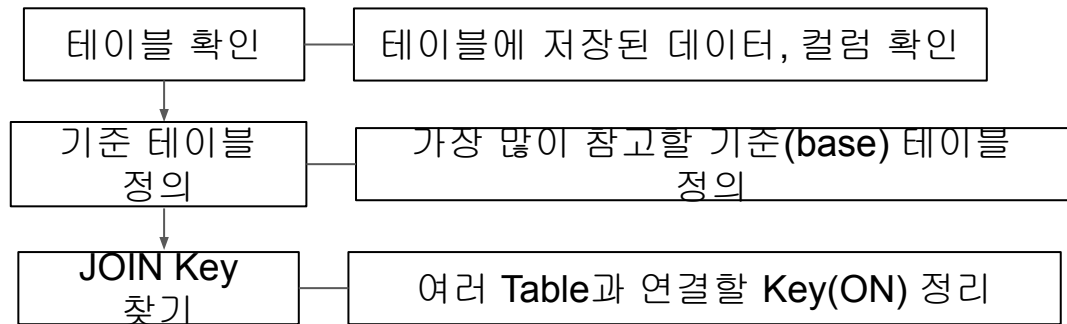
테이블에 저장된 데이터, 컬럼 확인



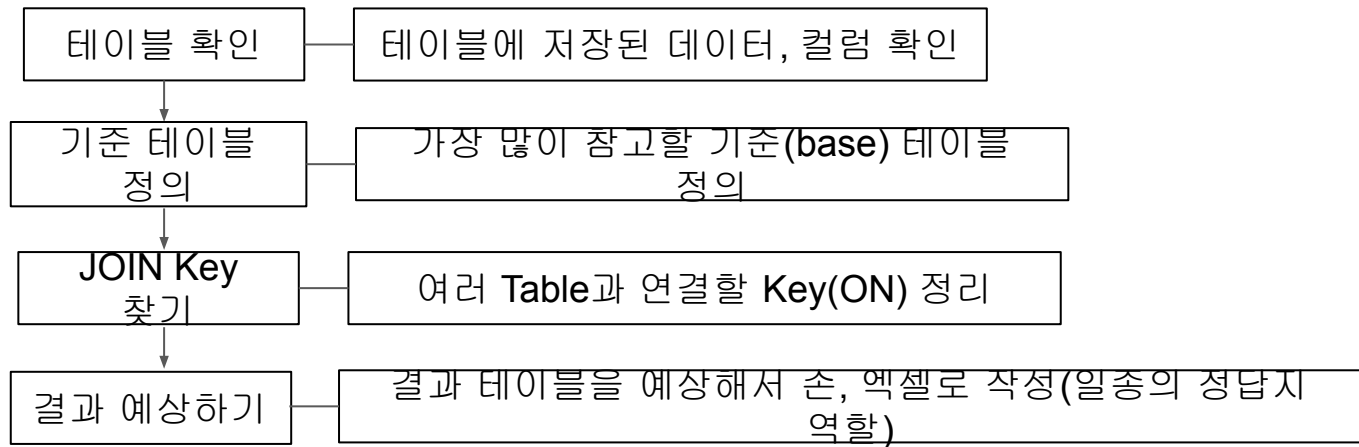
SQL JOIN 쿼리 작성하는 흐름



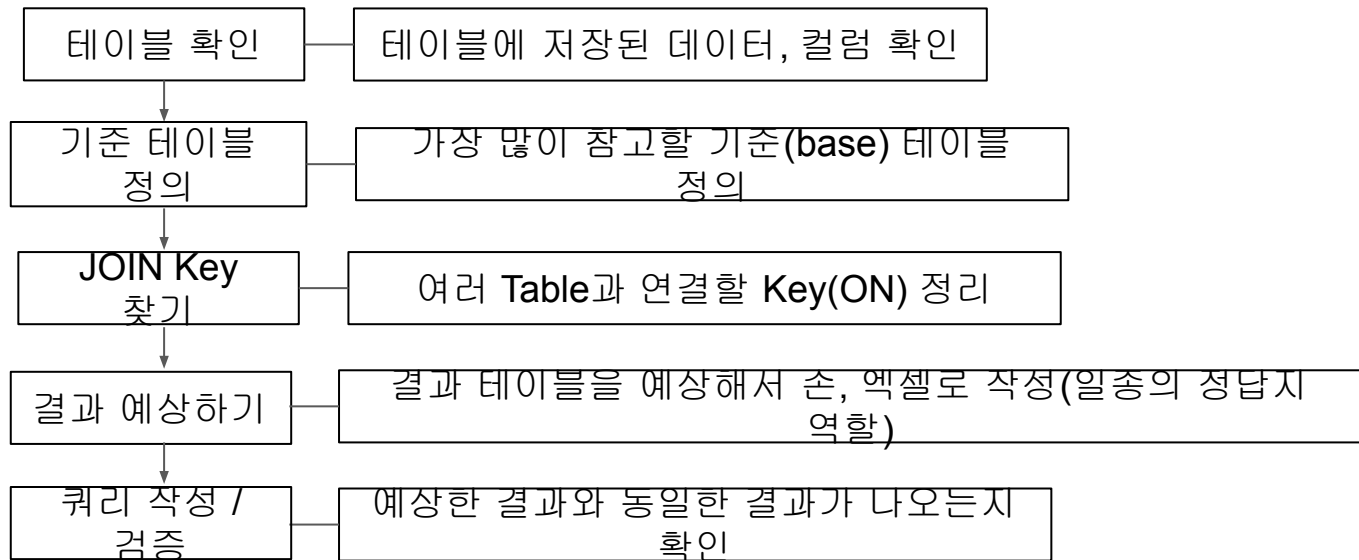
SQL JOIN 쿼리 작성하는 흐름



SQL JOIN 쿼리 작성하는 흐름



SQL JOIN 쿼리 작성하는 흐름



SQL JOIN 문법

FROM 하단에 JOIN 할 Table을 작성하고
ON 뒤에 공통된 컬럼(Key)를 작성

```
SELECT
    A.col1,
    A.col2,
    B.col11,
    B.col12
FROM table1 AS A
LEFT JOIN table2 AS B
ON A.key = B.key # Alias를 사용할 수 있음
```

테이블 이름이 길 수 있기 때문에 별칭(Alias)을 정의해줄 수 있음

여러 JOIN 별 쿼리 예시

	ON 필수 여부	쿼리 예시
INNER JOIN	O	<pre>SELECT col FROM table_a AS A INNER JOIN table_b AS B ON A.key = B.key</pre>
LEFT/RIGHT JOIN	O	<pre>SELECT col FROM table_a AS A LEFT/RIGHT JOIN table_b AS B ON A.key = B.key</pre>
FULL JOIN	O	<pre>SELECT col FROM table_a AS A FULL JOIN table_b AS B ON A.key = B.key</pre>
CROSS JOIN	X	<pre>SELECT col FROM table_a AS A CROSS JOIN table_b AS B</pre>

SQL JOIN 문법 확인하기

trainer_id 컬럼 기준으로 트레이너 데이터를 연결!

트레이너가 포획한 포켓몬 + 트레이너

id	trainer_id	pokemon_id	catch_date	location	id	id	kor_name	type1	type2	total	hp	attack	defense
1	1	1	2023-01-14	Seoul	1	1	이상해씨	Grass	Poison	318	45	49	49
2	2	4	2023-02-10	Busan	2	4	파이리	Fire		309	39	52	43
3	3	3	2023-03-05	Incheon	3	3	이상해꽃	Grass	Poison	525	80	82	83

포켓몬

id	kor_name	type1	type2	total	hp	attack	defense	speed	generation	is_legendary
1	이상해씨	Grass	Poison	318	45	49	49	45	1	FALSE
2	이상해풀	Grass	Poison	405	60	62	63	60	1	FALSE
3	이상해꽃	Grass	Poison	525	80	82	83	80	1	FALSE
4	파이리	Fire		309	39	52	43	65	1	FALSE
5	리자드	Fire		405	58	64	58	80	1	FALSE

BigQuery에서 JOIN하기

trainer_pokemon 기준

탐색기

+ 추가 <

🔍 입력하여 검색

리소스를 보는 중입니다.

별표표시된 리소스만 표시

- ▼ inflearn-bigquery ☆ ⋮
 - ▶ 외부 연결 ⋮
 - ▼ basic ☆ ⋮
 - battle ☆ ⋮
 - pokemon ☆ ⋮
 - trainer ☆ ⋮
 - trainer_pokemon ☆ ⋮

trainer_pokemon											
스키마 세부정보 미리보기 계보 데이터 프로필 신규 데이터 품질 신규											
행	id	trainer_id	pokemon_id	level	experience_poin	current_health	catch_date	catch_datetime	location	status	
1	30	5	12	19	750	40	2023-01-07	2023-01-07 22:28:00 UTC	Jeju	Active	
2	61	5	12	20	800	40	2023-01-09	2023-01-09 16:36:00 UTC	Jeju	Active	
3	75	15	9	29	1300	60	2023-01-19	2023-01-19 18:15:00 UTC	Jeju	Released	
4	28	20	9	45	3000	85	2023-01-15	2023-01-15 03:35:00 UTC	Jeju	Active	
5	54	15	73	45	2400	90	2023-02-22	2023-02-22 15:44:00 UTC	Jeju	Active	
6	15	2	66	48	2800	100	2023-03-05	2023-03-05 10:39:00 UTC	Jeju	Active	
7	29	9	81	49	3400	100	2023-03-25	2023-03-25 00:28:00 UTC	Jeju	Training	
8	60	7	81	55	4000	110	2023-03-30	2023-03-30 06:24:00 UTC	Jeju	Training	
9	208	88	43	5	140	22	2023-07-14	2023-07-14 00:56:00 UTC	Busan	Active	
10	265	32	20	6	190	20	2023-07-11	2023-07-11 04:52:00 UTC	Busan	Active	
11	143	21	96	7	200	25	2023-02-24	2023-02-24 12:06:00 UTC	Busan	Active	
12	41	5	37	8	320	30	2023-01-18	2023-01-18 04:31:00 UTC	Busan	Released	
13	184	3	85	8	350	25	2023-07-02	2023-07-02 19:48:00 UTC	Busan	Released	
14	32	8	4	9	300	30	2023-01-04	2023-01-04 05:23:00 UTC	Busan	Released	
15	165	59	51	10	400	25	2023-07-04	2023-07-04 09:49:00 UTC	Busan	Active	
16	63	8	4	12	400	35	2023-01-07	2023-01-07 18:05:00 UTC	Busan	Released	
17	58	16	94	19	750	40	2023-01-28	2023-01-28 13:54:00 UTC	Busan	Active	
18	46	2	12	21	800	40	2023-01-08	2023-01-08 04:54:00 UTC	Busan	Active	

BigQuery에서 JOIN하기

```
SELECT
```

```
  tp.*,
```

```
  t.*
```

```
FROM basic.trainer_pokemon AS tp
```

```
LEFT JOIN basic.trainer AS t
```

```
ON tp.trainer_id = t.id
```

BigQuery에서 JOIN하기

```
SELECT
  tp.*,
  t.*,
  p.*
FROM basic.trainer_pokemon AS tp
LEFT JOIN basic.trainer AS t
ON tp.trainer_id = t.id
LEFT JOIN basic.pokemon AS p
ON tp.pokemon_id = p.id
```


JOIN을 처음 공부할 때 헷갈렸던
부분

SQL JOIN을 처음 공부할 때 헷갈렸던 부분

- 1) 여러 JOIN 중 어떤 것을 사용해야 할까?
- 2) 어떤 Table을 왼쪽에 두고, 어떤 Table이 오른쪽에 가야할까?
- 3) 여러 Table을 연결할 수 있는걸까?
- 4) 컬럼은 모두 다 선택해야 할까?
- 5) NULL이 대체 뭐죠?

1) 여러 JOIN 중 어떤 것을 사용해야 할까?

- 하려고 하는 작업의 목적에 따라 JOIN을 선택해보기
 - 교집합 : INNER
 - 모두 다 조합 : CROSS
 - 그게 아니라면 LEFT 또는 RIGHT : LEFT를 추천, 하나를 계속 활용하는 것을 추천
- 쿼리 작성 템플릿에 예상하는 결과를 작성하고, 중간 결과도 생각하면서 찾아보기

2) 어떤 Table을 왼쪽에 두고, 어떤 Table이 오른쪽에 가야할까?

- LEFT JOIN의 경우
 - 기준이 되는 Table을 왼쪽에 두기
 - 기준에는 기준값이 존재하고, 우측에 데이터를 계속 추가

3) 여러 Table을 연결할 수 있는걸까?

- JOIN의 개수에 한계는 없음
- 너무 많이 JOIN하고 있는지 확인

```
SELECT
  table_a.col1,
  table_b.col2
FROM table_a
LEFT JOIN table_b
ON table_a.key = table_b.key
```

3) 여러 Table을 연결할 수 있는걸까?

- JOIN의 개수에 한계는 없음
- 너무 많이 JOIN하고 있는지 확인

```
SELECT
  table_a.col1,
  table_b.col2, table_c.col3
FROM table_a
LEFT JOIN table_b
ON table_a.key = table_b.key
LEFT JOIN table_c
ON table_a.key = table_c.key
```

4) 컬럼은 모두 다 선택해야 할까?

- 컬럼 선택은 데이터를 추출해서 무엇을 하고자?하냐에 따라 다름
- JOIN이 잘 되었나 확인하기 위해 처음엔 많은 컬럼을 선택해도 괜찮으나,
사용하지 않을 컬럼은 선택하지 않는 것이 BigQuery에서 비용을 줄일 수 있음
- id 같은 값은 Unique한지 확인하기 위해 자주 사용되므로 id는 자주 사용하는 편

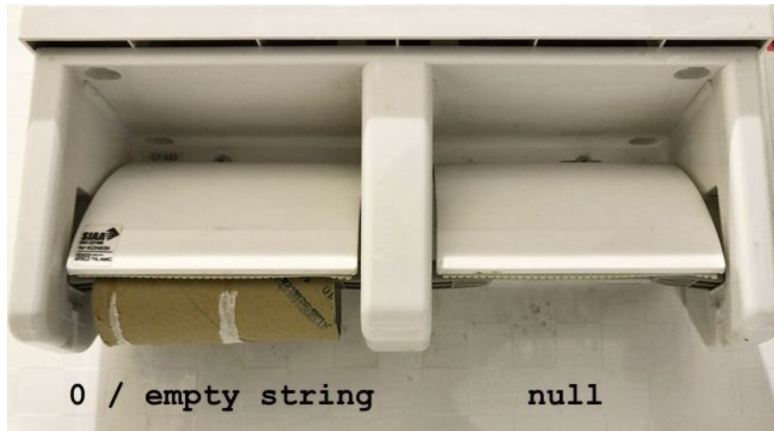
4) 컬럼은 모두 다 선택해야 할까?

- 컬럼 선택은 데이터를 추출해서 무엇을 하고자?하냐에 따라 다름
- JOIN이 잘 되었나 확인하기 위해 처음엔 많은 컬럼을 선택해도 괜찮으나,
사용하지 않을 컬럼은 선택하지 않는 것이 BigQuery에서 비용을 줄일 수 있음
- id 같은 값은 Unique한지 확인하기 위해 자주 사용되므로 id는 자주 사용하는 편

```
SELECT  
  table_a.*,  
  table_b.*  
FROM table_a  
LEFT JOIN table_b  
ON table_a.key = table_b.key
```


5) NULL이 대체 뭐죠?

- NULL : 값이 없음, 알 수 없음
- 0이나 공백과 다르게 값이 아예 없는 것
- JOIN에선 연결할 값이 없는 경우 나타남



이미지 출처 :

<https://twitter.com/raysato/status/833650208642719744>

JOIN 연습 문제

포켓몬으로 SQL JOIN 풀어보기

1. 트레이너가 보유한 포켓몬들은 얼마나 있는지 알 수 있는 쿼리를 작성해주세요

참고

- 보유했다는 정의는 status가 Active, Training인 경우를 의미
- Released는 방출했다는 것을 의미

예상 결과

pokemon_name	pokemon_cnt
피카츄	N
파이리	N
라이츄	N

포켓몬으로 SQL JOIN 풀어보기

2. 각 트레이너가 가진 포켓몬 중에서 'Grass' 타입의 포켓몬 수를 계산해주세요
(단, 편의를 위해 type1 기준으로 계산해주세요)

포켓몬으로 SQL JOIN 풀어보기

3. 트레이너의 고향(hometown)과 포켓몬을 포획한 위치(location)를 비교하여, 자신의 고향에서 포켓몬을 포획한 트레이너의 수를 계산해주세요.

참고

- status 상관없이 구해주세요

포켓몬으로 SQL JOIN 풀어보기

4. Master 등급인 트레이너들은 어떤 타입의 포켓몬을 제일 많이 보유하고 있을까요?

참고

- 보유했다는 정의는 1번 문제의 정의와 동일

포켓몬으로 SQL JOIN 풀어보기

5. Incheon 출신 트레이너들은 1세대, 2세대 포켓몬을 각각 얼마나 보유하고 있나요?

JOIN 정리

JOIN 정리

- JOIN : 여러 TABLE을 연결해야 할 때 사용하는 문법
- Key : 공통적으로 가지고 있는 컬럼

문법 예시

```
SELECT  
  t1.col, t2.col2  
FROM table1 AS t1  
LEFT JOIN table2 AS t2  
ON t1.key = t2.key
```

JOIN 종류

- INNER
- LEFT/RIGHT
- FULL
- CROSS

다양한 SQL JOIN 방법

KEY	
1	가
2	나
3	다

Table A

X

KEY	
1	A
2	B
4	C

Table B

INNER JOIN

1	가	A
2	나	B

LEFT JOIN

1	가	A
2	나	B
3	다	NULL

RIGHT JOIN

1	가	A
2	나	B
4	NULL	C

FULL JOIN

1	가	A
2	나	B
3	다	NULL
4	NULL	C

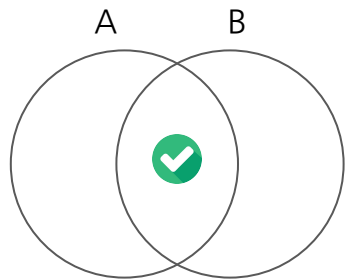
CROSS JOIN

1	가	1	A
1	가	2	B
1	가	4	C
2	나	1	A
2	나	2	B
2	나	4	C
3	다	1	A
3	다	2	B
3	다	4	C

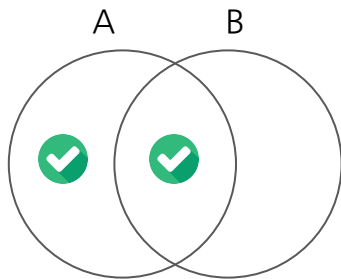


JOIN 집합 관점으로 생각하기

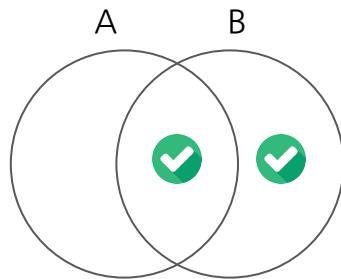
Inner Join



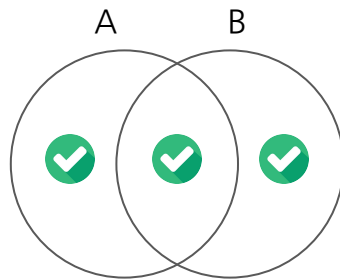
LEFT Join



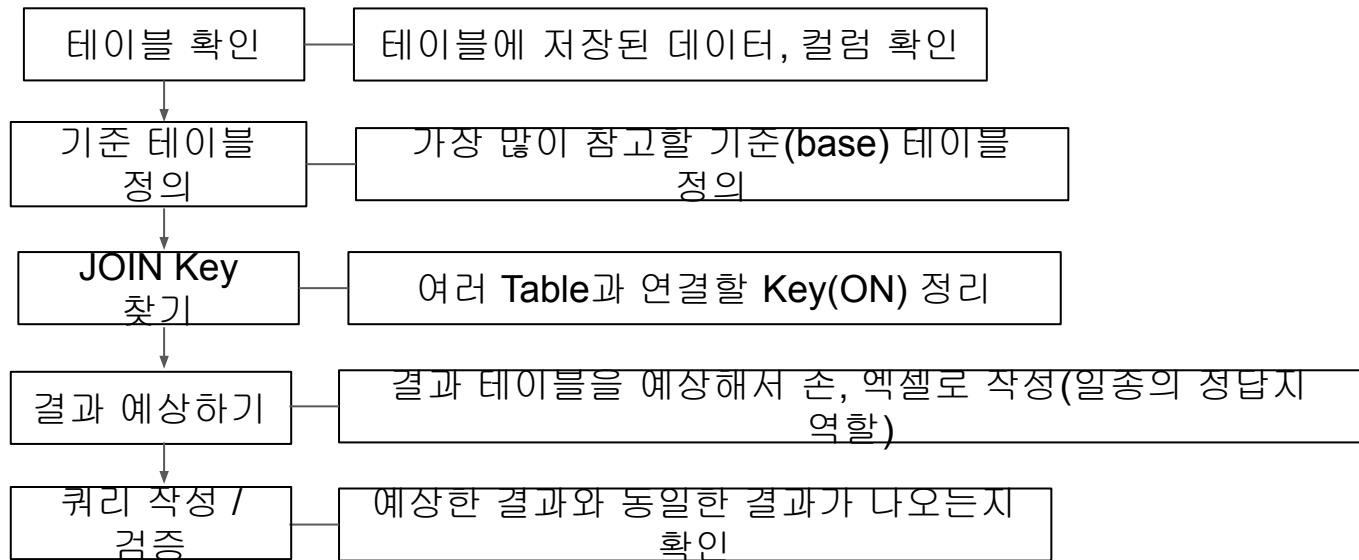
RIGHT Join



FULL Join



SQL JOIN 쿼리 작성하는 흐름



여러 JOIN 별 쿼리 예시

	ON 필수 여부	쿼리 예시
INNER JOIN	O	<pre>SELECT col FROM table_a AS A INNER JOIN table_b AS B ON A.key = B.key</pre>
LEFT/RIGHT JOIN	O	<pre>SELECT col FROM table_a AS A LEFT/RIGHT JOIN table_b AS B ON A.key = B.key</pre>
FULL JOIN	O	<pre>SELECT col FROM table_a AS A FULL JOIN table_b AS B ON A.key = B.key</pre>
CROSS JOIN	X	<pre>SELECT col FROM table_a AS A CROSS JOIN table_b AS B</pre>

#6. 데이터 결과 검증

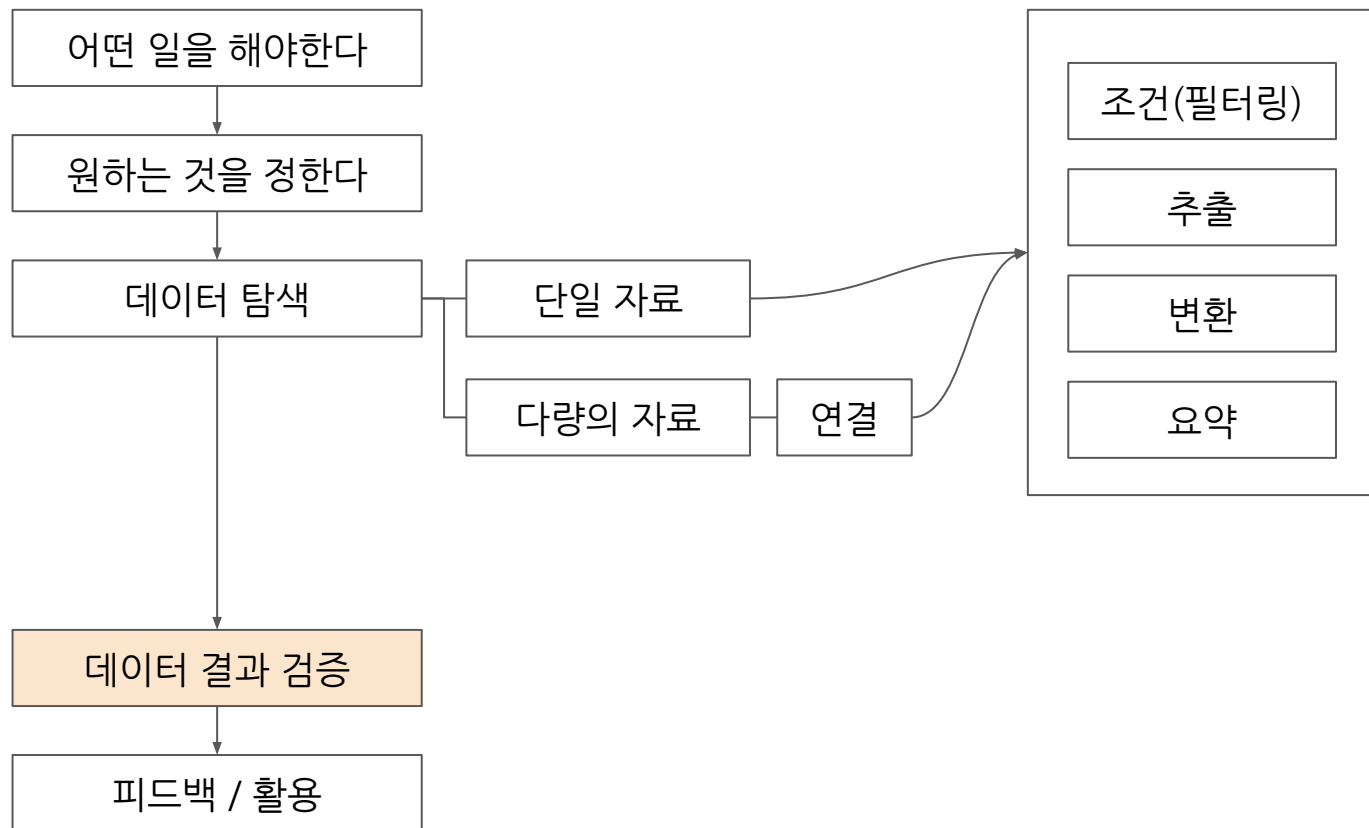
카일스쿨



이번 파트에서 다룰 내용

- 가독성을 챙기기 위한 SQL 스타일 가이드
- 데이터 결과 검증
- 데이터 결과 검증 예시

데이터를 활용하는 과정



가독성을 챙기기 위한 SQL 스타일 가이드

데이터 결과 검증을 하기 전에

실수는 언제 발생하는가?

- 문법을 잘못 알고 있는 경우
- 데이터를 파악하지 않고 쿼리를 작성하는 경우
- 쿼리가 복잡한 경우

데이터 결과 검증을 하기 전에

다른 사람의 쿼리를 봐야하는 경우 또는 내 쿼리를 다른 사람이 보는 경우

- 쿼리를 가독성 있게 잘 작성했으면 별도의 설명이 없어도 이해할 가능성이 존재
- 매번 설명을 해야하면, 이 쿼리를 보는 사람들이 많아질수록 내 시간을 쓰게 됨
- 쿼리를 변경해야 할 경우에 특정 부분만 바꿨는지, 전체를 바꿨는지도 파악하는 것이 쉬우면 좋음

가독성을 챙기기 위한 SQL 스타일 가이드

SQL 스타일 가이드

- <https://www.sqlstyle.guide/>

Mozilla(Firefox)의 SQL 스타일 가이드 

- https://docs.telemetry.mozilla.org/concepts/sql_style.html

말 그대로 “가이드”이기 때문에
꼭 지켜야 하는 것은 아님

조직마다 다른 가이드를 사용할 수 있음

1. 예약어는 대문자로 작성

SQL에서 문법적인 용도로 사용하고 있는 문자들은 대문자로 작성
예약어의 대표적인 예시 : SELECT, FROM, WHERE, 각종 함수

```
SELECT  
  col  
FROM table  
WHERE
```

2. 컬럼 이름은 snake_case로 작성

컬럼 이름은 CamelCase가 아닌 **snake_case**로 작성
(단, 회사의 기준이 CamelCase면 사용. 일관성이 중요)

```
SELECT  
  col1 AS event_status  
FROM table
```

3. 명시적 vs 암시적인 이름

Alias로 별칭을 지을 때는 명시적인 이름을 적용

AS a, AS b 등 컬럼의 의미를 한번 더 생각하게 하는 이름이 아닌
명시적인 것을 사용

JOIN 할 때 테이블의 이름도 명시적으로 할 수 있다면 명시적으로 진행하기

AS를 생략해서 별칭을 설정할 수도 있는데, AS를 쓰는 것도 명시적인 표현

4. 왼쪽 정렬

기본적으로 왼쪽 정렬을 기준으로 작성

```
SELECT  
  col  
FROM table  
WHERE 1=1
```

Not Good

```
SELECT col  
  FROM table  
WHERE 1=1
```


5. 예약어나 컬럼은 한 줄에 하나씩 권장

컬럼은 바로 주석처리할 수 있는 장점이 있기에 한 줄에 하나씩 작성

```
SELECT  
  col1,  
  col2,  
  col3  
FROM table  
WHERE
```

Not Good

```
SELECT col, col2, col3  
FROM table
```

6. 심표는 컬럼 바로 뒤에

의견이 갈리는 부분. 심표 앞 vs 뒤

BigQuery는 마지막 심표를 무시해서 뒤에 작성해도 무방

```
SELECT
  col1,
  col2,
  col3, # BigQuery는 정상 동작
FROM table
```

vs

```
SELECT
  col1
  , col2
  , col3
FROM table
```

WITH 구문

SQL 쿼리를 작성하다 생기는 일

```
SELECT  
  col, col2, col3 ...  
FROM  
  Table
```

WITH 구문

SQL 쿼리를 작성하다 생기는 일

```
SELECT  
  col, col2  
FROM (  
  SELECT  
    col, col2, col3 ...  
  FROM  
    Table  
)
```

WITH 구문

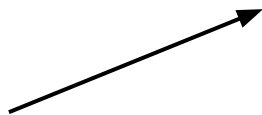
SQL 쿼리를 작성하다 생기는 일 => 점점 복잡해짐(가독성 하락)
- 만약 아래 쿼리가 다른 곳에서도 필요하다면 복사 붙여넣기

```
SELECT  
  col, col2  
FROM (  
  SELECT  
    col, col2, col3 ...  
  FROM  
    Table  
)
```

WITH 구문

WITH 문을 사용해 쿼리를 정의해서 재사용 가능

```
SELECT  
  col, col2  
FROM (  
  SELECT  
    col, col2, col3 ...  
  FROM  
    Table  
)
```



```
WITH temp_table AS (  
  SELECT  
    col, col2, col3 ...  
  FROM  
    Table  
)
```

```
SELECT  
  col, col2  
FROM temp_table
```

WITH 구문

- CTE(Common Table Expression)라고 표현
- SELECT 구문에 이름을 정해주는 것과 유사
- 쿼리 내에서 반복적으로 사용할 수 있음

WITH 구문 사용 방법

```
WITH name_a AS (  
  SELECT  
    col  
  FROM Table  
) , name_b AS (  
  SELECT  
    col2  
  FROM Table2  
)
```

```
SELECT  
  col  
FROM name_a
```


PARTITION

Table엔 Partition이란 것이 존재할 수 있음

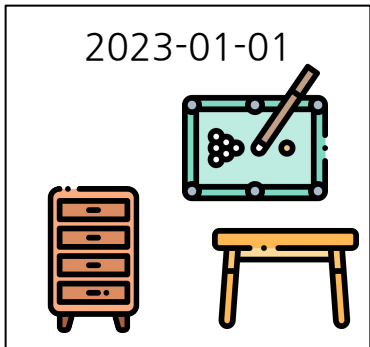
PARTITION

Table엔 Partition이란 것이 존재할 수 있음

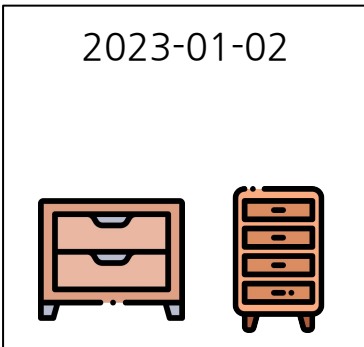
창고에 수많은 물건이 매일 많이 들어옴
특정 시기에 들어온 물건을 찾고 싶다면?
애초에 일자별로 정리하면 됨

물건 창고

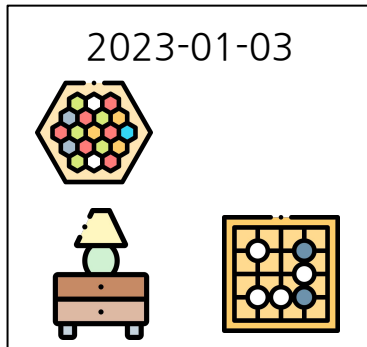
2023-01-01



2023-01-02



2023-01-03



PARTITION을 사용하면 좋은 점

- 1) 쿼리 성능 향상

- 전체 데이터를 스캔하는 것보다 파티션을 설정한 곳만 스캔하는 것이 더 빠름

- 2) 데이터 관리 용이성

- 특정 일자의 데이터를 모두 변경하거나 삭제해야 하면 파티션을 설정해서 삭제할 수 있음

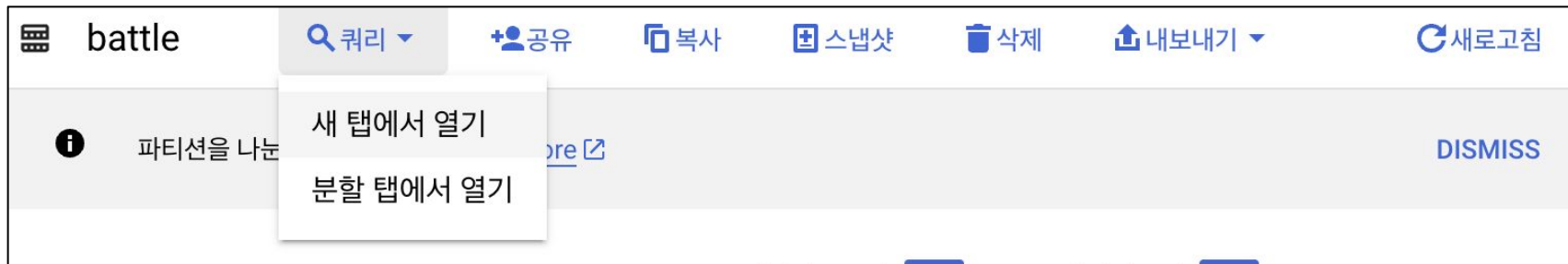
- 3) 비용

- 파티션에 해당되는 데이터만 스캔해서 비용을 줄일 수 있음
(BigQuery는 쿼리 용량에 비례해서 과금)

PARTITION Table : battle

battle 테이블은 battle_datetime이 파티션으로 설정된 테이블

battle 테이블을 클릭한 후, 새 탭에서 열기 클릭



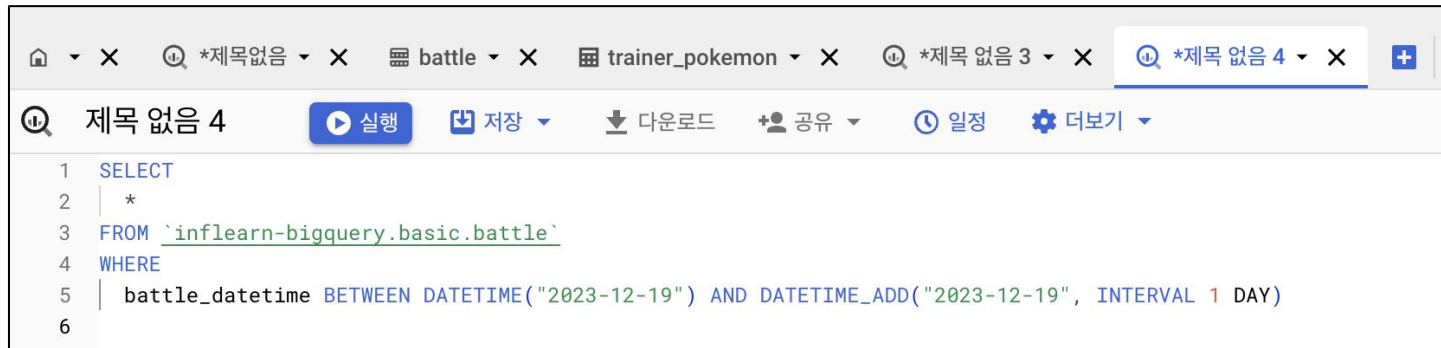
PARTITION Table : battle

다음과 같은 형태가 생성됨

이를 조금 더 보기 좋게 가공



```
1 SELECT FROM `inlearn-bigquery.basic.battle` WHERE battle_datetime BETWEEN DATETIME("2023-12-19") AND DATETIME_ADD("2023-12-19", INTERVAL 1 DAY) LIMIT 1000
```

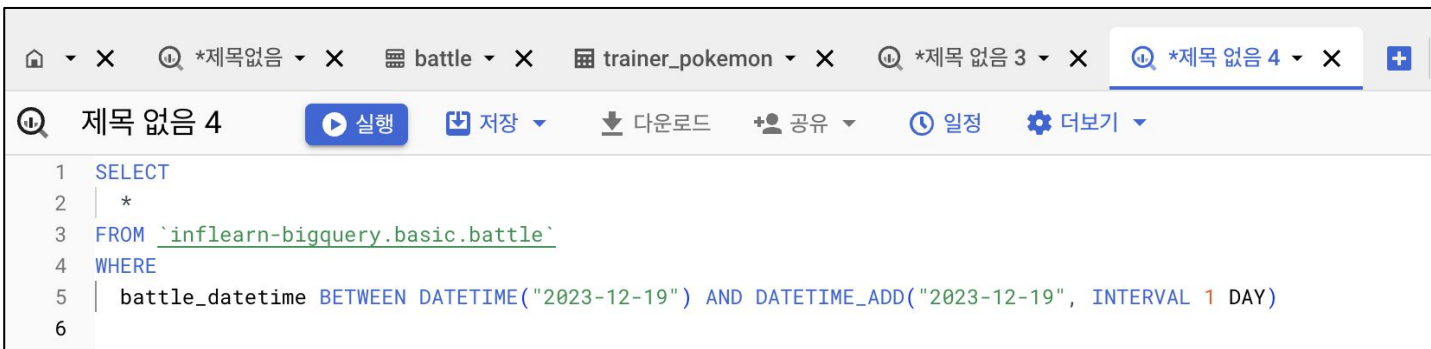


```
1 SELECT
2   *
3 FROM `inlearn-bigquery.basic.battle`
4 WHERE
5   battle_datetime BETWEEN DATETIME("2023-12-19") AND DATETIME_ADD("2023-12-19", INTERVAL 1 DAY)
6
```

PARTITION Table에서 쿼리하기

WHERE 절에 파티션 컬럼에 대해 조건을 설정해서 사용

아래 쿼리는 “2023-12-19”부터 하루 전날의 데이터를 추출하는 쿼리(저장된 데이터는 더 과거 데이터라 이 쿼리를 돌릴 때 결과는 나오지 않음)



The screenshot shows a BigQuery console interface. At the top, there are several tabs: a home icon, a search icon with text '*제목없음', a table icon with text 'battle', a table icon with text 'trainer_pokemon', a search icon with text '*제목없음 3', and a search icon with text '*제목없음 4' which is currently selected. Below the tabs is a toolbar with icons for search, execution (a blue play button), save, download, share, schedule, and settings. The main area displays a SQL query with line numbers 1 through 6 on the left. The query is as follows:

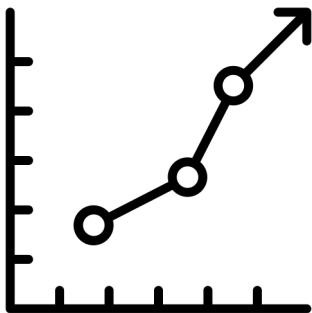
```
1 SELECT
2   *
3 FROM `infllearn-bigquery.basic.battle`
4 WHERE
5   battle_datetime BETWEEN DATETIME("2023-12-19") AND DATETIME_ADD("2023-12-19", INTERVAL 1 DAY)
6
```

데이터 결과 검증

카일스토리

데이터 직무라면 한번쯤 경험할 수 있는 이야기

- 갑작스럽게 데이터 추출을 해야하는 상황에 빠르게 데이터를 추출 후 공유



카일스토리

데이터 직무라면 한번쯤 경험할 수 있는 이야기

- 갑작스럽게 데이터 추출을 해야하는 상황에 빠르게 데이터를 추출 후 공유
- 데이터를 공유한 후, 혹시나 싶어 확인해보니.. 오류가 있었음
- 데이터를 다시 공유하면서 정정(이 때 심장 박동 높아져요..)



카일스토리

데이터 직무라면 한번쯤 경험할 수 있는 이야기

- 갑작스럽게 데이터 추출을 해야하는 상황에 빠르게 데이터를 추출 후 공유
 - 데이터를 공유한 후, 혹시나 싶어 확인해보니.. 오류가 있었음
 - 데이터를 다시 공유하면서 정정(이 때 심장 박동 높아져요..)
- => 데이터 결과 검증을 어떻게 할 수 있을까?에 대한 이야기
- SQL 강의에서 데이터 결과 검증에 대한 내용을 다루는 곳은 거의 없음



데이터 결과 검증을 잘하기 위한 마인드셋

데이터 결과 검증을 하지 못해서 실수할 수 있다

다만 그 실수가 반복되는 것은 문제

항상 그 실수 이후에 어떻게 해야 다시 반복되지 않을지 생각하는 것이 필요
(회고도 필수)

데이터 결과 검증(Data Result Validation)의 정의

SQL 쿼리 후 얻은 결과가 예상과 일치하는지 확인하는 과정

목적 : 분석 결과의 정확성, 신뢰성 확보



데이터 결과 검증(Data Result Validation)의 정의

SQL 쿼리 후 얻은 결과가 예상과 일치하는지 확인하는 과정

목적 : 분석 결과의 정확성, 신뢰성 확보

방법은 심플

- 내가 기대하는 예상 결과를 정의
- 쿼리 작성
- 두개가 일치하는지 비교



데이터 결과 검증(Data Result Validation)의 정의

SQL 쿼리 후 얻은 결과가 예상과 일치하는지 확인하는 과정

목적 : 분석 결과의 정확성, 신뢰성 확보

방법은 심플

- 내가 기대하는 예상 결과를 정의
- 쿼리 작성
- 두개가 일치하는지 비교

제일 중요한 부분

- 문제를 잘 정의하고, 미리 작성해보기
- 도메인 특수성(이런 규칙 등) 잘 파악하기
- SQL 쿼리 템플릿과 맥락이 유사



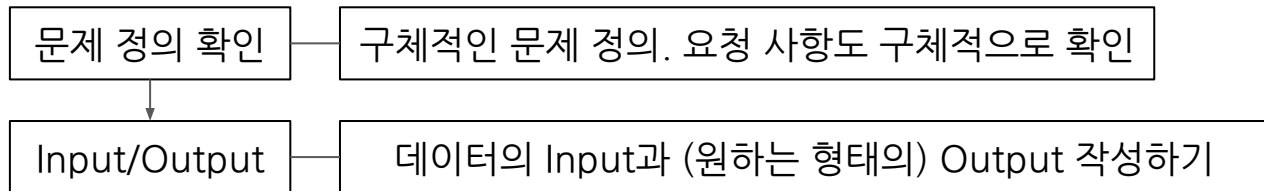
데이터 결과를 검증하는 흐름

문제 정의 확인

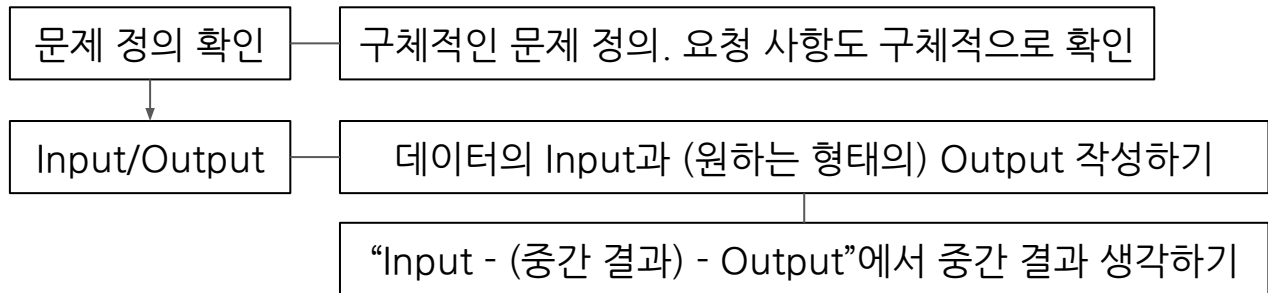
구체적인 문제 정의. 요청 사항도 구체적으로 확인



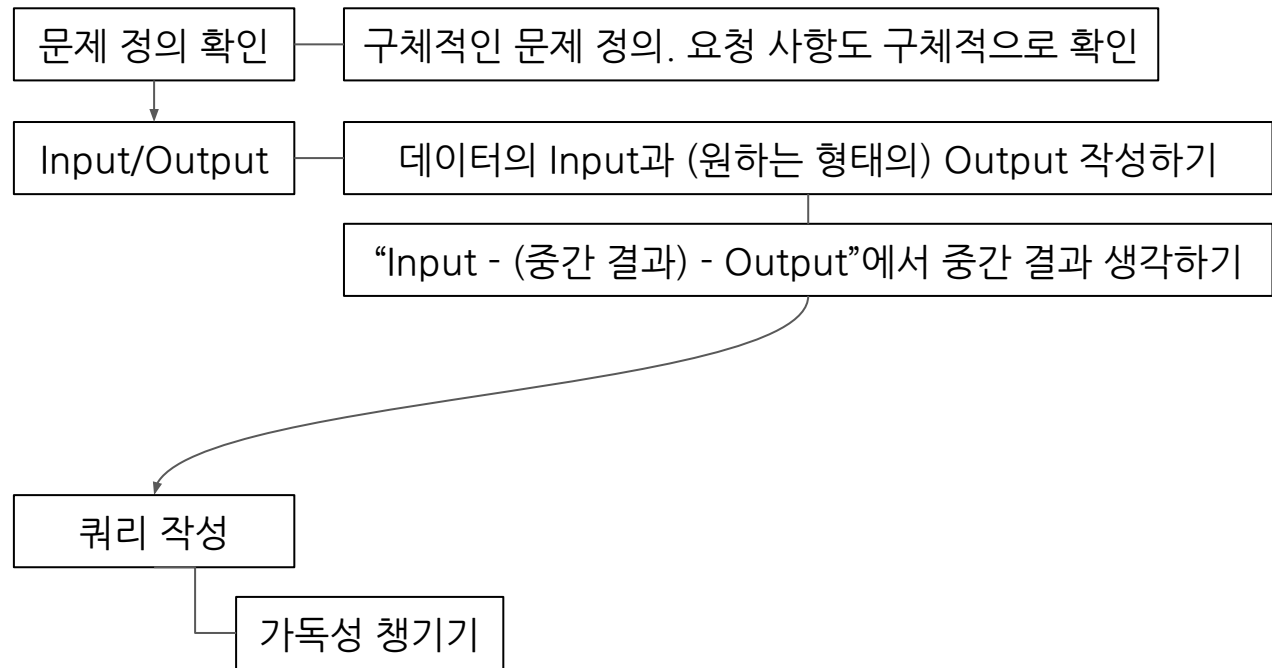
데이터 결과를 검증하는 흐름



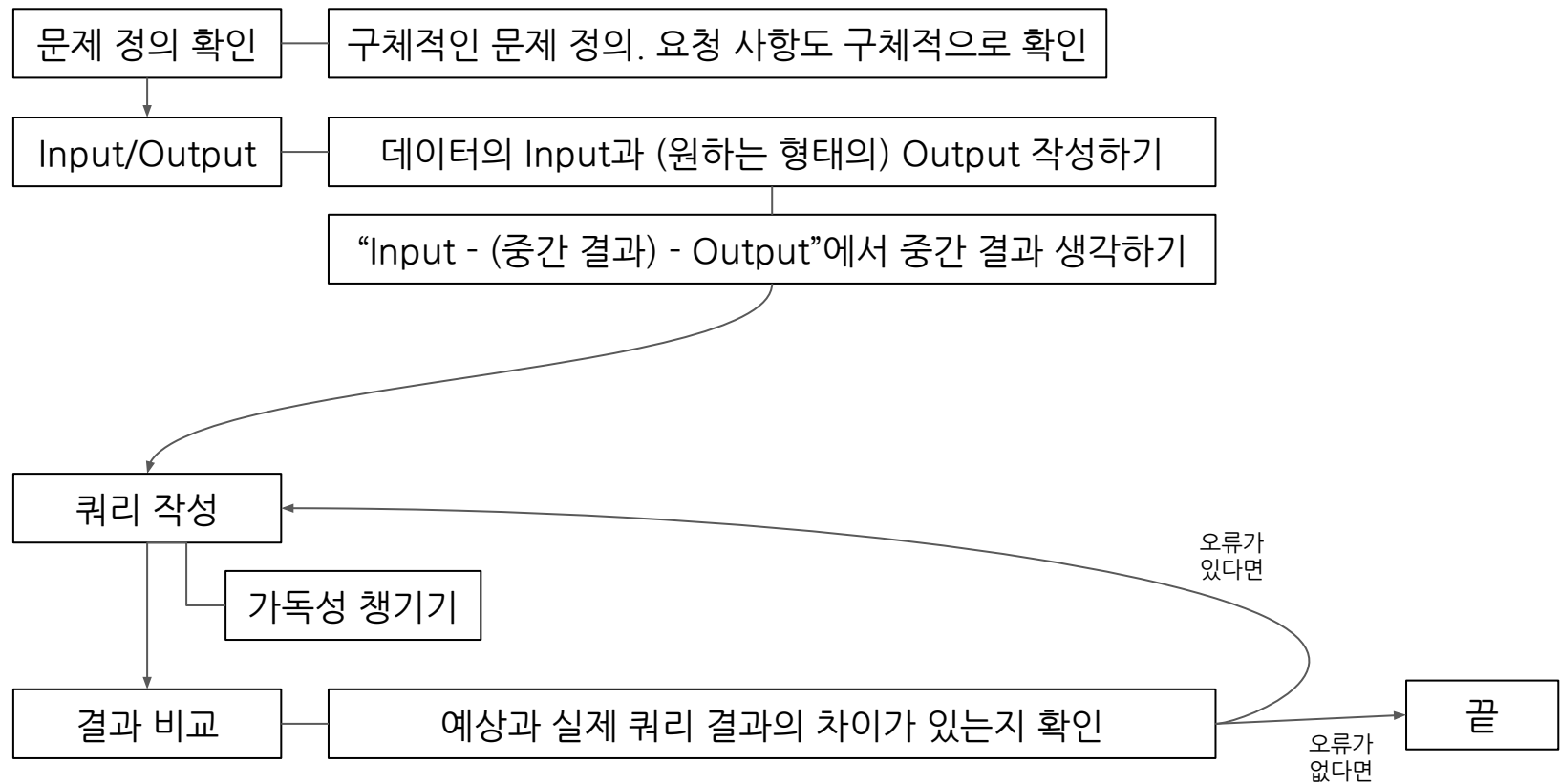
데이터 결과를 검증하는 흐름



데이터 결과를 검증하는 흐름



데이터 결과를 검증하는 흐름



데이터 결과 검증할 때 자주 활용하는 SQL 쿼리

대표적으로 활용하는 SQL 문법

- 1) COUNT(*) : 행 수를 확인. 의도한 데이터의 행 개수가 맞는가?
- 2) NOT NULL : 특정 컬럼에 NULL이 존재하는가? 필수 필드가 비어있지 않는가?
- 3) DISTINCT : 데이터의 고유값을 확인해 중복 여부 확인
- 4) IF문, CASE WHEN : 의도와 같다면 TRUE, 아니면 FALSE

1) + 3)

=> SELECT COUNT(DISTINCT col), COUNT(col) 두 컬럼을 보고 개수를 비교

제가 데이터 결과 검증을 할 때 활용하는 방식

1) 특정 user_id로 필터링을 걸어서 확인

- 1명의 데이터 확인(예 : WHERE user_id = 402)
- 결과를 예상할 때 Raw 데이터에서 하나씩 눈으로 세고 적어둠(예상 결과)
- 1명의 데이터의 예상 결과와 쿼리 결과가 동일한지 확인
- 다른 user_id 3~4건 더 추가해서 확인(여러 케이스가 존재할 수 있음)
- 3~4개에서 동일한 결과가 나오면 user_id 조건을 삭제

제가 데이터 결과 검증을 할 때 활용하는 방식

2) 샘플 데이터 생성하기

- WITH 문을 사용해 예시 데이터를 생성한 후, 결과를 예상하고 쿼리 작성
- 복잡한 데이터에서 하기 전에, 쿼리 자체가 올바른지 확인할 때 주로 사용

🔍 제목 없음 실행 쿼리 완료됨

```
1 WITH sample_data AS (  
2   SELECT 'shirt' AS product_type, 't-shirt' AS product_name, 3 AS product_count  
3   UNION ALL  
4   SELECT 'shirt', 't-shirt', 8  
5   UNION ALL  
6   SELECT NULL, 'polo', 25  
7   UNION ALL  
8   SELECT 'pants', 'jeans', 6  
9 )  
10 SELECT  
11 | *  
12 FROM sample_data
```

접근성 옵션을 보려면 Option+F1 키를 누르세요

쿼리 결과 결과 저장 데이터 탐색

작업 정보 결과 차트 미리보기 JSON 실행 세부정보 실행 그래프

행	product_type	product_name	product_count
1	shirt	t-shirt	3
2	shirt	t-shirt	8
3	null	polo	25
4	pants	jeans	6

데이터 결과 검증 예시

데이터 결과 검증 예시 문제

여러분은 포켓몬 트레이너들의 배틀 성적을 분석하는 작업을 맡게 되었습니다.
각 트레이너가 진행한 배틀의 승리 비율을 계산해야 하며, 배틀에 참여한 횟수가 9회 이상인 경우만 계산합니다

데이터 결과 검증 예시 문제

여러분은 포켓몬 트레이너들의 배틀 성적을 분석하는 작업을 맡게 되었습니다.
각 트레이너가 진행한 배틀의 승리 비율을 계산해야 하며, 배틀에 참여한 횟수가 9회 이상인 경우만 계산합니다

떠올릴 수 있는 생각

- 승리 비율(= 승리한 횟수 / 총 배틀 횟수)을 바로 구하면 되겠다!

데이터 결과 검증 예시 문제

여러분은 포켓몬 트레이너들의 배틀 성적을 분석하는 작업을 맡게 되었습니다.
각 트레이너가 진행한 배틀의 승리 비율을 계산해야 하며, 배틀에 참여한 횟수가 9회 이상인 경우만 계산합니다

떠올릴 수 있는 생각

- 승리 비율(= 승리한 횟수 / 총 배틀 횟수)을 바로 구하면 되겠다!

=> 이 문제는 간단한 문제라서 이렇게 해도 틀릴 가능성이 적음

그러나 현업에서 쿼리를 작성할 때는 복잡할 때도 있으니 데이터 결과 검증을 시도하면서 풀어보기

데이터 결과 검증을 한다면 어떻게 해보시겠어요?
(멈추고 생각해보기!)

데이터 결과 검증 예시 문제

여러분은 포켓몬 트레이너들의 배틀 성적을 분석하는 작업을 맡게 되었습니다.
각 트레이너가 진행한 배틀의 승리 비율을 계산해야 하며, 배틀에 참여한 횟수가 9회 이상인 경우만 계산합니다

제가 생각한 데이터 결과 검증 프로세스 흐름

- 1) 전체 데이터 파악
- 2) 특정 user_id 선정
- 3) 승률 직접 COUNT : 결과 예상
- 4) 쿼리 작성
- 5) 실제와 비교
- 6) 맞다면 특정 유저 조건 제외

1) 전체 데이터 파악

파악할 테이블 : battle 테이블

battle								
쿼리 공유 복사 스냅샷 삭제 내보내기								
파티션을 나눈 테이블입니다. Learn more								
스키마 세부정보 미리보기 계보 데이터 프로파일 신규 데이터 품질 신규								
행	id	player1_id	player2_id	winner_id	battle_date	battle_datetime	battle_timestamp	
1	16	10	12	10	2023-07-04	2023-07-04T01:33:24	2023-07-03 16:33:24 UTC	
2	17	13	15	13	2023-07-04	2023-07-04T13:28:15	2023-07-04 04:28:15 UTC	
3	18	14	16	16	2023-07-04	2023-07-04T01:49:49	2023-07-03 16:49:49 UTC	
4	19	17	19	19	2023-07-04	2023-07-04T18:48:24	2023-07-04 09:48:24 UTC	
5	20	18	20	20	2023-07-04	2023-07-04T23:36:38	2023-07-04 14:36:38 UTC	
6	21	1	4	null	2023-07-04	2023-07-04T21:44:00	2023-07-04 12:44:00 UTC	
7	22	2	3	2	2023-07-04	2023-07-04T10:38:49	2023-07-04 01:38:49 UTC	
8	85	5	14	null	2023-07-15	2023-07-15T09:53:55	2023-07-15 00:53:55 UTC	
9	88	8	17	null	2023-07-15	2023-07-15T03:37:02	2023-07-14 18:37:02 UTC	
10	84	4	13	4	2023-07-15	2023-07-15T04:31:41	2023-07-14 19:31:41 UTC	
11	86	6	15	6	2023-07-15	2023-07-15T01:55:11	2023-07-14 16:55:11 UTC	
12	87	7	16	7	2023-07-15	2023-07-15T00:31:36	2023-07-14 15:31:36 UTC	
13	89	9	18	9	2023-07-15	2023-07-15T01:01:21	2023-07-14 16:01:21 UTC	
14	90	10	19	10	2023-07-15	2023-07-15T22:43:51	2023-07-15 13:43:51 UTC	
15	91	11	20	11	2023-07-15	2023-07-15T02:40:30	2023-07-14 17:40:30 UTC	
16	77	17	5	5	2023-07-14	2023-07-14T17:53:10	2023-07-14 08:53:10 UTC	
17	78	18	6	6	2023-07-14	2023-07-14T07:43:44	2023-07-13 22:43:44 UTC	
18	79	19	7	null	2023-07-14	2023-07-14T15:09:34	2023-07-14 06:09:34 UTC	

2) 특정 user_id 선정

특정 user_id(battle에선 playerN_id)를 선정
회사에선 자신의 user_id를 사용하지만, 여기선 임의로 선택. 7

```
SELECT
*
FROM `basic.battle`
WHERE
player1_id = 7
```

쿼리 결과

작업 정보 <u>결과</u> 차트 미리보기 JSON 실행 세부정보 실행 그래프								
행	id	player1_id	player2_id	winner_id	battle_date	battle_datetime	battle_timestamp	
1	47	7	14	7	2023-07-09	2023-07-09T00:36:49	2023-07-08 15:36:49 UTC	
2	4	7	8	8	2023-07-01	2023-07-01T18:17:40	2023-07-01 09:17:40 UTC	
3	87	7	16	7	2023-07-15	2023-07-15T00:31:36	2023-07-14 15:31:36 UTC	
4	67	7	15	7	2023-07-12	2023-07-12T10:34:14	2023-07-12 01:34:14 UTC	

2) 특정 user_id 선정

앞의 쿼리는 틀릴 수 있음. 테이블 다시 확인해보기

battle								
쿼리 공유 복사 스냅샷 삭제 내보내기								
파티션을 나눈 테이블입니다. Learn more								
스키마 세부정보 미리보기 계보 데이터 프로필 신규 데이터 품질 신규								
행	id	player1_id	player2_id	winner_id	battle_date	battle_datetime	battle_timestamp	
1	16	10	12	10	2023-07-04	2023-07-04T01:33:24	2023-07-03 16:33:24 UTC	
2	17	13	15	13	2023-07-04	2023-07-04T13:28:15	2023-07-04 04:28:15 UTC	
3	18	14	16	16	2023-07-04	2023-07-04T01:49:49	2023-07-03 16:49:49 UTC	
4	19	17	19	19	2023-07-04	2023-07-04T18:48:24	2023-07-04 09:48:24 UTC	
5	20	18	20	20	2023-07-04	2023-07-04T23:36:38	2023-07-04 14:36:38 UTC	
6	21	1	4	null	2023-07-04	2023-07-04T21:44:00	2023-07-04 12:44:00 UTC	
7	22	2	3	2	2023-07-04	2023-07-04T10:38:49	2023-07-04 01:38:49 UTC	
8	85	5	14	null	2023-07-15	2023-07-15T09:53:55	2023-07-15 00:53:55 UTC	
9	88	8	17	null	2023-07-15	2023-07-15T03:37:02	2023-07-14 18:37:02 UTC	
10	84	4	13	4	2023-07-15	2023-07-15T04:31:41	2023-07-14 19:31:41 UTC	
11	86	6	15	6	2023-07-15	2023-07-15T01:55:11	2023-07-14 16:55:11 UTC	
12	87	7	16	7	2023-07-15	2023-07-15T00:31:36	2023-07-14 15:31:36 UTC	
13	89	9	18	9	2023-07-15	2023-07-15T01:01:21	2023-07-14 16:01:21 UTC	
14	90	10	19	10	2023-07-15	2023-07-15T22:43:51	2023-07-15 13:43:51 UTC	
15	91	11	20	11	2023-07-15	2023-07-15T02:40:30	2023-07-14 17:40:30 UTC	
16	77	17	5	5	2023-07-14	2023-07-14T17:53:10	2023-07-14 08:53:10 UTC	
17	78	18	6	6	2023-07-14	2023-07-14T07:43:44	2023-07-13 22:43:44 UTC	
18	79	19	7	null	2023-07-14	2023-07-14T15:09:34	2023-07-14 06:09:34 UTC	

2) 특정 user_id 선정

특정 user_id(battle에선 playerN_id)를 선정
회사에선 자신의 user_id를 사용하지만, 여기선 임의로 선택. 7

SELECT

*

FROM `basic.battle`

WHERE

(player1_id = 7) OR (player2_id = 7)

작업 정보		결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	id	player1_id	player2_id	winner_id	battle_date	battle_datetime	battle_timestamp
1	33	3	7	3	2023-07-06	2023-07-06T17:10:03	2023-07-06 08:10:03 UTC
2	24	6	7	6	2023-07-05	2023-07-05T10:31:49	2023-07-05 01:31:49 UTC
3	47	7	14	7	2023-07-09	2023-07-09T00:36:49	2023-07-08 15:36:49 UTC
4	4	7	8	8	2023-07-01	2023-07-01T18:17:40	2023-07-01 09:17:40 UTC
5	60	20	7	7	2023-07-11	2023-07-11T14:25:05	2023-07-11 05:25:05 UTC
6	67	7	15	7	2023-07-12	2023-07-12T10:34:14	2023-07-12 01:34:14 UTC
7	87	7	16	7	2023-07-15	2023-07-15T00:31:36	2023-07-14 15:31:36 UTC
8	13	5	7	7	2023-07-03	2023-07-03T22:22:24	2023-07-03 13:22:24 UTC
9	79	19	7	null	2023-07-14	2023-07-14T15:09:34	2023-07-14 06:09:34 UTC

3) 승률 직접 COUNT : 결과 예상

player_id(player1_id, player2_id)가 7인 유저의 승률

총 9회 중 5번 = $5/9 = 0.555556$

작업 정보		결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	id	player1_id	player2_id	winner_id	battle_date	battle_datetime	battle_timestamp
1	33	3	7	3	2023-07-06	2023-07-06T17:10:03	2023-07-06 08:10:03 UTC
2	24	6	7	6	2023-07-05	2023-07-05T10:31:49	2023-07-05 01:31:49 UTC
3	47	7	14	7	2023-07-09	2023-07-09T00:36:49	2023-07-08 15:36:49 UTC
4	4	7	8	8	2023-07-01	2023-07-01T18:17:40	2023-07-01 09:17:40 UTC
5	60	20	7	7	2023-07-11	2023-07-11T14:25:05	2023-07-11 05:25:05 UTC
6	67	7	15	7	2023-07-12	2023-07-12T10:34:14	2023-07-12 01:34:14 UTC
7	87	7	16	7	2023-07-15	2023-07-15T00:31:36	2023-07-14 15:31:36 UTC
8	13	5	7	7	2023-07-03	2023-07-03T22:22:24	2023-07-03 13:22:24 UTC
9	79	19	7	null	2023-07-14	2023-07-14T15:09:34	2023-07-14 06:09:34 UTC

4) 쿼리 작성

- 트레이너가 승리한 비율 구하기
 - 트레이너가 참여한 배틀의 수 구하기
 - 트레이너가 승리한 배틀의 수 구하기
- 두개 조합해서 승리한 비율 구할 수 있음
- 단, 이 쿼리를 어떻게 해야할까? 쉽게 가능할까?
- 배틀의 수가 9 이상만 추출
- (예상 정답) player_id가 7의 승리한 비율은 0.55556이 나와야 함

4) 쿼리 작성

통합 데이터 생성(player1, 2 구분을 하지 않아도 되는 테이블 => trainer_id 생성)

```
SELECT
*
FROM (
  SELECT
    id AS battle_id,
    player1_id AS trainer_id,
    winner_id
  FROM `basic.battle`
  UNION ALL
  SELECT
    id AS battle_id,
    player2_id AS trainer_id,
    winner_id
  FROM `basic.battle`
)
ORDER BY battle_id
```

작업 정보	결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	battle_id ▼	trainer_id ▼	winner_id ▼			
1	1	2	1			
2	1	1	1			
3	2	4	null			
4	2	3	null			
5	3	6	6			
6	3	5	6			
7	4	8	8			
8	4	7	8			
9	5	10	10			
10	5	9	10			

4) 쿼리 작성

trainer_id =7의 총 배틀 횟수를 구하는 쿼리

```
WITH battle_basic AS (  
  SELECT  
    id AS battle_id,  
    player1_id AS trainer_id,  
    winner_id  
  FROM `basic.battle`  
  UNION ALL  
  SELECT  
    id AS battle_id,  
    player2_id AS trainer_id,  
    winner_id  
  FROM `basic.battle`  
)  
  
SELECT  
  trainer_id,  
  COUNT(*) AS total_battles,  
  COUNT(DISTINCT battle_id) AS unique_battles  
FROM battle_basic  
WHERE trainer_id = 7  
GROUP BY  
  trainer_id
```

쿼리 결과

작업 정보

결과

차트

미리보기

JSON

실행 세부정보

실행 그래프

행	trainer_id ▼	total_battles ▼	unique_battles ▼	
1	7	9	9	

4) 쿼리 작성

trainer_id와 winner_id를 조합해서 WIN, LOSE, DRAW를 만들어보면 어떨까?

```
WITH battle_basic AS (  
  SELECT  
    id AS battle_id,  
    player1_id AS trainer_id,  
    winner_id  
  FROM `basic.battle`  
  UNION ALL  
  SELECT  
    id AS battle_id,  
    player2_id AS trainer_id,  
    winner_id  
  FROM `basic.battle`  
)  
  
SELECT  
  *,  
  CASE  
    WHEN trainer_id = winner_id THEN "WIN"  
    WHEN winner_id IS NULL THEN "DRAW"  
    ELSE "LOSE"  
  END AS battle_result  
FROM battle_basic  
WHERE trainer_id = 7
```

작업 정보	결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	battle_id ▼	trainer_id ▼	winner_id ▼	battle_result ▼		
1	60	7	7	WIN		
2	33	7	3	LOSE		
3	24	7	6	LOSE		
4	13	7	7	WIN		
5	79	7	<i>null</i>	DRAW		
6	4	7	8	LOSE		
7	67	7	7	WIN		
8	47	7	7	WIN		
9	87	7	7	WIN		

4) 쿼리 작성, 5) 실제와 비교

COUNTIF를 사용해 값 구하기

```
WITH battle_basic AS (  
  SELECT  
    ... # 길어져서 생략  
  FROM `basic.battle`  
) , battle_with_result AS (  
  SELECT  
    *,  
    CASE  
      WHEN trainer_id = winner_id THEN "WIN"  
      WHEN winner_id IS NULL THEN "DRAW"  
      ELSE "LOSE"  
    END AS battle_result  
  FROM battle_basic  
  WHERE trainer_id = 7  
)  
  
SELECT  
  trainer_id,  
  COUNTIF(battle_result = "WIN") AS win_count,  
  COUNT(battle_id) AS total_battle_count,  
  COUNTIF(battle_result = "WIN")/COUNT(DISTINCT battle_id) AS win_ratio  
FROM battle_with_result  
GROUP BY  
  trainer_id
```

쿼리 결과

작업 정보

결과

차트

미리보기

JSON

실행 세부정보

실행 그래프

행	trainer_id ▾	win_count ▾	total_battle_count ▾	win_ratio ▾
1	7	5	9	0.555555555555...

6) 맞다면 특정 유저 조건 제외

WHERE의 trainer_id 제거 후, 총 배틀 횟수가 9 이상만 조건

```
WITH battle_basic AS (
  SELECT
    ... # 길어져서 생략
  FROM `basic.battle`
), battle_with_result AS (
  SELECT
    *,
    CASE
      WHEN trainer_id = winner_id THEN "WIN"
      WHEN winner_id IS NULL THEN "DRAW"
      ELSE "LOSE"
    END AS battle_result
  FROM battle_basic
  # WHERE trainer_id = 7 주석 처리
)

SELECT
  trainer_id,
  COUNTIF(battle_result = "WIN") AS win_count,
  COUNT(battle_id) AS total_battle_count,
  COUNTIF(battle_result = "WIN")/COUNT(DISTINCT battle_id) AS win_ratio
FROM battle_with_result
GROUP BY
  trainer_id
HAVING
  total_battle_count >= 9
```

작업 정보	결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	trainer_id	win_count	total_battle_count	win_ratio		
5	15	2	10	0.2		
6	16	4	10	0.4		
7	18	2	9	0.222222222222...		
8	2	6	11	0.545454545454...		
9	3	3	10	0.3		
10	11	5	10	0.5		
11	17	1	10	0.1		
12	1	9	11	0.818181818181...		
13	4	2	10	0.2		
14	8	1	9	0.111111111111...		
15	5	5	10	0.5		
16	7	5	9	0.555555555555...		
17	12	6	10	0.6		
18	13	2	10	0.2		

6) 맞다면 특정 유저 조건 제외

trainer_id = 7인 경우엔 동일
나머지 trainer_id도 확인 후에 쿼리 확정

작업 정보		결과	차트	미리보기	JSON	실행 세부정보	실행 그래프
행	trainer_id	win_count	total_battle_count	win_ratio			
5	15	2	10	0.2			
6	16	4	10	0.4			
7	18	2	9	0.222222222222...			
8	2	6	11	0.545454545454...			
9	3	3	10	0.3			
10	11	5	10	0.5			
11	17	1	10	0.1			
12	1	9	11	0.818181818181...			
13	4	2	10	0.2			
14	8	1	9	0.111111111111...			
15	5	5	10	0.5			
16	7	5	9	0.555555555555...			
17	12	6	10	0.6			
18	13	2	10	0.2			

이 예시에서 예상한 결과

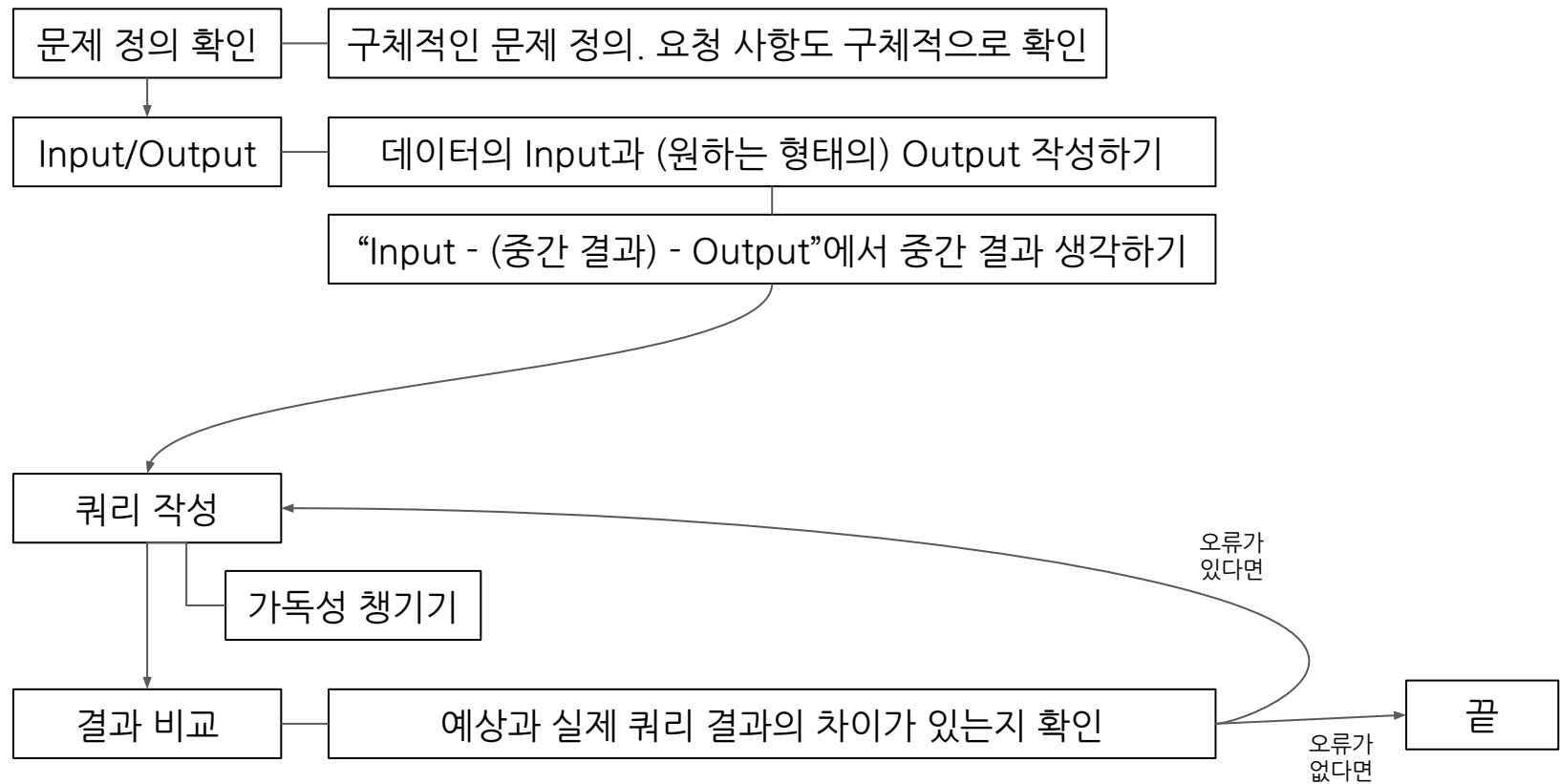
trainer_id = 7일 때 배틀의 횟수 : 9

승리한 횟수 : 5

승리한 비율 : 0.55555...

데이터 결과 검증 정리

데이터 결과를 검증하는 흐름



데이터 결과 검증 정리

대표적으로 활용하는 SQL 문법

- 1) COUNT(*) : 행 수를 확인. 의도한 데이터의 행 개수가 맞는가?
- 2) NOT NULL : 특정 컬럼에 NULL이 존재하는가? 필수 필드가 비어있지 않는가?
- 3) DISTINCT : 데이터의 고유값을 확인해 중복 여부 확인
- 4) IF문, CASE WHEN : 의도와 같다면 TRUE, 아니면 FALSE

예시 문제에서 제가 접근한 방식

- 1) 전체 데이터 파악
- 2) 특정 user_id 선정
- 3) 승률 직접 COUNT : 결과 예상
- 4) 쿼리 작성
- 5) 실제와 비교
- 6) 맞다면 특정 유저 조건 제외

#7. BigQuery 입문 연습 문제

카일스쿨



연습 문제

1. 각 트레이너별로 가진 포켓몬의 평균 레벨을 계산하고, 그 중 평균 레벨이 높은 TOP 3 트레이너의 이름과 보유한 포켓몬의 수, 평균 레벨을 출력해주세요.

연습 문제

2. 각 포켓몬 타입1을 기준으로 가장 많이 포획된(방출 여부 상관없음) 포켓몬의 타입1, 포켓몬의 이름과 포획 횟수를 출력해주세요.

연습 문제

3. 전설의 포켓몬을 보유한 트레이너들은 전설의 포켓몬과 일반 포켓몬을 얼마나 보유하고 있을까요? (트레이너의 이름을 같이 출력해주세요)

연습 문제

4. 가장 승리가 많은 트레이너 ID, 트레이너의 이름, 승리한 횟수, 보유한 포켓몬의 수, 평균 포켓몬의 레벨을 출력해주세요. 단, 포켓몬의 레벨은 소수점 2째 자리에서 반올림해주세요

참고

- 반올림 함수 : ROUND

연습 문제

5. 트레이너가 잡았던 포켓몬의 총 공격력(attack)과 방어력(defense)의 합을 계산하고, 이 합이 가장 높은 트레이너를 찾으세요.

연습 문제

6. 각 포켓몬의 최고 레벨과 최저 레벨을 계산하고, 레벨 차이가 가장 큰 포켓몬의 이름을 출력하세요.

연습 문제

7. 각 트레이너가 가진 포켓몬 중에서 공격력(attack)이 100 이상인 포켓몬과 100 미만인 포켓몬의 수를 각각 계산해주세요. 트레이너의 이름과 두 조건에 해당하는 포켓몬의 수를 출력해주세요

#8. 앞으로

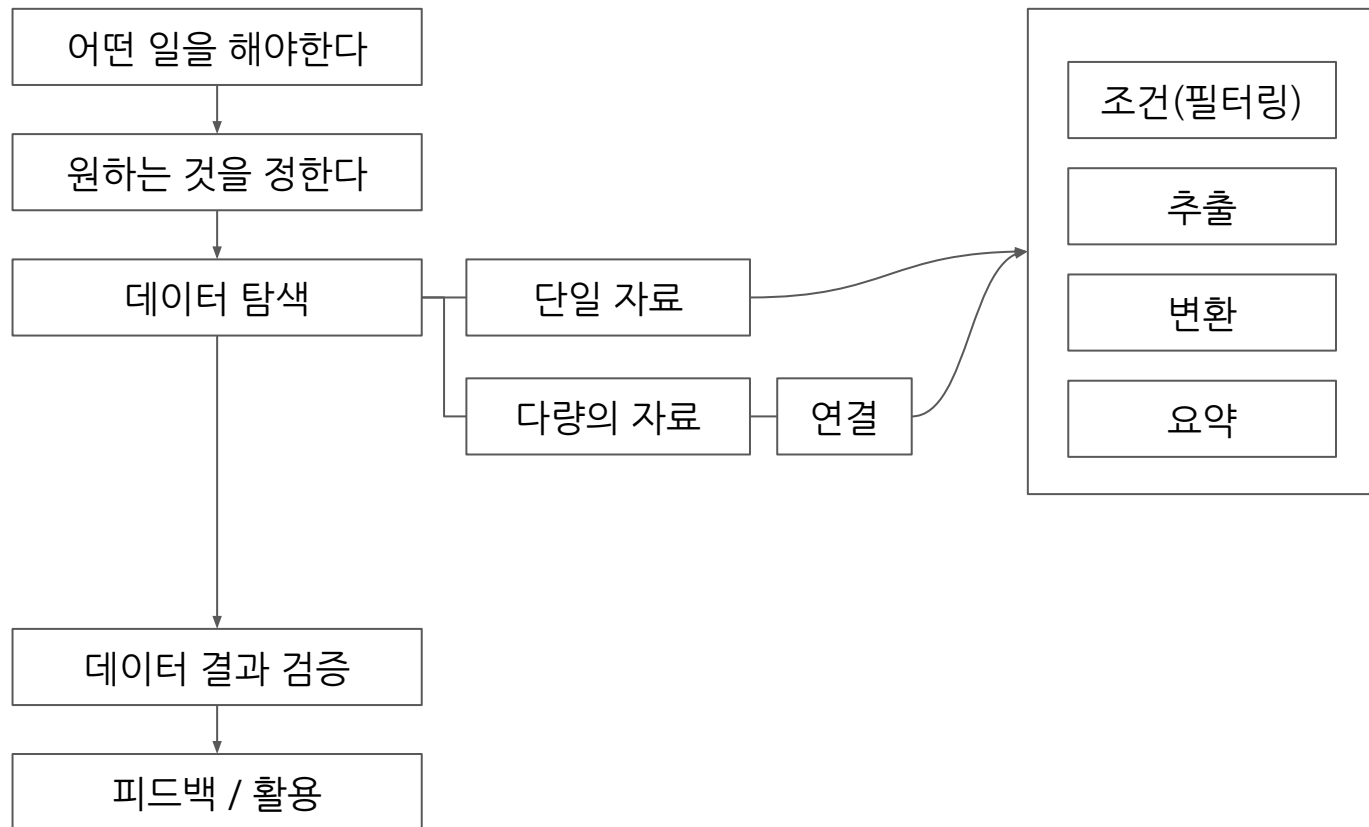
카일스쿨



초보자를 위한 BigQuery(SQL) 입문 총 정리

- 데이터 활용 과정
- 데이터 탐색 : 조건과 추출
- 데이터 탐색 : 요약, 집계
- SQL 쿼리 작성하는 흐름
 - 쿼리 작성 템플릿, 생산성 도구
 - 오류를 디버깅하는 방법
- 데이터 탐색 : 변환
 - 데이터 타입과 타입 별 함수
 - BigQuery 공식 문서 확인하는 방법
- 다양한 자료를 연결 : JOIN
- 데이터 결과 검증
 - 가독성을 챙기기 위한 SQL 스타일 가이드

BigQuery(입문) 총 정리



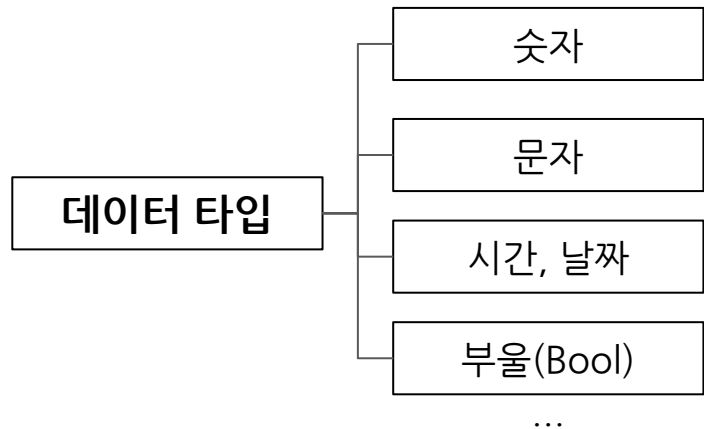
BigQuery Query 문 구조

```
WITH CTE AS (  
  SELECT  
    col,  
    col2,  
  FROM Table  
)
```

```
SELECT  
  a.col3,  
  b.col4,  
  COUNT(DISTINCT a.id) AS cnt  
FROM table_a AS a  
LEFT JOIN table_b AS b  
ON a.id = b.id  
WHERE  
  a.col3 >= 3  
GROUP BY  
  a.col3,  
  b.col4  
HAVING  
  cnt >= 2  
ORDER BY cnt DESC  
LIMIT 10
```



데이터 타입



쿼리 작성 템플릿

쿼리를 작성하는 목표, 확인할 지표 :

쿼리 계산 방법 :

데이터의 기간 :

사용할 테이블 :

Join KEY :

데이터 특징 :

SELECT

FROM

WHERE

이렇게 글로 작성하면 쿼리 작성이 더 수월

다양한 SQL JOIN 방법

KEY

1	가
2	나
3	다

Table A

X

KEY

1	A
2	B
4	C

Table B

INNER JOIN

1	가	A
2	나	B

LEFT JOIN

1	가	A
2	나	B
3	다	NULL

RIGHT JOIN

1	가	A
2	나	B
4	NULL	C

FULL JOIN

1	가	A
2	나	B
3	다	NULL
4	NULL	C

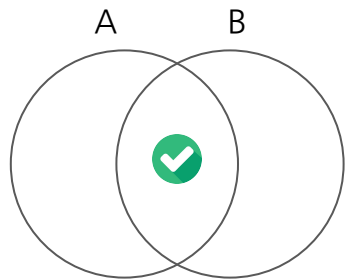
CROSS JOIN

1	가	1	A
1	가	2	B
1	가	4	C
2	나	1	A
2	나	2	B
2	나	4	C
3	다	1	A
3	다	2	B
3	다	4	C

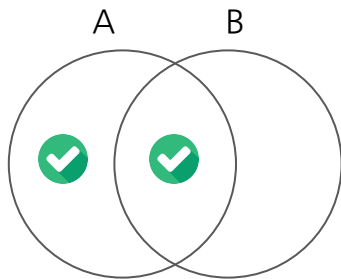


JOIN 집합 관점으로 생각하기

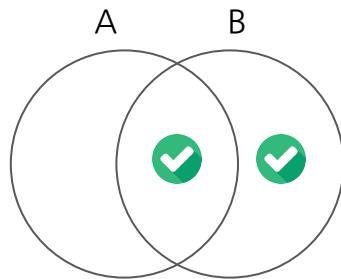
Inner Join



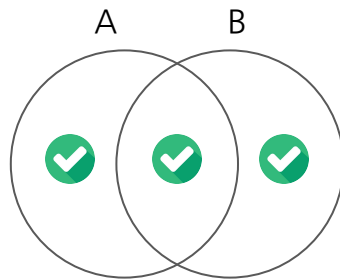
LEFT Join



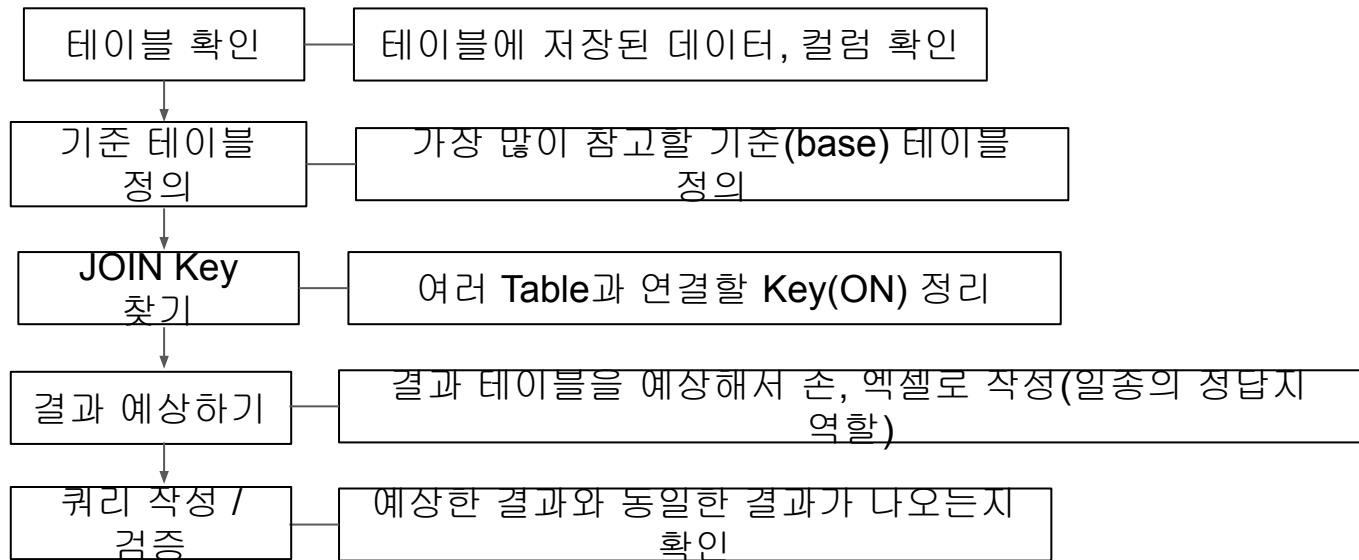
RIGHT Join



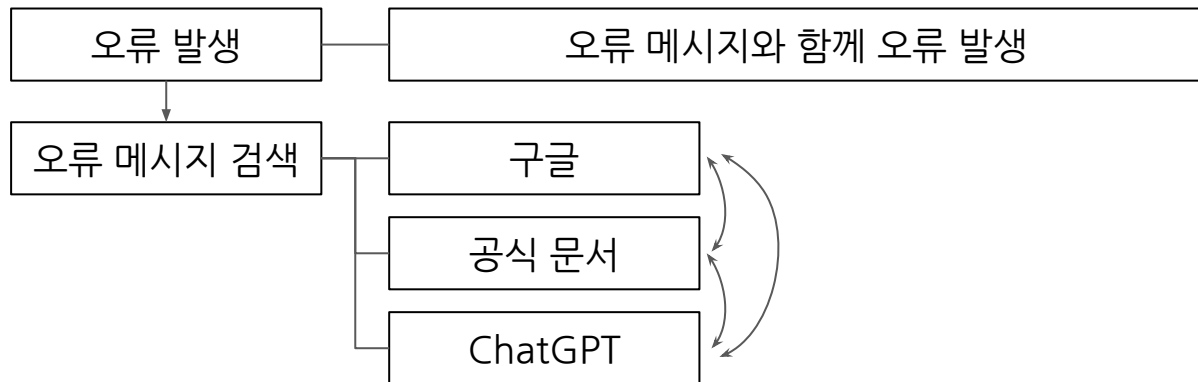
FULL Join



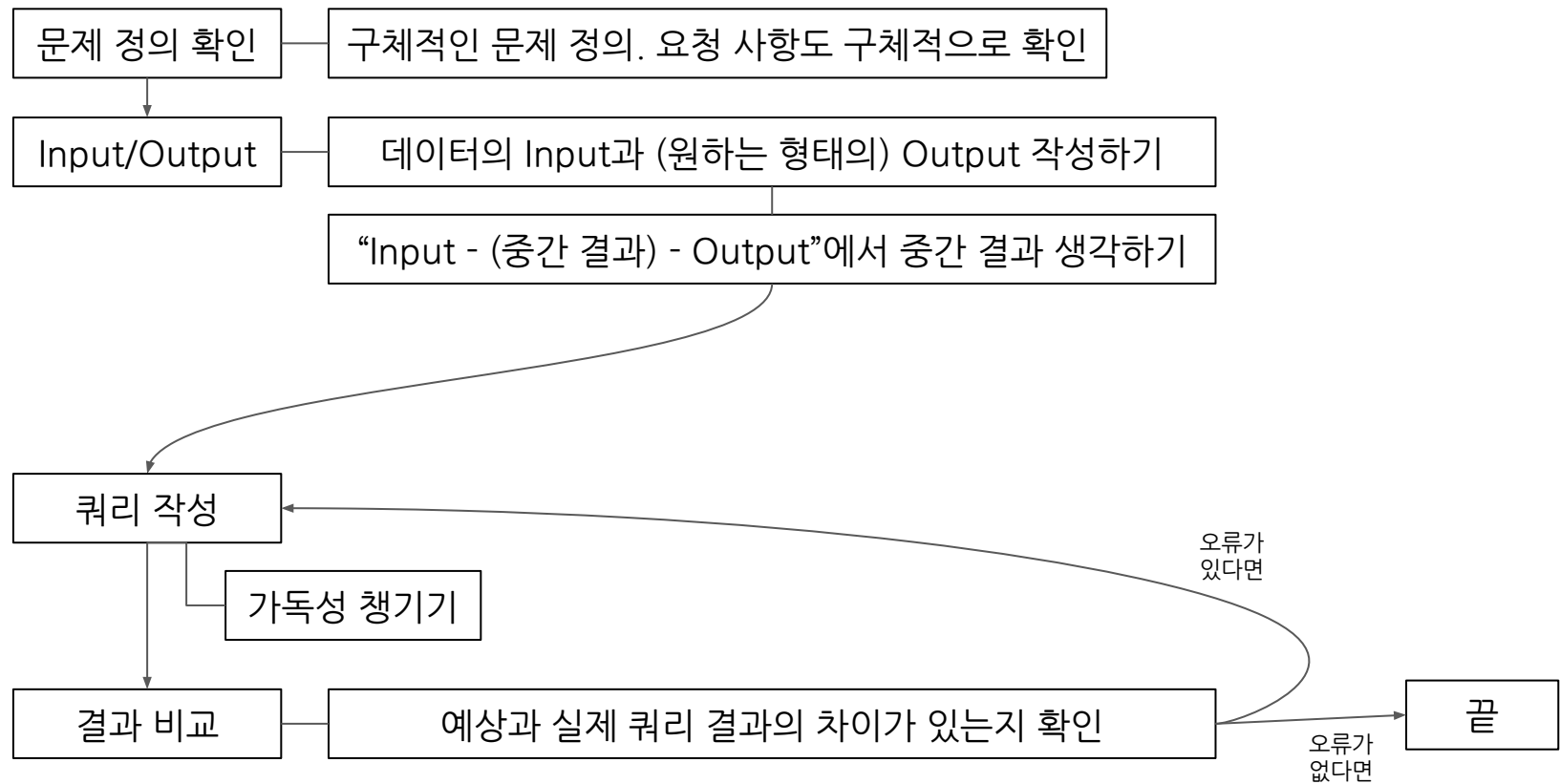
SQL JOIN 쿼리 작성하는 흐름



오류 메시지 해결의 핵심



데이터 결과를 검증하는 흐름



앞으로 배우면 좋은 내용

- 복잡한 자료형 다루기 : ARRAY, STRUCT
- 복잡한 데이터 가공하기
- BigQuery 결과를 스프레드시트, BI에 연동하기
- 유용한 변환 함수(윈도우 함수 등)
- 유용한 기능(스크립팅 등)

공식 문서를 토대로 학습하시는 것도 추천 :)

앞으로 배우면 좋은 내용

- 바로 준비하는 강의 : BigQuery 활용편. 회사에서 실제로 활용하는 과정에 대해 다룰 예정
- Google Analytics, Firebase, DB 데이터를 활용해 데이터 분석하는 흐름을 제공 예정

문제 정의, Product Analytics 이론이 궁금하신 경우

=> PM을 위한 데이터 리터러시 강의 (<https://inf.run/jfWT>)

SQL을 교육해야 하는 분들을 위한 가이드

- SQL 학습을 <기본> - <적용>으로 나눌 때, 중요한 부분은 <적용>
- 이 파트는 회사 내에서 진행해야 함
 - 데이터 분석가가 <기본> 내용을 가르치지 않고, <적용>에 중점적으로 진행하는 것을 추천

SQL을 교육해야 하는 분들을 위한 가이드

- 1) 사내에서 SQL을 학습하고 싶은 분들에게 “어떤 목적으로 SQL을 공부하고 싶은가?”를 설문 받기
- 2) SQL을 학습할 때 <기본> 부분은 이 강의로 함께 수강
 - 혼자 학습하지 않고, 사람들과 같이 수강
- 3) 회사의 데이터를 잘 이해할 수 있는 환경을 마련한 후(데이터 디스커버리, 데이터 스키마 정리 등) 매 파트에서 회사의 데이터를 활용한 문제를 숙제로 내기
- 4) 오피스아워를 운영해 문제에 대한 풀이 제공
- 5) 어느정도 익숙해졌다면, 1)에서 나온 목적 중 난이도 별로 문제 제공
 - 실제 업무에 적용할 수 있도록 문제를 출제
 - 혹은 회사에서 애드혹 요청이 왔던 내용을 같이 풀어볼 수도 있음
- 6) 풀이에서 사용한 쿼리는 회사 문서함에 모아서 정리하기
- 7) 점점 문제를 늘리면서 문제집 형태로 제공
- 8) 사내 문제에 대한 피드백을 수집해서 학습 자료 개선

마치며

BigQuery(SQL)은 어려운 프로그래밍 언어가 아니며, 많은 분들이 활용 중

해외에선 기획, 영업 직무에서도 BigQuery를 많이 활용하며, 한국에서도 데이터 문화가 발전한 회사에선 스스로 데이터를 추출

여러분들의 앞으로 커리어 발전에 도움이 되길 바랄게요 :)

후기를 남겨주세요!

후기를 남겨주시면 앞으로 어떤 강의를 만들지 방향성을 잡을 때 기준을 삼고, 이후 강의가 나올 때 메일 드릴게요

<https://bit.ly/infllearn-bigquery-feedback>

